

jsc::bounded_vector<>

contiguous container for immovable elements

Mateusz Zych



conference

code::dive

About Me

About Me



Visual and Parallel Computing Group

OpenGL & Vulkan graphics drivers for iGPUs

About Me



Visual and Parallel Computing Group

OpenGL & Vulkan graphics drivers for iGPUs



Radeon Technologies Group

AMDVLK: Vulkan graphics driver for Radeon GPUs

About Me



Visual and Parallel Computing Group

OpenGL & Vulkan graphics drivers for iGPUs



Radeon Technologies Group

AMDVLK: Vulkan graphics driver for Radeon GPUs



Jülich Supercomputing Centre

Eventify: event-based fine-grained tasking system

FMSolvr: Fast Multipole Method library for N-body problem

introduction

introduction

- *For the last 30 years `std::vector<>` has been the default choice for a sequence container in C++.*

introduction

- *For the last 30 years `std::vector<>` has been the default choice for a sequence container in C++.*
- *The goal of this talk is to show an alternative design, suitable for real-time systems and GPU programming.*

introduction

- *For the last 30 years `std::vector<>` has been the default choice for a sequence container in C++.*
- *The goal of this talk is to show an alternative design, suitable for real-time systems and GPU programming.*
- *To justify the design choices of the `jsc::bounded_vector<>`*

introduction

- *For the last 30 years `std::vector<>` has been the default choice for a sequence container in C++.*
- *The goal of this talk is to show an alternative design, suitable for real-time systems and GPU programming.*
- *To justify the design choices of the `jsc::bounded_vector<>`*
 - *the original motivating use case will be explored, and*

introduction

- *For the last 30 years `std::vector<>` has been the default choice for a sequence container in C++.*
- *The goal of this talk is to show an alternative design, suitable for real-time systems and GPU programming.*
- *To justify the design choices of the `jsc::bounded_vector<>`*
 - *the original motivating use case will be explored, and*
 - *the internals of the C++ standard library will be reviewed, with an emphasis on containers, allocators, iterators and ranges.*

introduction

- *However, before diving into technical details, the `jsc::bounded_vector<>` will be previewed, providing a glimpse of the design towards which this talk will be building.*

The image features a dramatic sky scene. The upper half is a deep, clear blue sky, while the lower half is filled with a dense, undulating layer of white, puffy clouds that resemble a sea of cotton. The word "preview" is centered in the middle of the image, written in a white, elegant script font.

preview

create from size and generator

create from size and generator



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    return jsc::bounded_vector
    {
        size, [index = std::ptrdiff_t { 0 }] mutable
        {
            return std::latch { index++ };
        }
    };
}
```

create_vector(long):

```
sd     zero,0(a0)
sd     zero,8(a0)
sd     zero,16(a0)
bne    a1,zero,.L14
ret
```

.L14:

```
addi   sp,sp,-32
li     a5,-1
sd     ra,24(sp)
srli   a5,a5,3
bgt    a1,a5,.L15
sd     a0,8(sp)
slli   a0,a1,2
sd     a1,0(sp)
call   operator new(unsigned long)@plt
ld     a2,8(sp)
ld     a1,0(sp)
li     a5,0
ld     a4,8(a2)
sd     a1,16(a2)
sd     a0,0(a2)
```

.L4:

```
slli   a3,a4,2
```

create from size and generator



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    return jsc::bounded_vector
    {
        size, [index = std::ptrdiff_t { 0 }] mutable
        {
            return std::latch { index++ };
        }
    };
}
```

create_vector(long):

```
sd     zero,0(a0)
sd     zero,8(a0)
sd     zero,16(a0)
bne    a1,zero,.L14
ret
```

.L14:

```
addi   sp,sp,-32
li     a5,-1
sd     ra,24(sp)
srli   a5,a5,3
bgt    a1,a5,.L15
sd     a0,8(sp)
slli   a0,a1,2
sd     a1,0(sp)
call   operator new(unsigned long)@plt
ld     a2,8(sp)
ld     a1,0(sp)
li     a5,0
ld     a4,8(a2)
sd     a1,16(a2)
sd     a0,0(a2)
```

.L4:

```
slli   a3,a4,2
```


create from size and generator



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    return jsc::bounded_vector
    {
        size, [index = std::ptrdiff_t { 0 }] mutable
        {
            return std::latch { index++ };
        }
    };
}
```

```
sd     zero,8(a0)
sd     zero,16(a0)
bne    a1,zero,.L14
ret

.L14:
addi   sp,sp,-32
li     a5,-1
sd     ra,24(sp)
srli   a5,a5,3
bgt    a1,a5,.L15
sd     a0,8(sp)
slli   a0,a1,2
sd     a1,0(sp)
call   operator new(unsigned long)@plt
ld     a2,8(sp)
ld     a1,0(sp)
li     a5,0
ld     a4,8(a2)
sd     a1,16(a2)
sd     a0,0(a2)

.L4:
slli   a3,a4,2
add    a3,a0,a3
sw     a5,0(a3)
addi   a4,a4,1
```

create from size and generator



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    return jsc::bounded_vector
    {
        size, [index = std::ptrdiff_t { 0 }] mutable
        {
            return std::latch { index++ };
        }
    };
}
```

```
ld    a2,8(sp)
ld    a1,0(sp)
li    a5,0
ld    a4,8(a2)
sd    a1,16(a2)
sd    a0,0(a2)

.L4:
slli  a3,a4,2
add   a3,a0,a3
sw    a5,0(a3)
addi  a4,a4,1
sd    a4,8(a2)
addi  a5,a5,1
bne   a1,a5,.L4
ld    ra,24(sp)
mv    a0,a2
addi  sp,sp,32
jr    ra

.L15:
li    a0,8
call  __cxa_allocate_exception@plt
la    a5,_ZTVSt9bad_alloc
addi  a5,a5,16
sd    a5,0(a0)
la    a2,_ZNSt9bad_allocD1Ev
```

create from size and generator



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    return jsc::bounded_vector
    {
        size, [index = std::ptrdiff_t { 0 }] mutable
        {
            return std::latch { index++ };
        }
    };
}
```

```
ld    a4,8(a2)
sd    a1,16(a2)
sd    a0,0(a2)

.L4:
slli  a3,a4,2
add   a3,a0,a3
sw    a5,0(a3)
addi  a4,a4,1
sd    a4,8(a2)
addi  a5,a5,1
bne   a1,a5,.L4
ld    ra,24(sp)
mv    a0,a2
addi  sp,sp,32
jr    ra

.L15:
li    a0,8
call  __cxa_allocate_exception@plt
la    a5,_ZTVSt9bad_alloc
addi  a5,a5,16
sd    a5,0(a0)
la    a2,_ZNSt9bad_allocD1Ev
la    a1,_ZTISt9bad_alloc
call  __cxa_throw@plt
```

create from size and generator

 COMPILER EXPLORER

RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3



```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    auto vector = std::vector<std::latch> { };

    vector.reserve(std::size_t(size));

    for (const auto index : std::views::indices(size))
    {
        vector.emplace_back(index);
    }

    return vector;
}
```

create from size and generator



The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left. To its right are three buttons: 'RISC-V 64-bit', 'GCC 16.1.0', and a text input field containing '-std=c++26 -O3'. A red close button is on the far right of the header. Below the header is a large text area containing C++ code. The code defines a function 'create_vector' that takes a 'std::ptrdiff_t size' and returns a 'std::vector<std::latch>'. Inside the function, a vector is created and reserved, then a loop uses 'std::views::indices' to populate the vector with 'std::latch' objects. The code is as follows:

```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    auto vector = std::vector<std::latch> { };

    vector.reserve(std::size_t(size));

    for (const auto index : std::views::indices(size))
    {
        vector.emplace_back(index);
    }

    return vector;
}
```

- Unfortunately, `std::vector<>` does not provide the size-and-generator constructor.

create from size and generator



The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left. To its right are three tabs: 'RISC-V 64-bit', 'GCC 16.1.0', and '-std=c++26 -O3'. A red close button is on the far right of the header. Below the header is a large text area containing C++ code. The code defines a function 'create_vector' that takes a 'std::ptrdiff_t size' and returns a 'std::vector<std::latch>'. Inside the function, a vector is created and reserved, then elements of 'std::latch' are added using 'emplace_back' in a loop from 0 to 'size-1'.

```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    auto vector = std::vector<std::latch> { };

    vector.reserve(std::size_t(size));

    for (const auto index : std::views::indices(size))
    {
        vector.emplace_back(index);
    }

    return vector;
}
```

- *Unfortunately,*
`std::vector<>` *does not provide the size-and-generator constructor.*
- *Critically,*
growing a `std::vector<>` *requires MoveConstructible elements.*

create from size and generator



The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left. To its right are three tabs: 'RISC-V 64-bit', 'GCC 16.1.0', and '-std=c++26 -O3'. A red close button is on the far right of the header. Below the header is a large text area containing C++ code. The code defines a function 'create_vector' that takes a 'std::ptrdiff_t size' and returns a 'std::vector<std::latch>'. Inside the function, a vector is created and reserved, then elements of 'std::latch' are emplaced back into it for the specified size. Finally, the vector is returned.

```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    auto vector = std::vector<std::latch> { };

    vector.reserve(std::size_t(size));

    for (const auto index : std::views::indices(size))
    {
        vector.emplace_back(index);
    }

    return vector;
}
```

- *Unfortunately,*
`std::vector<>` *does not provide the size-and-generator constructor.*
- *Critically,*
growing a `std::vector<>` *requires MoveConstructible elements.*
- *Of course,*
`std::latch` *is immovable – after all it's a synchronization primitive.*

create from materializable range

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return jsc::bounded_vector
    {
        std::views::indices(size) |
        std::views::transform(create_latch)
    };
}
```

```
create_vector(long):
    sd     zero,0(a0)
    sd     zero,8(a0)
    sd     zero,16(a0)
    bne    a1,zero,.L14
    ret

.L14:
    addi   sp,sp,-32
    li     a5,-1
    sd     ra,24(sp)
    srli   a5,a5,3
    bgt    a1,a5,.L15
    sd     a0,8(sp)
    slli   a0,a1,2
    sd     a1,0(sp)
    call   operator new(unsigned long)@plt
    ld     a3,8(sp)
    ld     a1,0(sp)
    ld     a4,8(a3)
    sd     a0,0(a3)
    sd     a1,16(a3)
    slli   a5,a4,2
    add    a0,a0,a5
    li     a5,0
```

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return jsc::bounded_vector
    {
        std::views::indices(size) |
        std::views::transform(create_latch)
    };
}
```

create_vector(long):

```
sd     zero,0(a0)
sd     zero,8(a0)
sd     zero,16(a0)
bne    a1,zero,.L14
ret
```

.L14:

```
addi   sp,sp,-32
li     a5,-1
sd     ra,24(sp)
srli   a5,a5,3
bgt    a1,a5,.L15
sd     a0,8(sp)
slli   a0,a1,2
sd     a1,0(sp)
call   operator new(unsigned long)@plt
ld     a3,8(sp)
ld     a1,0(sp)
ld     a4,8(a3)
sd     a0,0(a3)
sd     a1,16(a3)
slli   a5,a4,2
add    a0,a0,a5
li     a5,0
```

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return jsc::bounded_vector
    {
        std::views::indices(size) |
        std::views::transform(create_latch)
    };
}
```

```
sd      zero,16(a0)
bne     a1,zero,.L14
ret

.L14:
addi    sp,sp,-32
li      a5,-1
sd      ra,24(sp)
srli    a5,a5,3
bgt     a1,a5,.L15
sd      a0,8(sp)
slli    a0,a1,2
sd      a1,0(sp)
call    operator new(unsigned long)@plt
ld      a3,8(sp)
ld      a1,0(sp)
ld      a4,8(a3)
sd      a0,0(a3)
sd      a1,16(a3)
slli    a5,a4,2
add     a0,a0,a5
li      a5,0

.L4:
addi    a4,a4,1
sw      a5,0(a0)
sd      a4,8(a3)
```

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return jsc::bounded_vector
    {
        std::views::indices(size) |
        std::views::transform(create_latch)
    };
}
```

```
ld    a1,0(sp)
ld    a4,8(a3)
sd    a0,0(a3)
sd    a1,16(a3)
slli  a5,a4,2
add   a0,a0,a5
li    a5,0

.L4:
addi  a4,a4,1
sw    a5,0(a0)
sd    a4,8(a3)
addi  a5,a5,1
addi  a0,a0,4
bne   a1,a5,.L4
ld    ra,24(sp)
mv    a0,a3
addi  sp,sp,32
jr    ra

.L15:
li    a0,8
call  __cxa_allocate_exception@plt
la    a5,_ZTVSt9bad_alloc
addi  a5,a5,16
sd    a5,0(a0)
la    a2,_ZNSt9bad_allocD1Ev
```

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
#include <jsc/bounded_vector.hpp>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return jsc::bounded_vector
    {
        std::views::indices(size) |
        std::views::transform(create_latch)
    };
}
```

```
sd    a1,16(a3)
slli  a5,a4,2
add   a0,a0,a5
li    a5,0

.L4:
addi  a4,a4,1
sw    a5,0(a0)
sd    a4,8(a3)
addi  a5,a5,1
addi  a0,a0,4
bne   a1,a5,.L4
ld    ra,24(sp)
mv    a0,a3
addi  sp,sp,32
jr    ra

.L15:
li    a0,8
call  __cxa_allocate_exception@plt
la    a5,_ZTVSt9bad_alloc
addi  a5,a5,16
sd    a5,0(a0)
la    a2,_ZNSt9bad_allocD1Ev
la    a1,_ZTISt9bad_alloc
call  __cxa_throw@plt
```

create from materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3

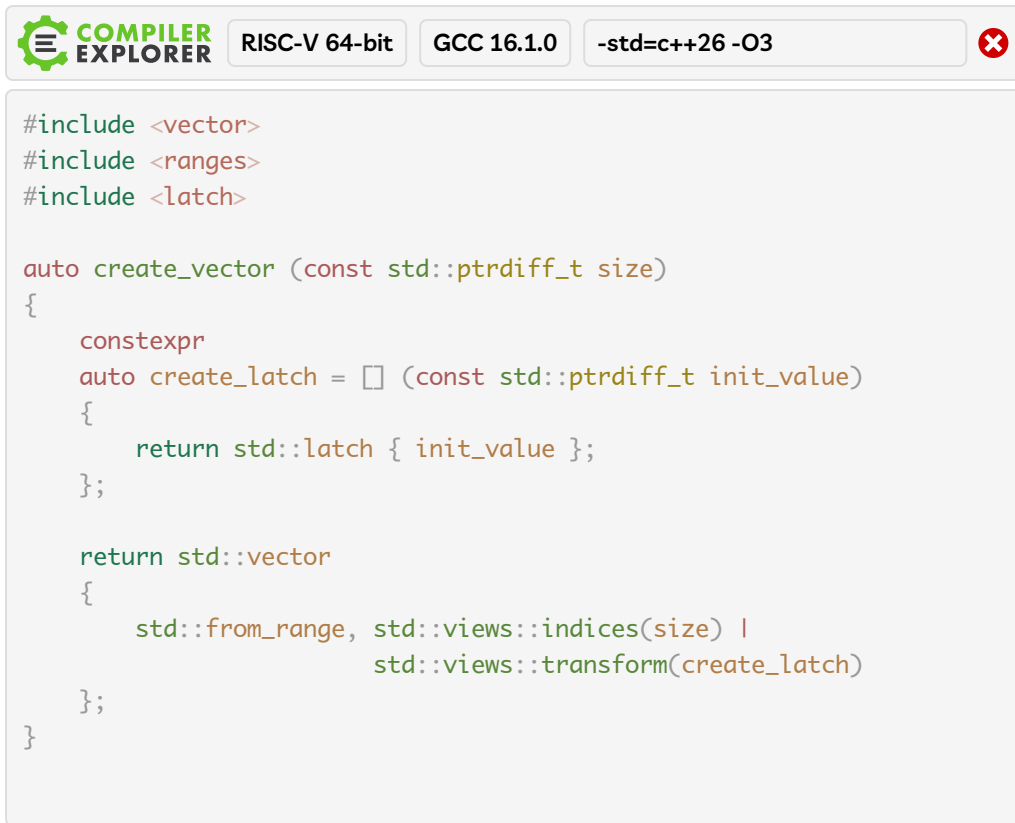


```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return std::vector
    {
        std::from_range, std::views::indices(size) |
                        std::views::transform(create_latch)
    };
}
```

create from materializable range

The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header bar with the 'Compiler Explorer' logo on the left, and three buttons in the center: 'RISC-V 64-bit', 'GCC 16.1.0', and '-std=c++26 -O3'. A red close button is on the far right. Below the header is a large text area containing C++ code. The code defines a function 'create_vector' that takes a 'size' parameter and returns a 'std::vector'. Inside the function, a 'constexpr' variable 'create_latch' is defined as an empty range. The function then returns a 'std::vector' constructed from a range using 'std::from_range', with 'std::views::indices(size)' and 'std::views::transform(create_latch)' as arguments.

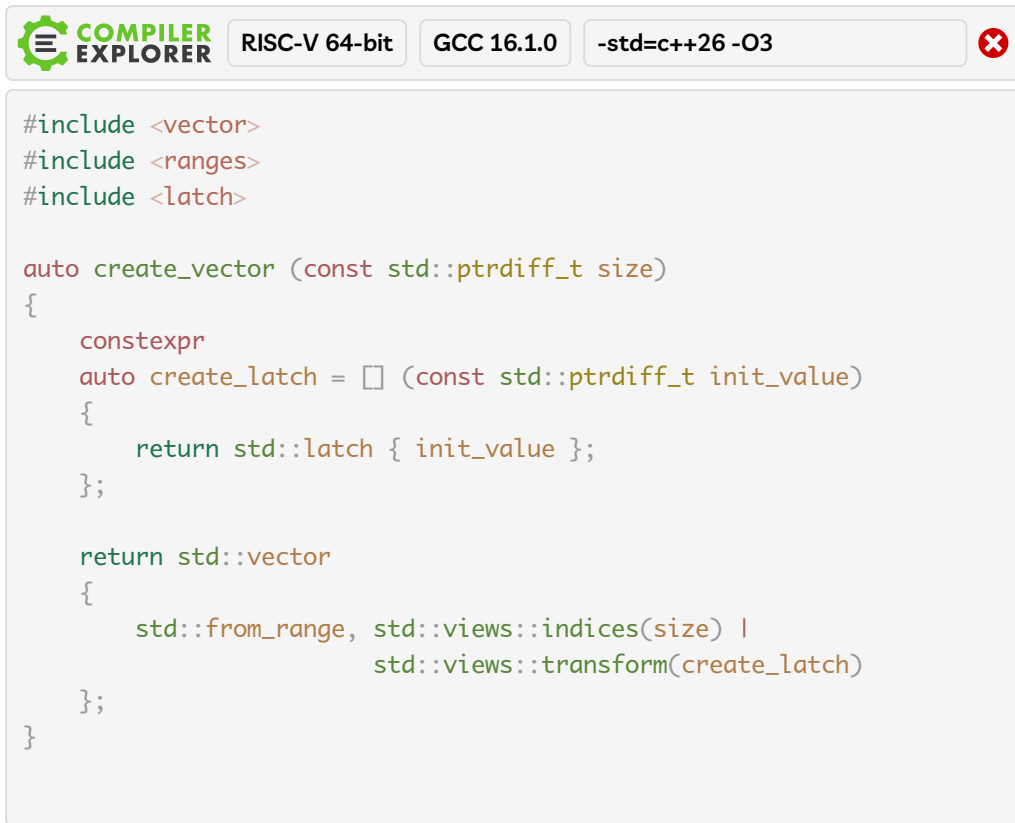
```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return std::vector
    {
        std::from_range, std::views::indices(size) |
                          std::views::transform(create_latch)
    };
}
```

- *Unfortunately, AllocatorAwareContainers do not support guaranteed copy elision.*

create from materializable range



```
#include <vector>
#include <ranges>
#include <latch>

auto create_vector (const std::ptrdiff_t size)
{
    constexpr
    auto create_latch = [] (const std::ptrdiff_t init_value)
    {
        return std::latch { init_value };
    };

    return std::vector
    {
        std::from_range, std::views::indices(size) |
                        std::views::transform(create_latch)
    };
}
```

- *Unfortunately, `AllocatorAwareContainers` do not support guaranteed copy elision.*
- *Sadly, `std::vector<>` is an `AllocatorAwareContainer`.*

create from list of values

create from list of values



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3 -DNDEBUG -I.



```
auto initial_website_files ()
{
    // predefine initial files
    //
    return jsc::bounded_vector
    {{
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    }};
    //
    // will append dependent files after parsing initial files
}
```

create from list of values

 COMPILER EXPLORER

RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3



```
auto initial_website_files ()
{
    // predefine intial files
    //
    return std::vector
    {
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    };
    //
    // will append dependent files after parsing initial files
}
```

create from list of values

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3

```
auto initial_website_files ()
{
    // predefine intial files
    //
    return std::vector
    {
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    };
    //
    // will append dependent files after parsing initial files
}
```

- A `std::initializer_list<>` refers to a backing array of `const` elements that cannot be moved out of the backing array.

create from list of values

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3

```
auto initial_website_files ()
{
    // predefine intial files
    //
    return std::vector
    {
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    };
    //
    // will append dependent files after parsing initial files
}
```

- A `std::initializer_list<>` refers to a backing array of `const` elements that cannot be moved out of the backing array.
- Naturally, `std::ifstream` is move-only, as it manages a unique resource: namely, a file handle.

create from list of values

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3 ✓

```
auto initial_website_files ()
{
    // predefine intial files
    //
    auto files = std::array
    {
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    };

    return std::vector
    {
        std::from_range, files | std::views::as_rvalue
    };
    //
    // will append dependent files after parsing initial files
}
```

- A `std::initializer_list<>` refers to a backing array of `const` elements that cannot be moved out of the backing array.
- Naturally, `std::ifstream` is move-only, as it manages a unique resource: namely, a file handle.

create from list of values

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3 ✓

```
auto initial_website_files ()
{
    // predefine intial files
    //
    auto files = std::array
    {
        std::ifstream { "config.json" },
        std::ifstream { "style.css" },
        std::ifstream { "script.js" },
        std::ifstream { "index.html" },
    };

    return std::vector
    {
        std::from_range, files | std::views::as_rvalue
    };
    //
    // will append dependent files after parsing initial files
}
```

- A `std::initializer_list<>` refers to a backing array of `const` elements that cannot be moved out of the backing array.
- Naturally, `std::ifstream` is move-only, as it manages a unique resource: namely, a file handle.
- The range adaptor `std::views::as_rvalue` forces the from-range constructor to move elements out of the range.

create from consumable range

create from consumable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3 -DNDEBUG -I.



```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return jsc::bounded_vector { std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

create from consumable range



RISC-V 64-bit

GCC 16.1.0

-std=c++26 -O3



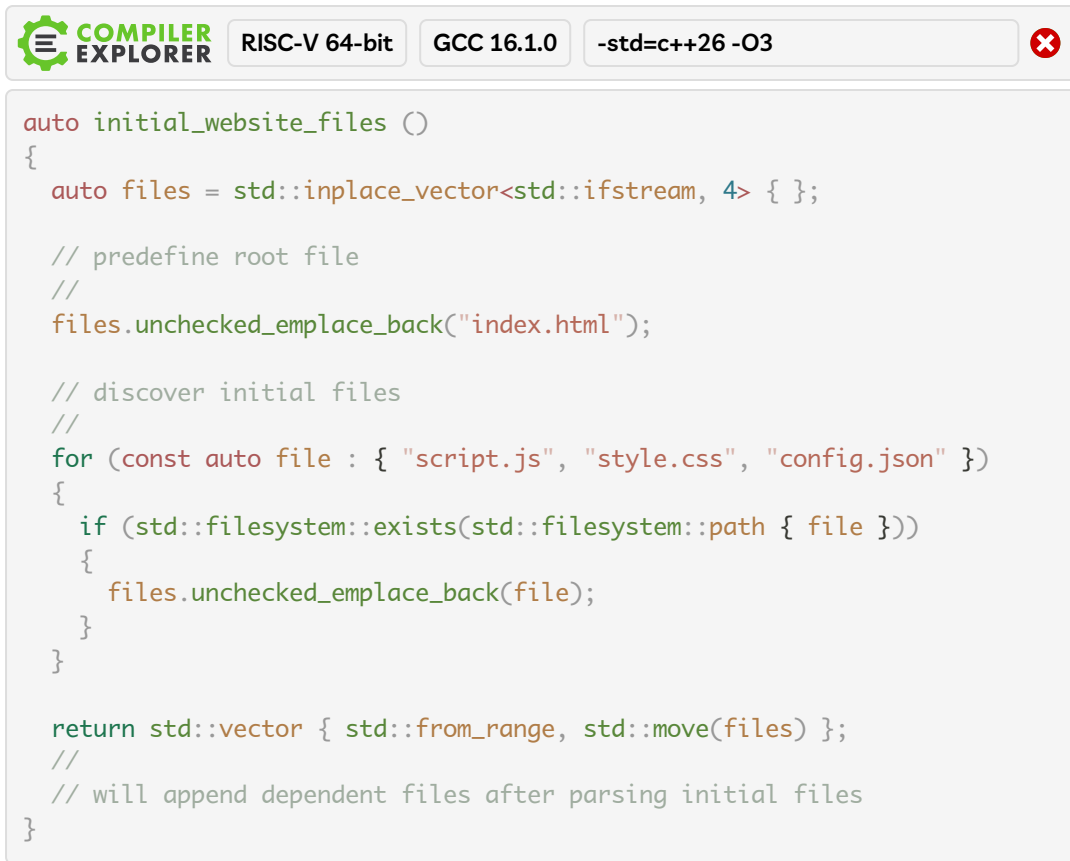
```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

create from consumable range



The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left, and three buttons: 'RISC-V 64-bit', 'GCC 16.1.0', and a dropdown menu currently showing '-std=c++26 -O3'. A red close button is on the far right of the header. Below the header is a large text area containing C++ code. The code defines a function 'auto initial_website_files ()' which initializes a 'std::inplace_vector' of 'std::ifstream' objects. It predefines 'index.html' and then iterates over a list of files ('script.js', 'style.css', 'config.json') to check if they exist and add them to the vector. Finally, it returns a 'std::vector' created from the range of the 'files' vector using 'std::from_range' and 'std::move'.

```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

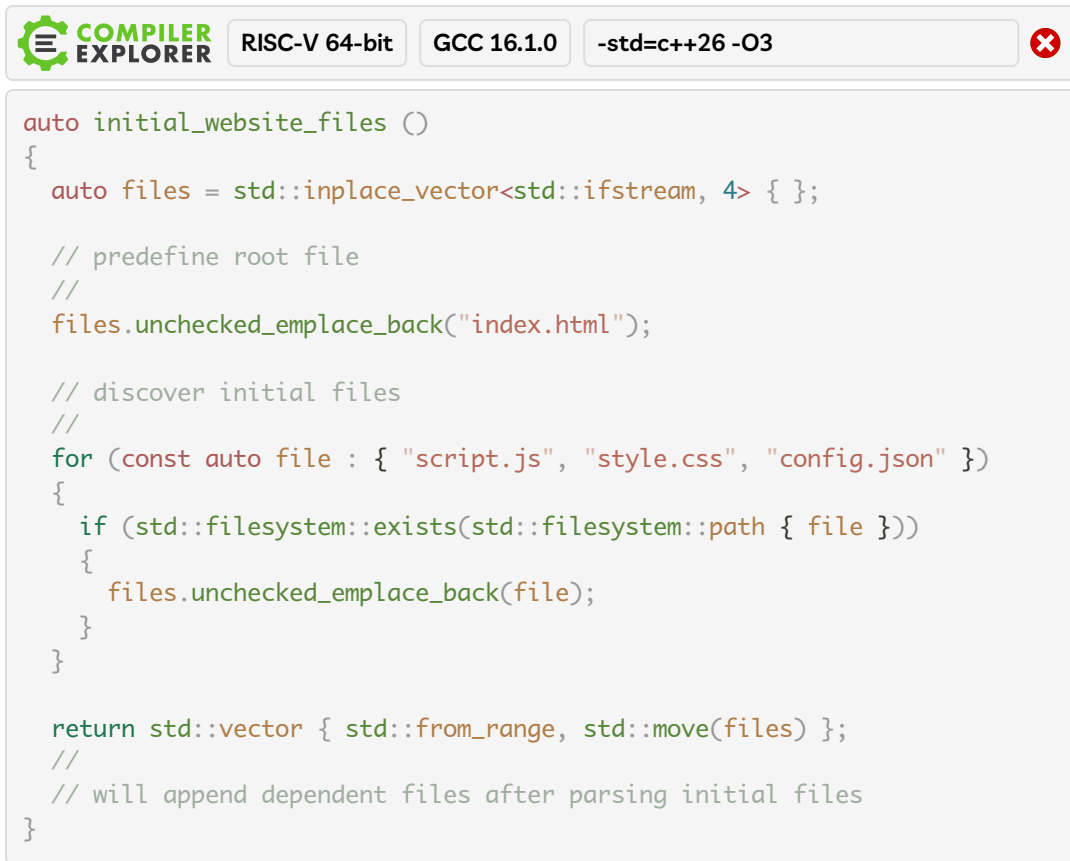
    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

- *Surprisingly, the from-range constructor will:*

create from consumable range



The image shows a screenshot of the Compiler Explorer web application. At the top, there is a header with the 'COMPILER EXPLORER' logo on the left, and three buttons: 'RISC-V 64-bit', 'GCC 16.1.0', and a dropdown menu currently showing '-std=c++26 -O3'. A red close button is on the far right. Below the header is a large text area containing C++ code. The code defines a function 'auto initial_website_files ()' which creates a vector 'files' using 'std::inplace_vector<std::ifstream, 4>'. It predefines 'index.html' and then iterates over an array of file names ('script.js', 'style.css', 'config.json'), checking if each file exists and adding it to the vector. Finally, it returns a 'std::vector' constructed from the 'files' range using 'std::from_range' and 'std::move'.

```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

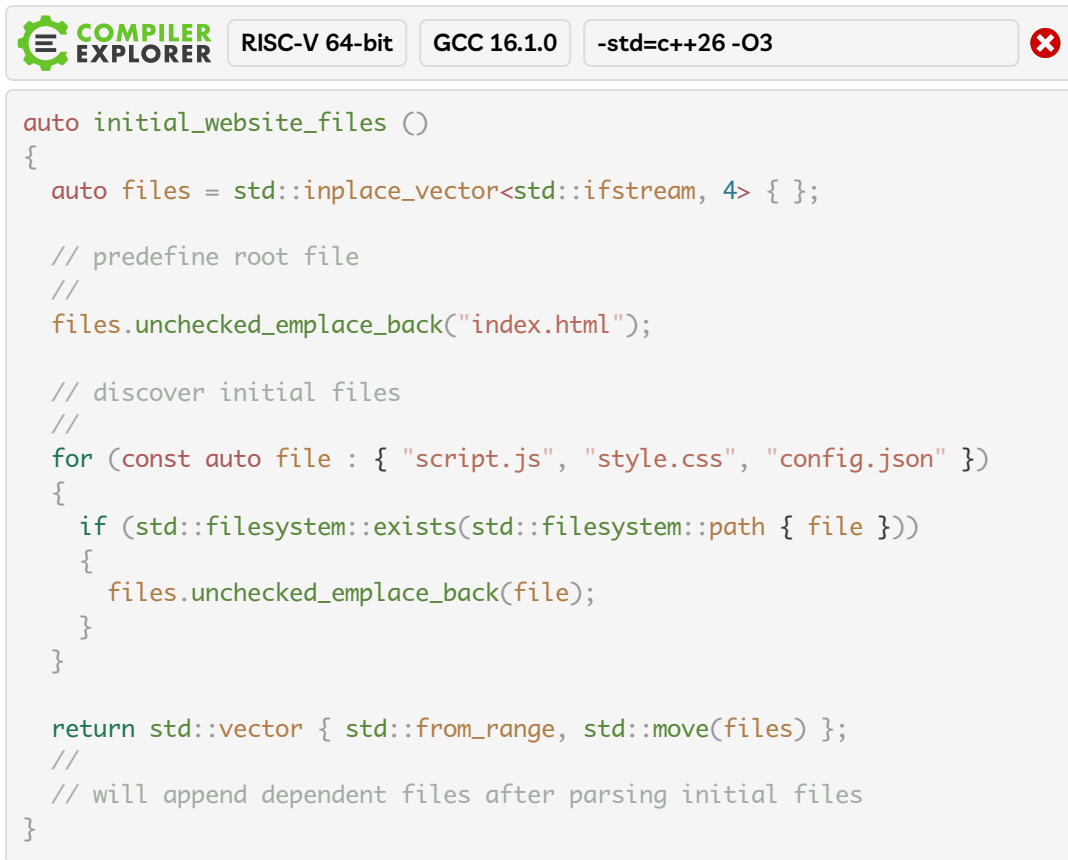
    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

- *Surprisingly, the from-range constructor will:*
 - *copy the elements from the range, when `std::ranges::range_reference_t<>` is an lvalue reference, or*

create from consumable range



COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3

```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

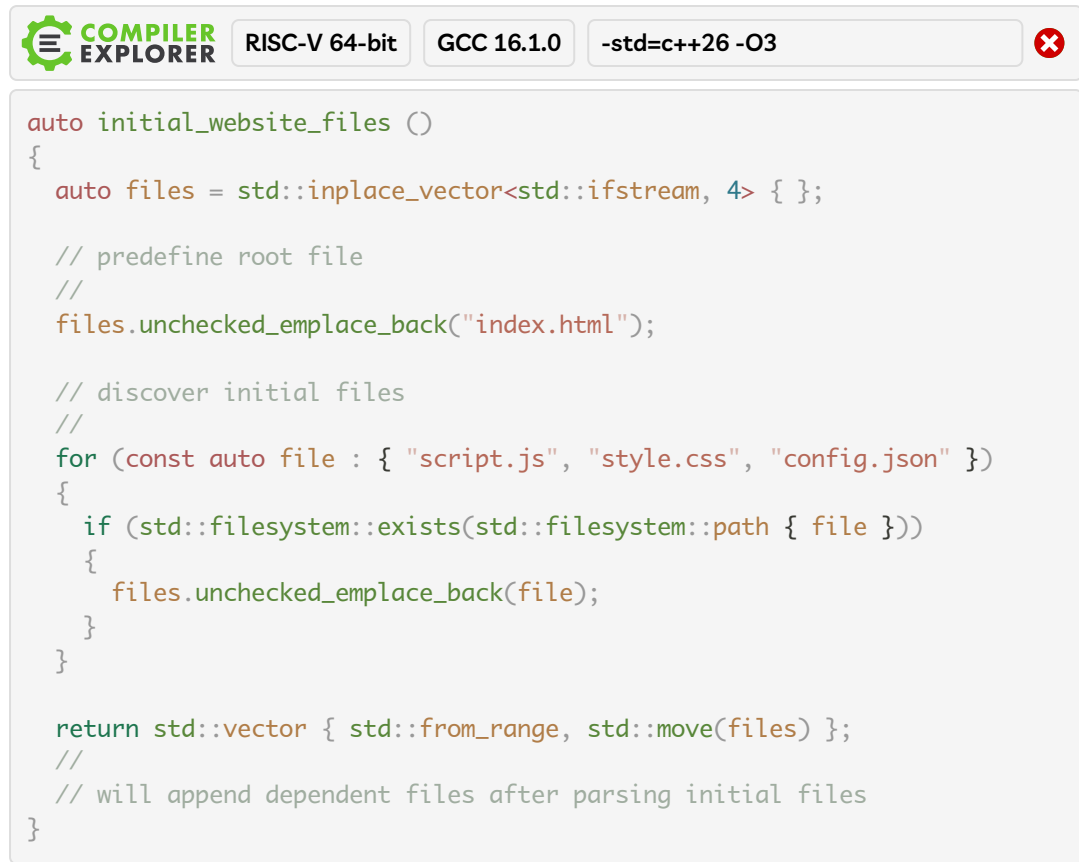
    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

- *Surprisingly, the from-range constructor will:*
 - *copy the elements from the range, when `std::ranges::range_reference_t<>` is an lvalue reference, or*
 - *move the elements out of the range, when `std::ranges::range_reference_t<>` is an rvalue reference or an object.*

create from consumable range



The image shows a screenshot of the Compiler Explorer web interface. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left, and three buttons: 'RISC-V 64-bit', 'GCC 16.1.0', and '-std=c++26 -O3'. A red close button is on the far right. Below the header, a large text area contains C++ code. The code defines a function 'auto initial_website_files ()' which creates a vector 'files' using 'std::inplace_vector<std::ifstream, 4>'. It then populates 'files' with 'index.html' and several other files ('script.js', 'style.css', 'config.json') using 'files.unchecked_emplace_back'. Finally, it returns 'std::vector { std::from_range, std::move(files) }'.

```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

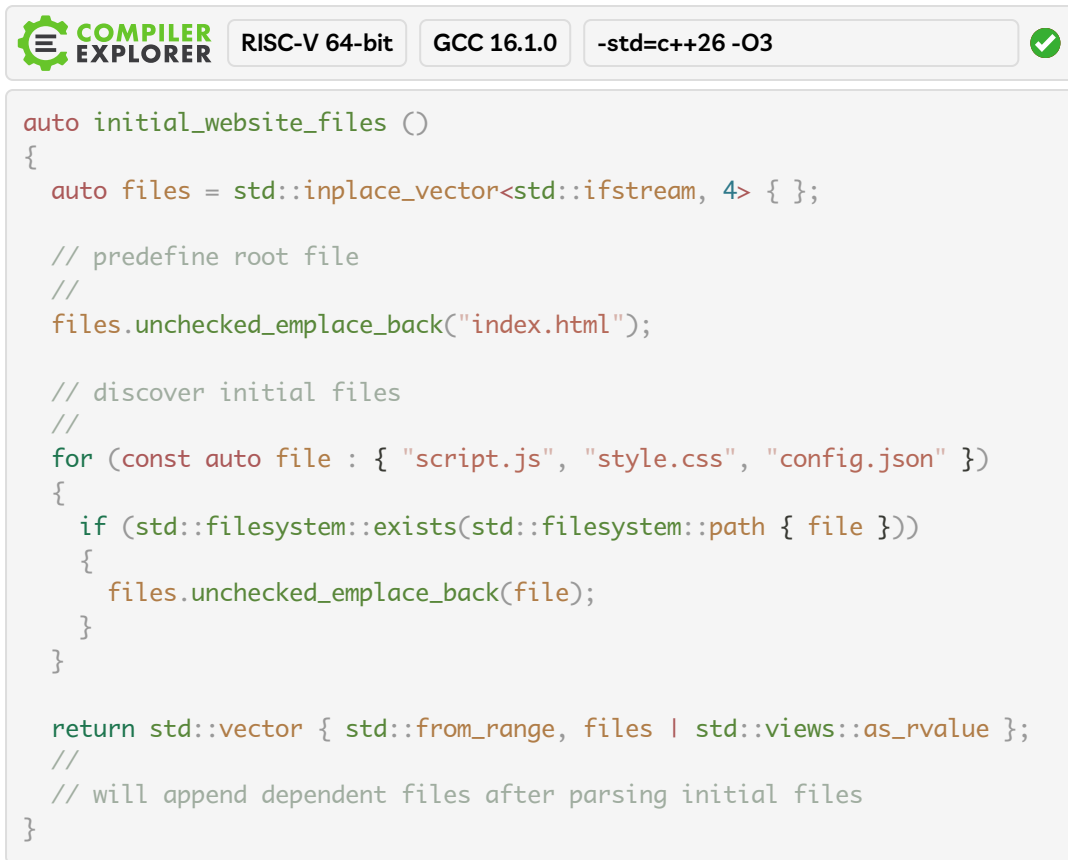
    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, std::move(files) };
    //
    // will append dependent files after parsing initial files
}
```

- *Surprisingly, the from-range constructor will:*
 - *copy the elements from the range, when `std::ranges::range_reference_t<>` is an lvalue reference, or*
 - *move the elements out of the range, when `std::ranges::range_reference_t<>` is an rvalue reference or an object.*
- *Meaning, the from-range constructor will always copy the range's elements, when given a Container.*

create from consumable range



COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++26 -O3

```
auto initial_website_files ()
{
    auto files = std::inplace_vector<std::ifstream, 4> { };

    // predefine root file
    //
    files.unchecked_emplace_back("index.html");

    // discover initial files
    //
    for (const auto file : { "script.js", "style.css", "config.json" })
    {
        if (std::filesystem::exists(std::filesystem::path { file }))
        {
            files.unchecked_emplace_back(file);
        }
    }

    return std::vector { std::from_range, files | std::views::as_rvalue };
    //
    // will append dependent files after parsing initial files
}
```

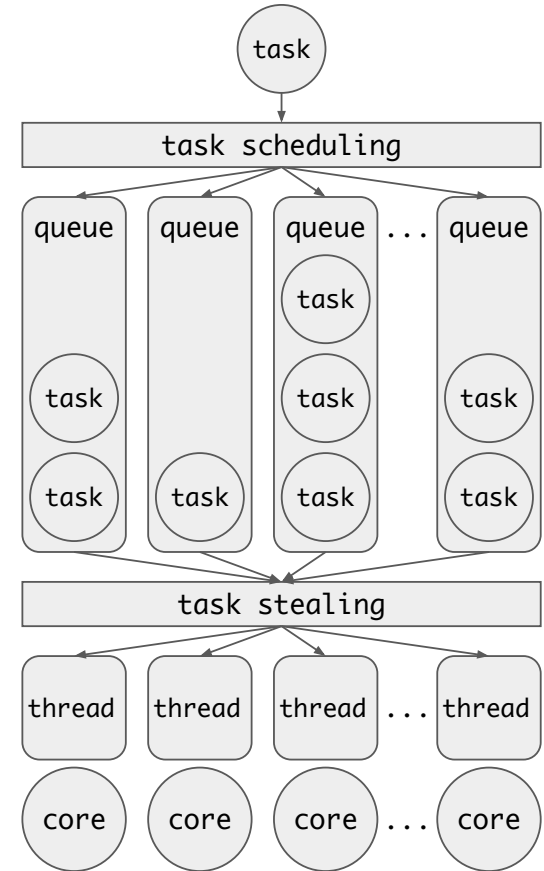
- *Surprisingly, the from-range constructor will:*
 - *copy the elements from the range, when `std::ranges::range_reference_t<>` is an lvalue reference, or*
 - *move the elements out of the range, when `std::ranges::range_reference_t<>` is an rvalue reference or an object.*
- *For a range expression*
`container | std::views::as_rvalue`,
the `std::ranges::range_reference_t<>` is always an rvalue reference.

A wide, powerful waterfall flows over a dark, rocky ledge. The water is turbulent and white with foam as it falls into a deep pool below. The background shows a flat landscape with some distant trees and buildings under a clear sky. The text "deep dive" is centered over the waterfall in a white, italicized serif font.

deep dive

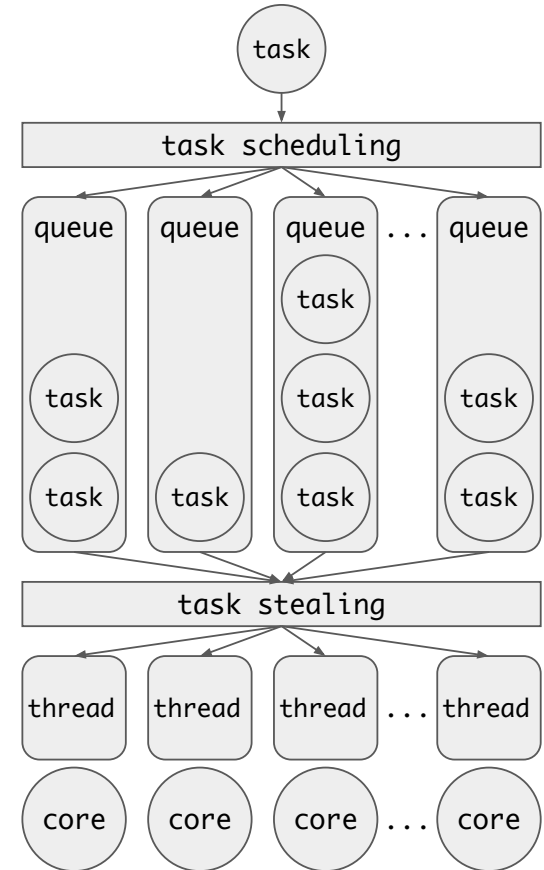
tasking system: thread pool, task queues

tasking system: thread pool, task queues



tasking system: thread pool, task queues

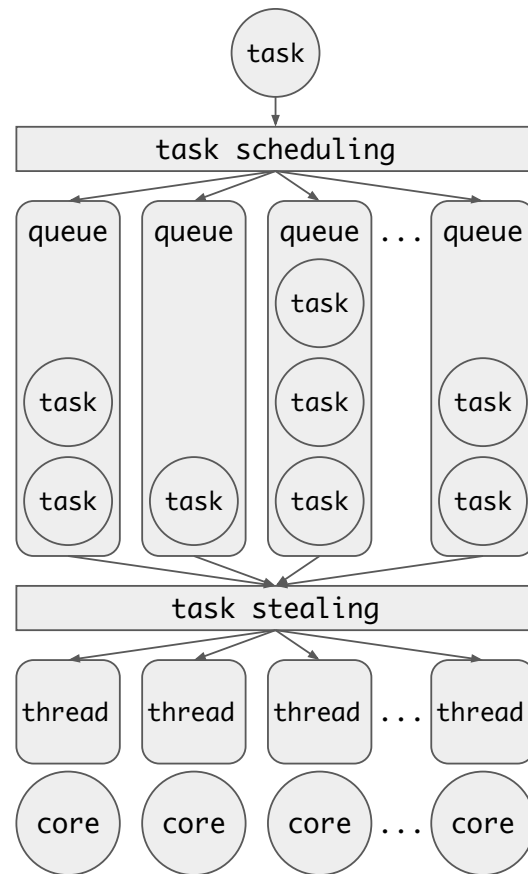
The run-time system consists of:



tasking system: thread pool, task queues

The run-time system consists of:

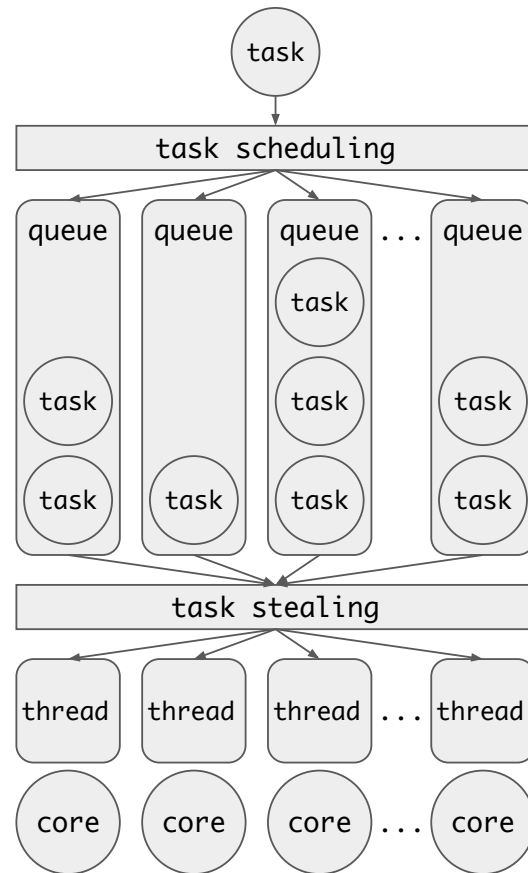
- pool of worker threads (one per core)*



tasking system: thread pool, task queues

The run-time system consists of:

- *pool of worker threads (one per core)*
- *queue of tasks (ready for execution)*

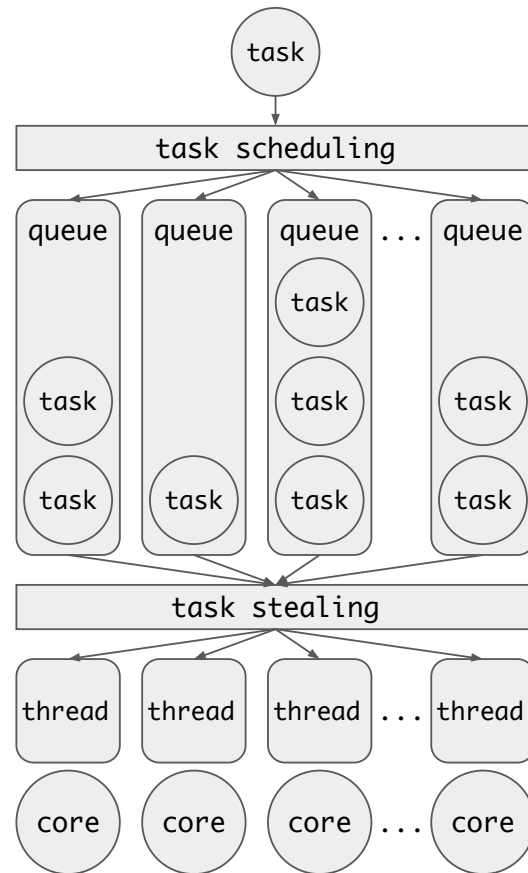


tasking system: thread pool, task queues

The run-time system consists of:

- *pool of worker threads (one per core)*
- *queue of tasks (ready for execution)*

Work is distributed evenly across workers via:



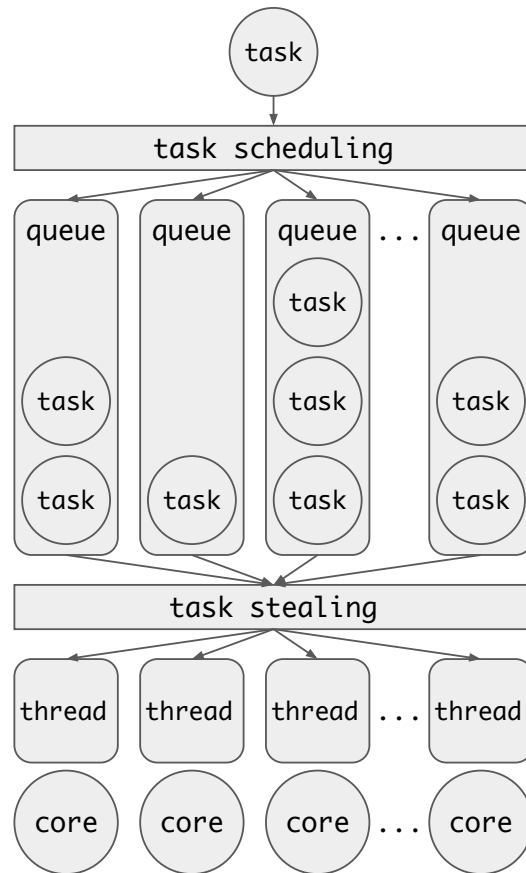
tasking system: thread pool, task queues

The run-time system consists of:

- *pool of worker threads (one per core)*
- *queue of tasks (ready for execution)*

Work is distributed evenly across workers via:

- *round-robin scheduling (full or busy queues are skipped)*



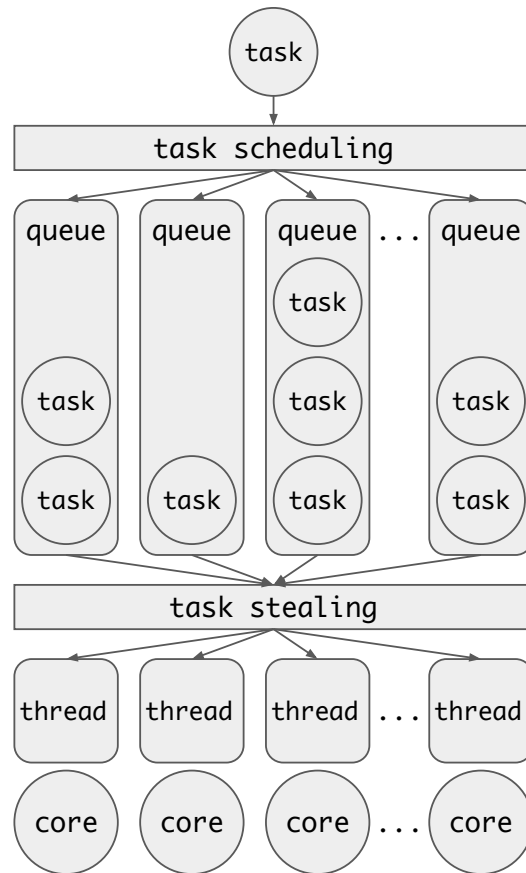
tasking system: thread pool, task queues

The run-time system consists of:

- *pool of worker threads (one per core)*
- *queue of tasks (ready for execution)*

Work is distributed evenly across workers via:

- *round-robin scheduling (full or busy queues are skipped)*
- *task-stealing (when queue is empty or busy)*



tasking system: task graph

tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

tasking system: task graph

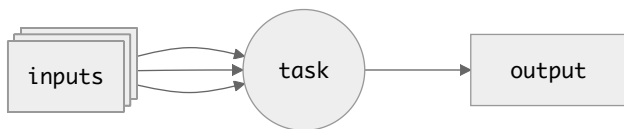
*Task is a unit of work (C++ function)
to be executed on a worker thread.*

```
auto task (inputs ...) -> output;
```

tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

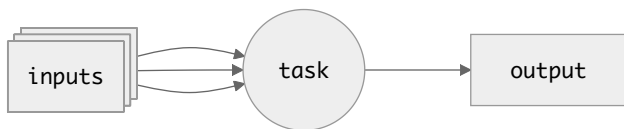
```
auto task (inputs ...) -> output;
```



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

```
auto task (inputs ...) -> output;
```

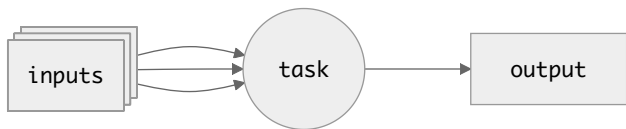


*Tasks are connected (form a DAG),
because of the data flow.*

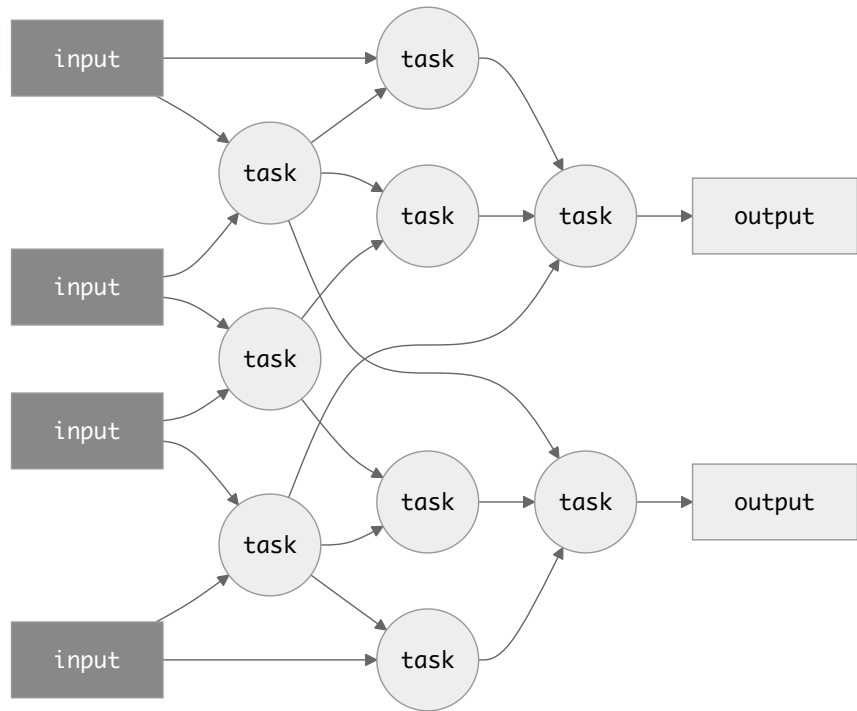
tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

```
auto task (inputs ...) -> output;
```



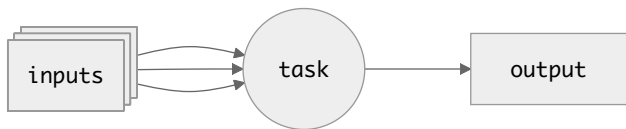
*Tasks are connected (form a DAG),
because of the data flow.*



tasking system: task graph

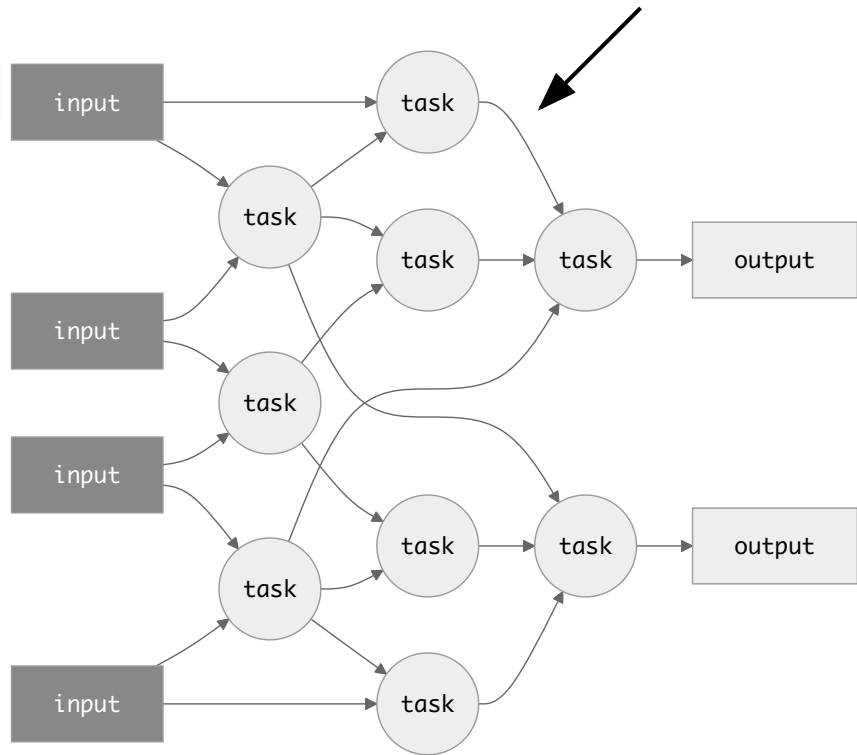
*Task is a unit of work (C++ function)
to be executed on a worker thread.*

```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

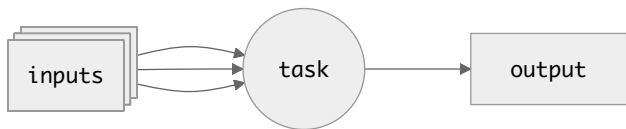
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

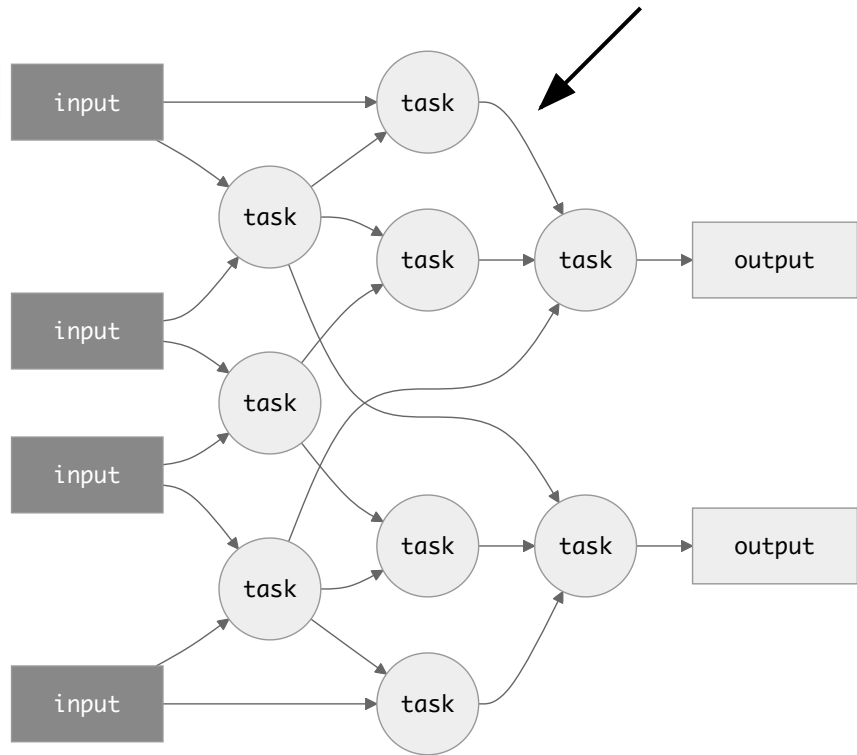
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

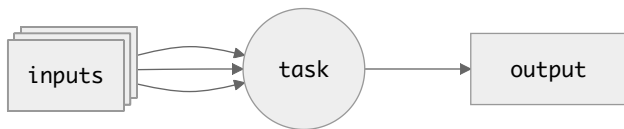
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

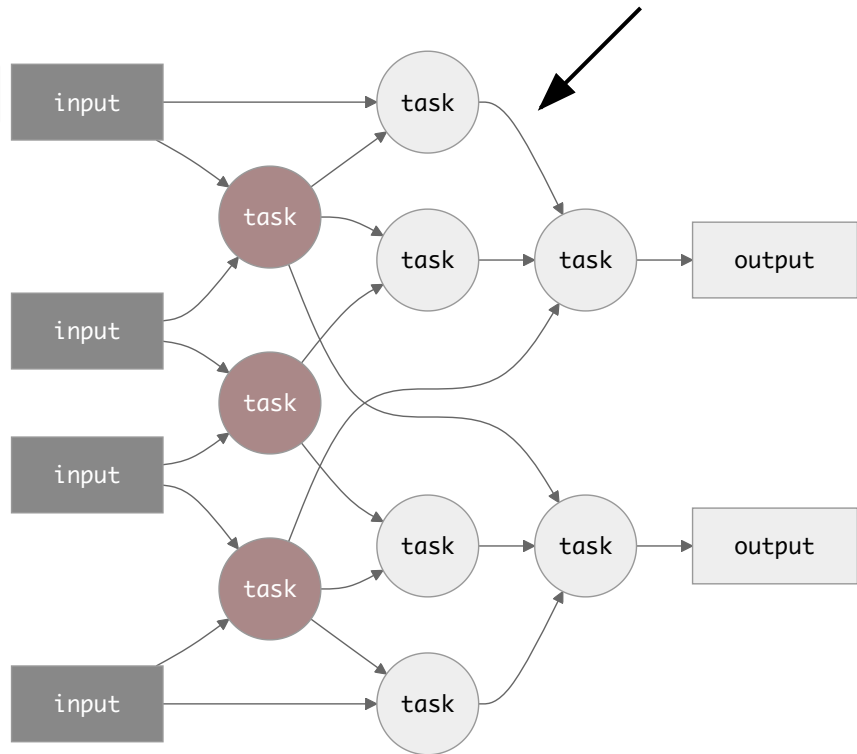
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

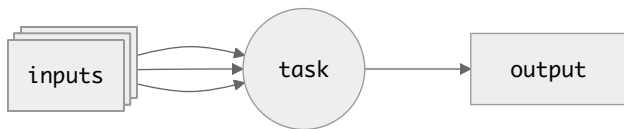
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

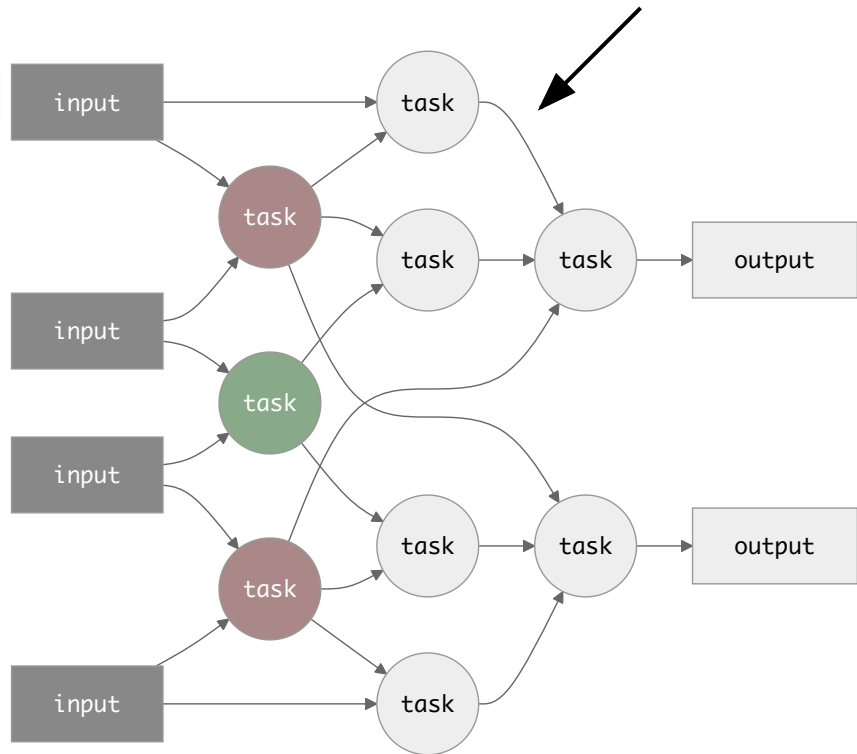
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

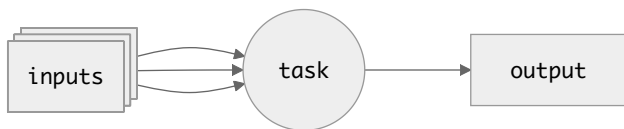
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

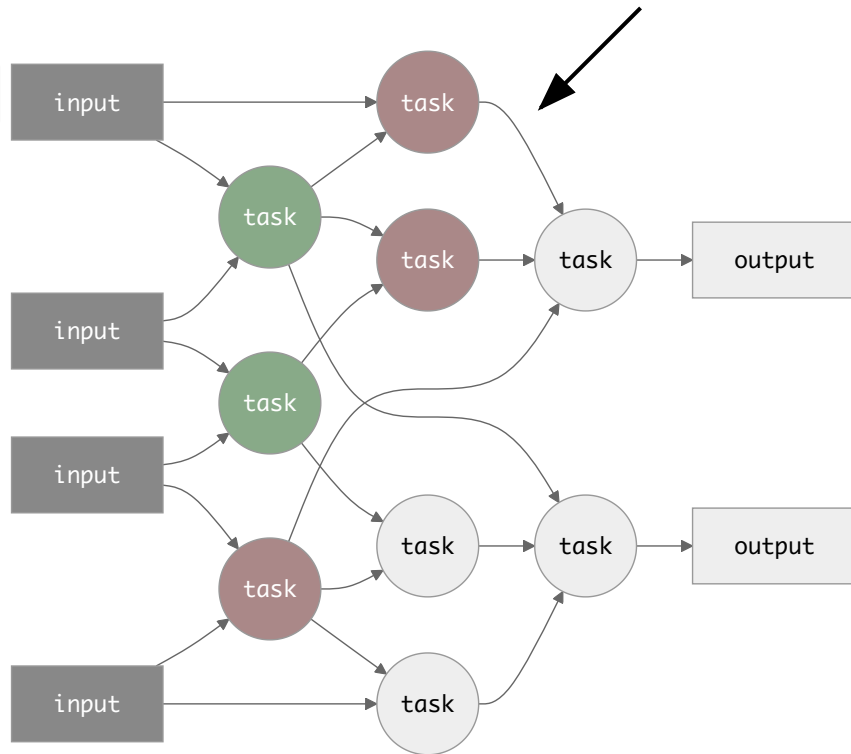
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

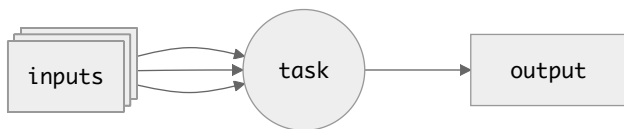
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

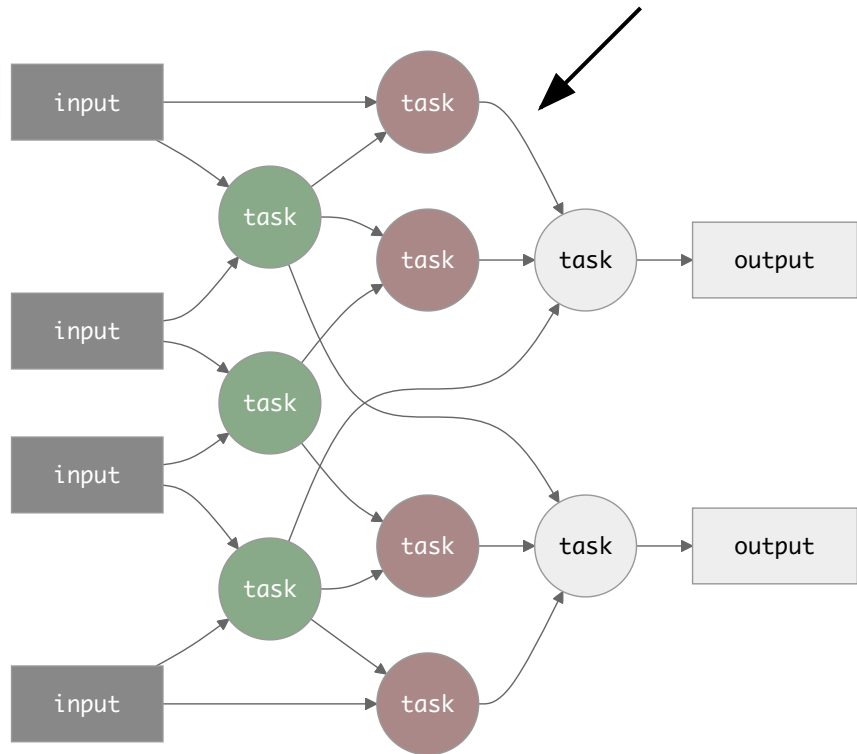
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

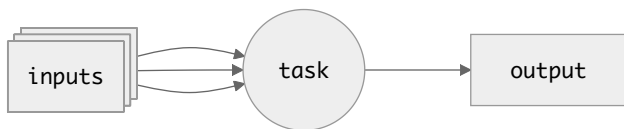
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

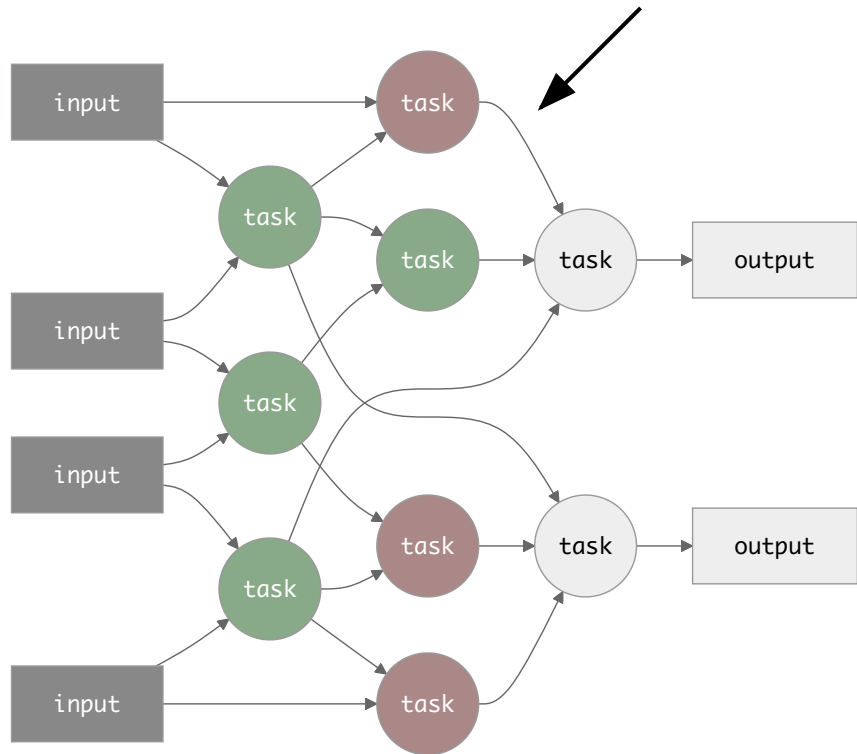
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

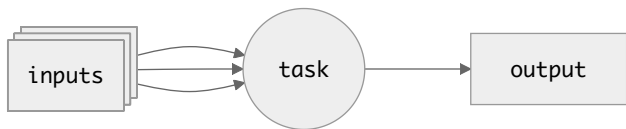
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

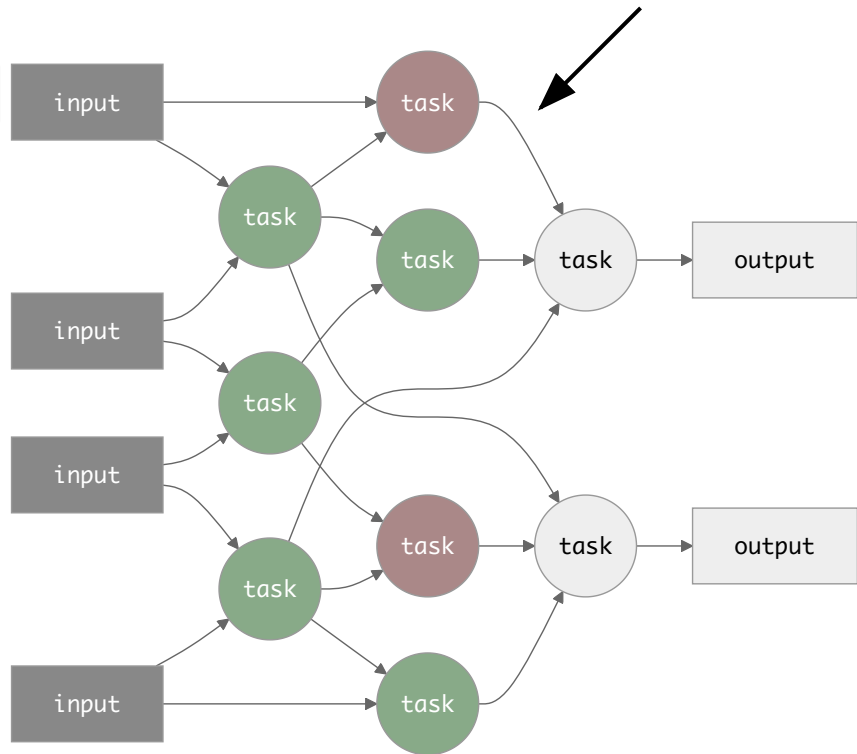
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

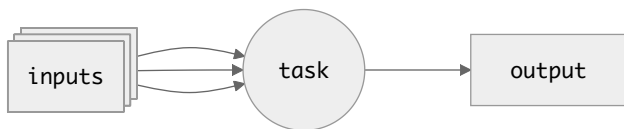
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

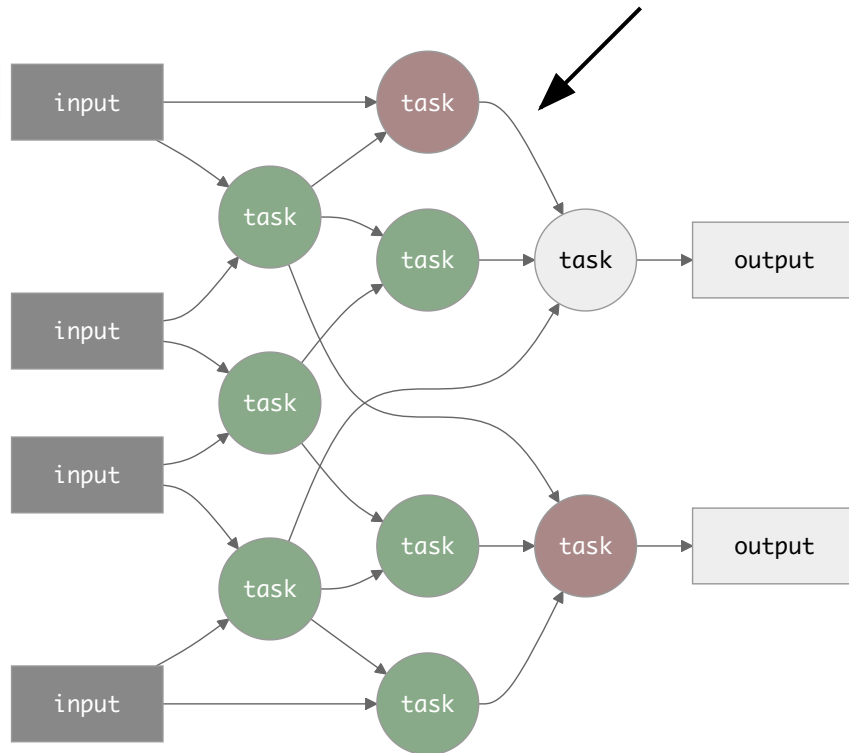
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

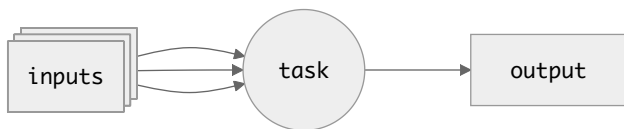
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

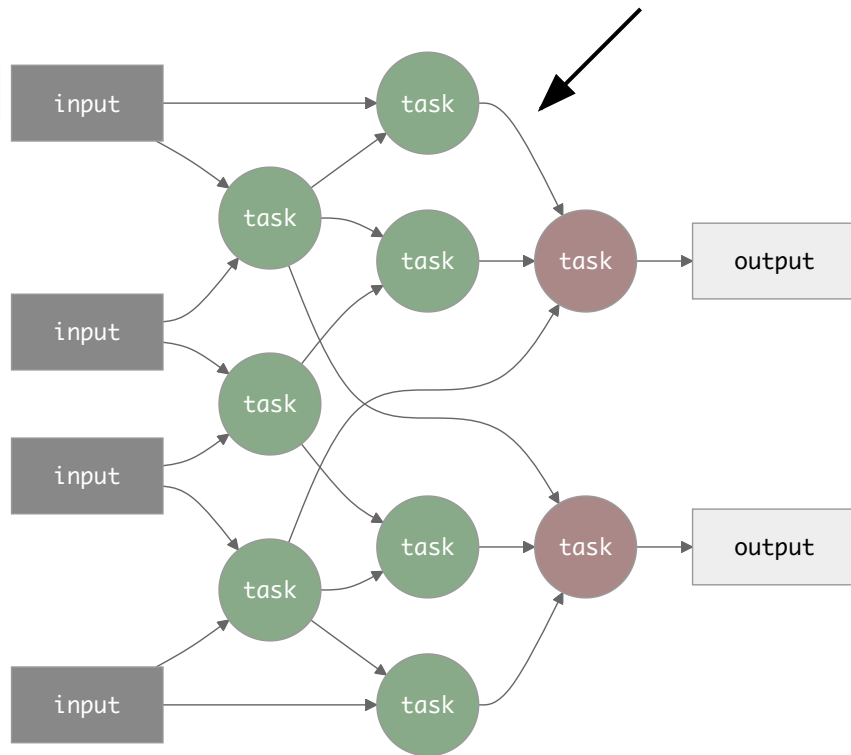
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

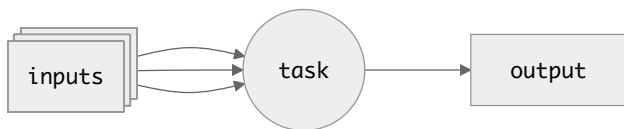
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

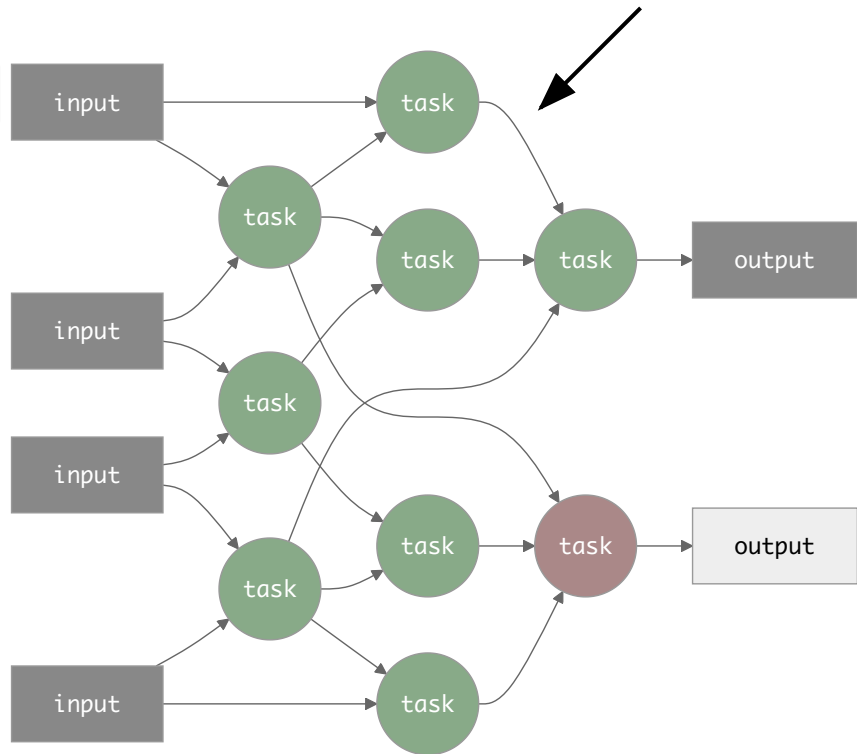
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

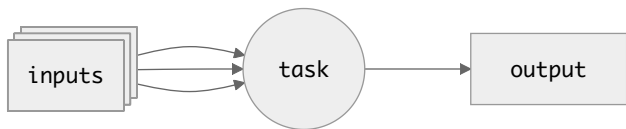
*the output from one task
is used as
an input to another task*



tasking system: task graph

*Task is a unit of work (C++ function)
to be executed on a worker thread.*

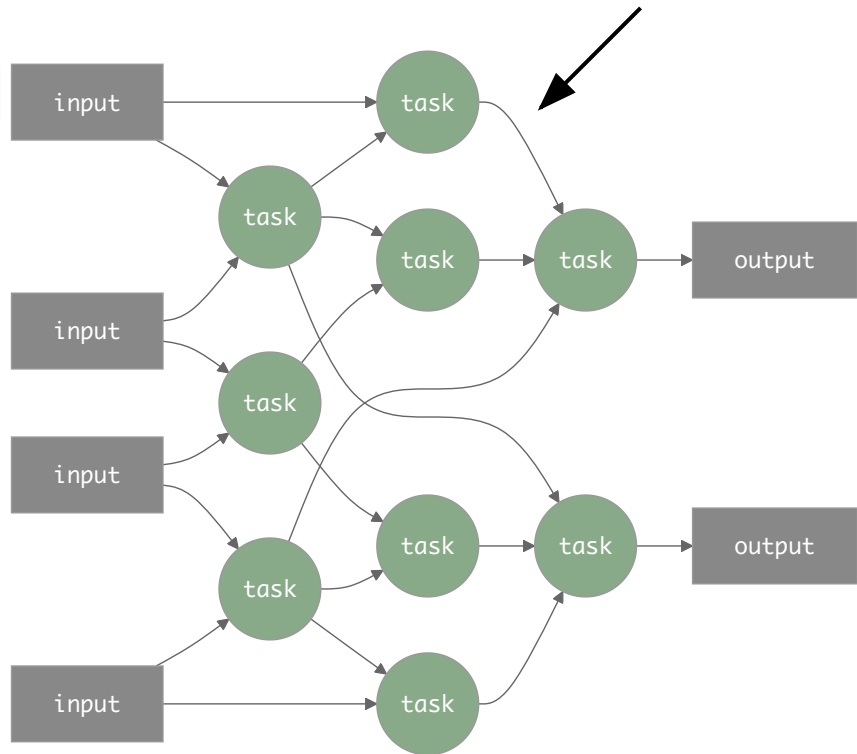
```
auto task (inputs ...) -> output;
```



*Tasks are connected (form a DAG),
because of the data flow.*

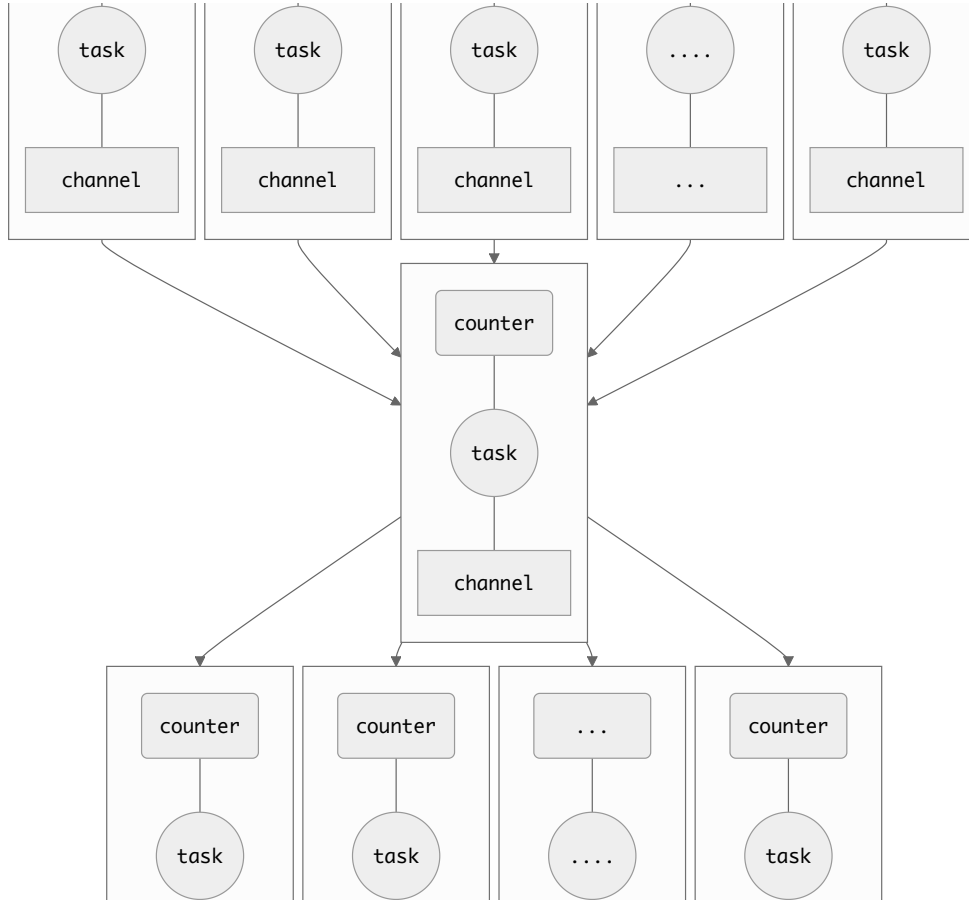
*Dependencies between tasks form,
because all inputs of a task
must be ready (computed)
before its execution starts.*

*the output from one task
is used as
an input to another task*

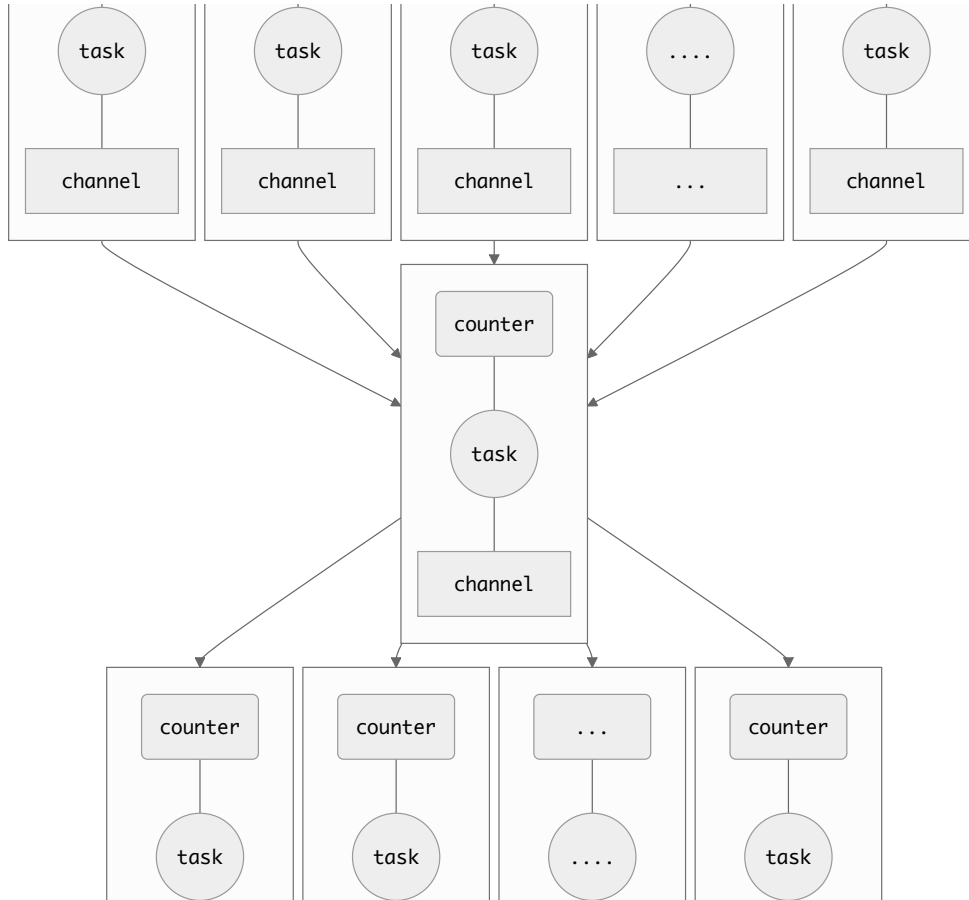


tasking system: synchronization

tasking system: synchronization

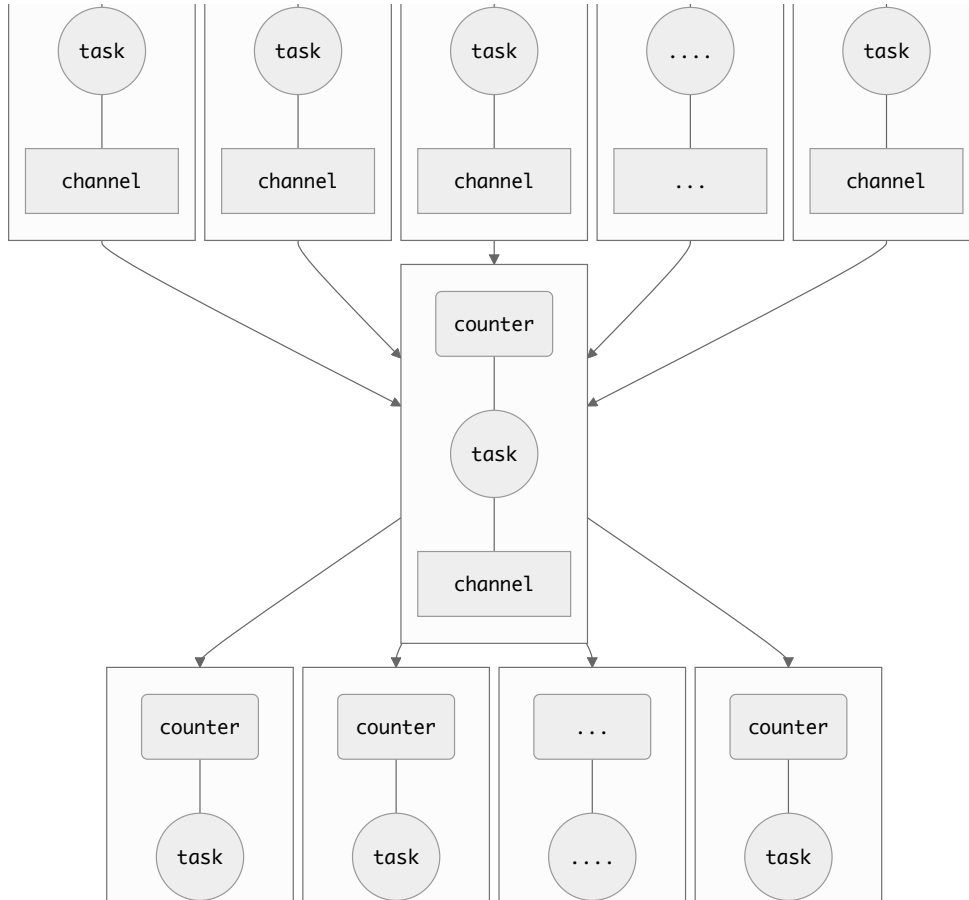


tasking system: synchronization



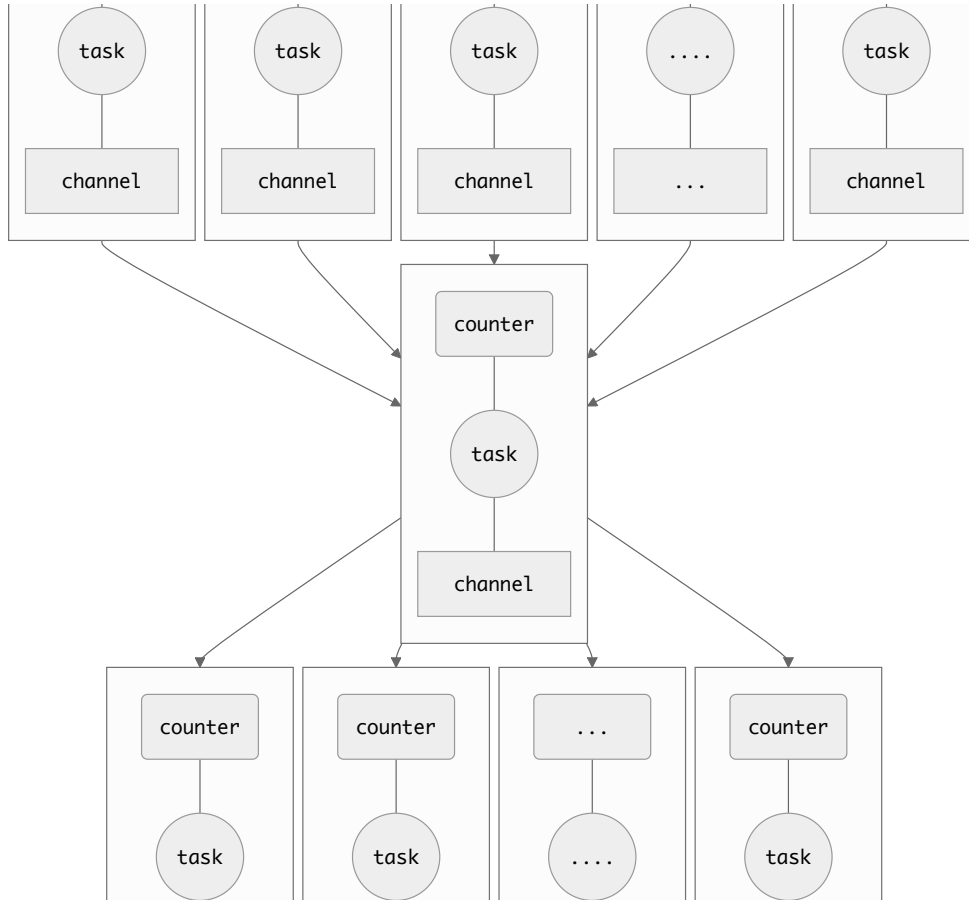
- *The output of a task is written into the communication channel, then is read by dependent tasks.*

tasking system: synchronization



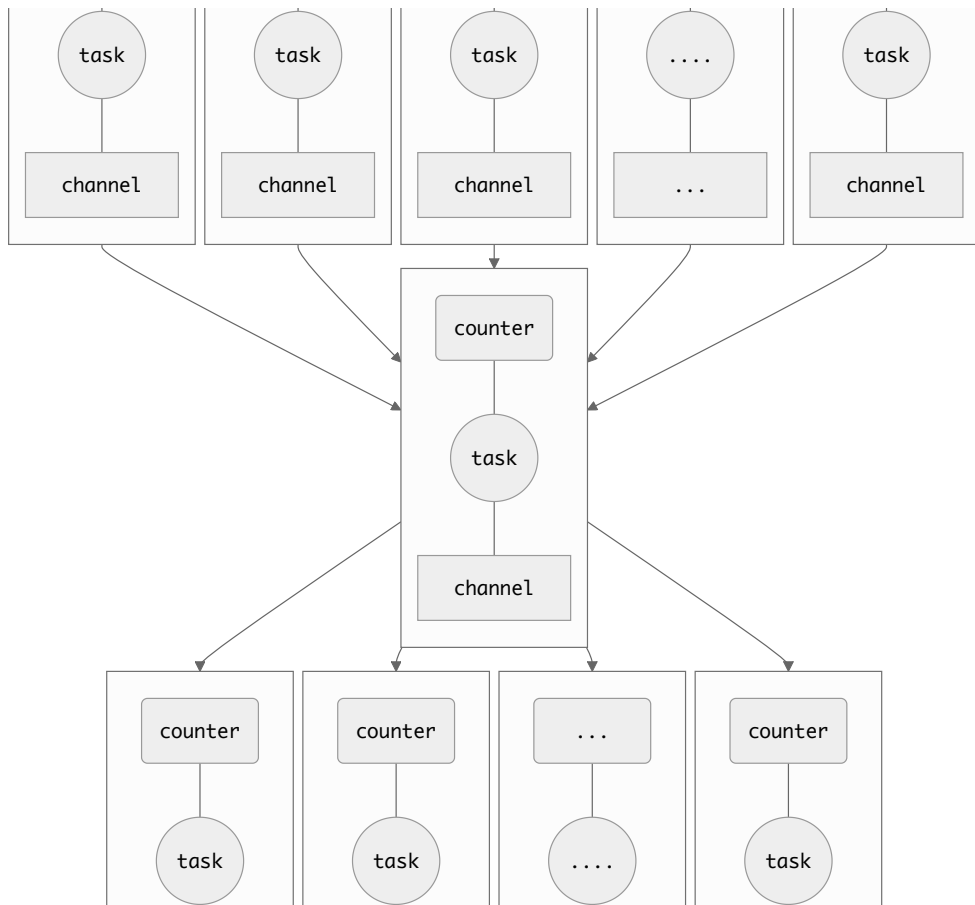
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*

tasking system: synchronization



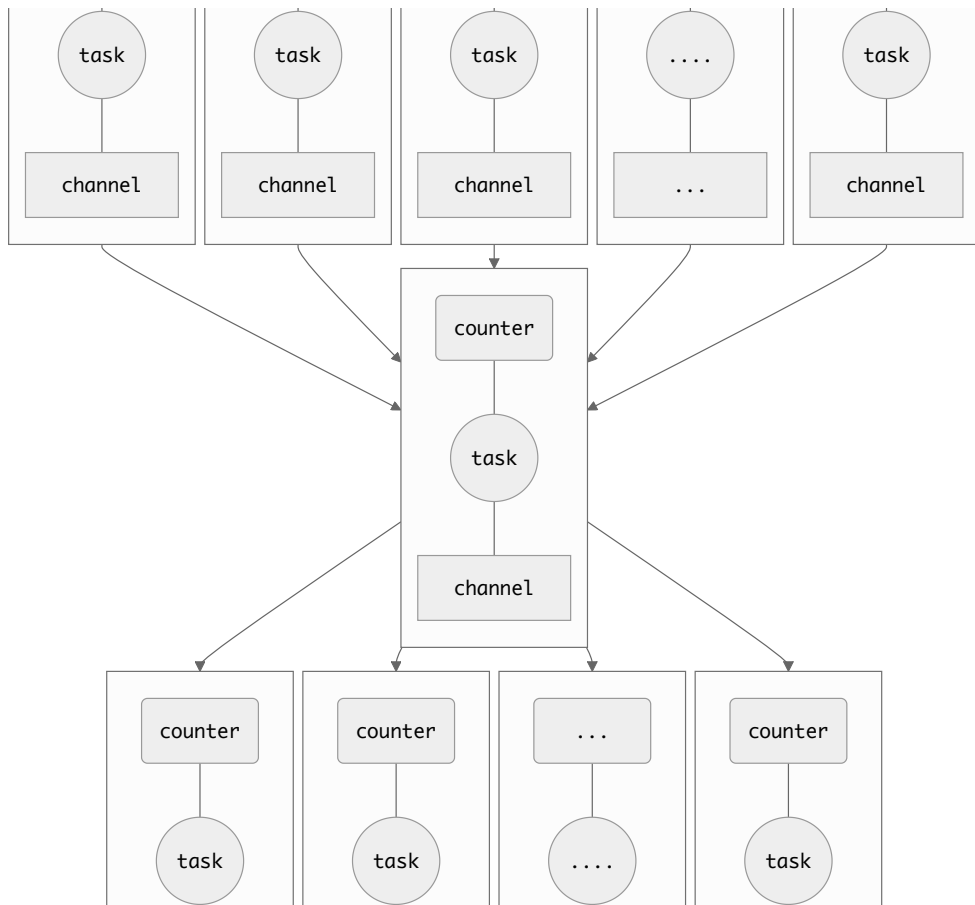
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*

tasking system: synchronization



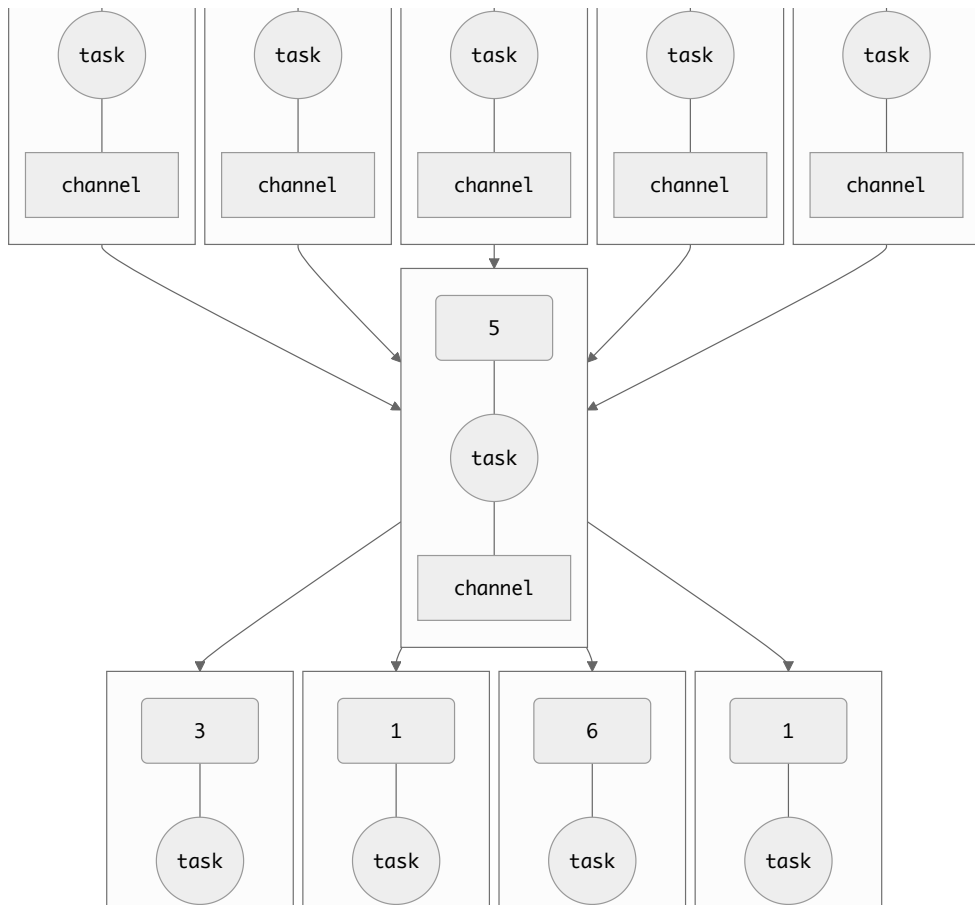
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*

tasking system: synchronization



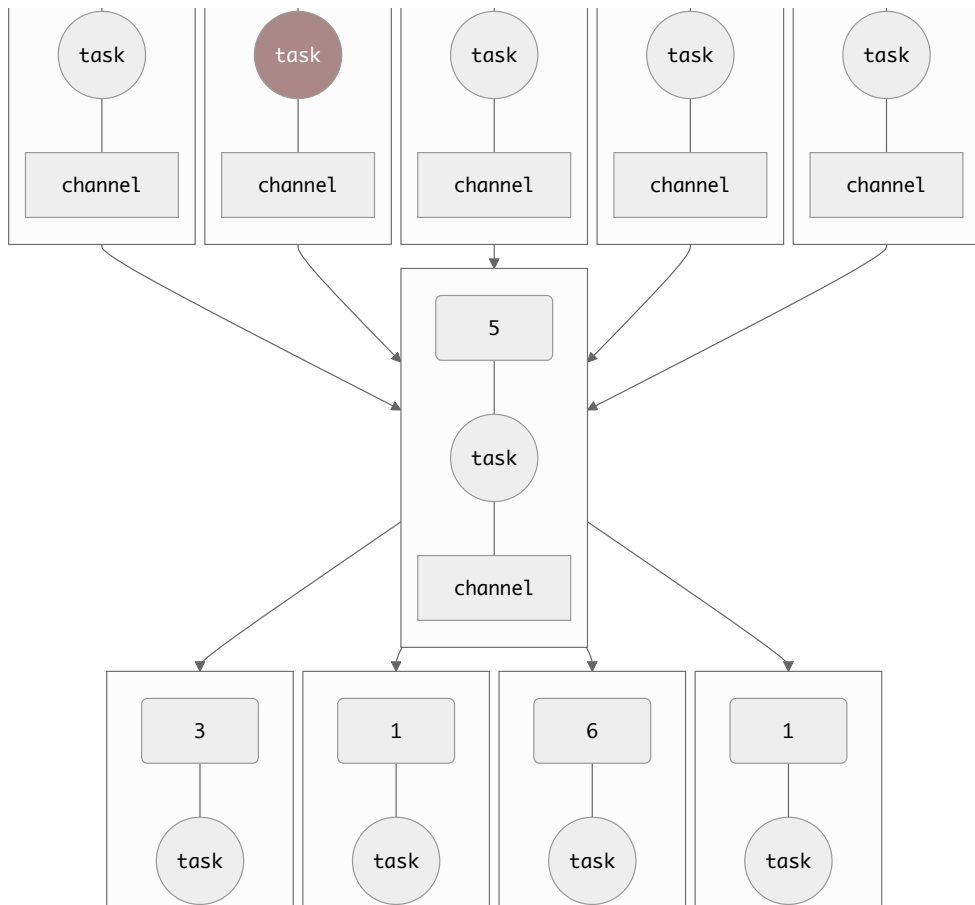
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



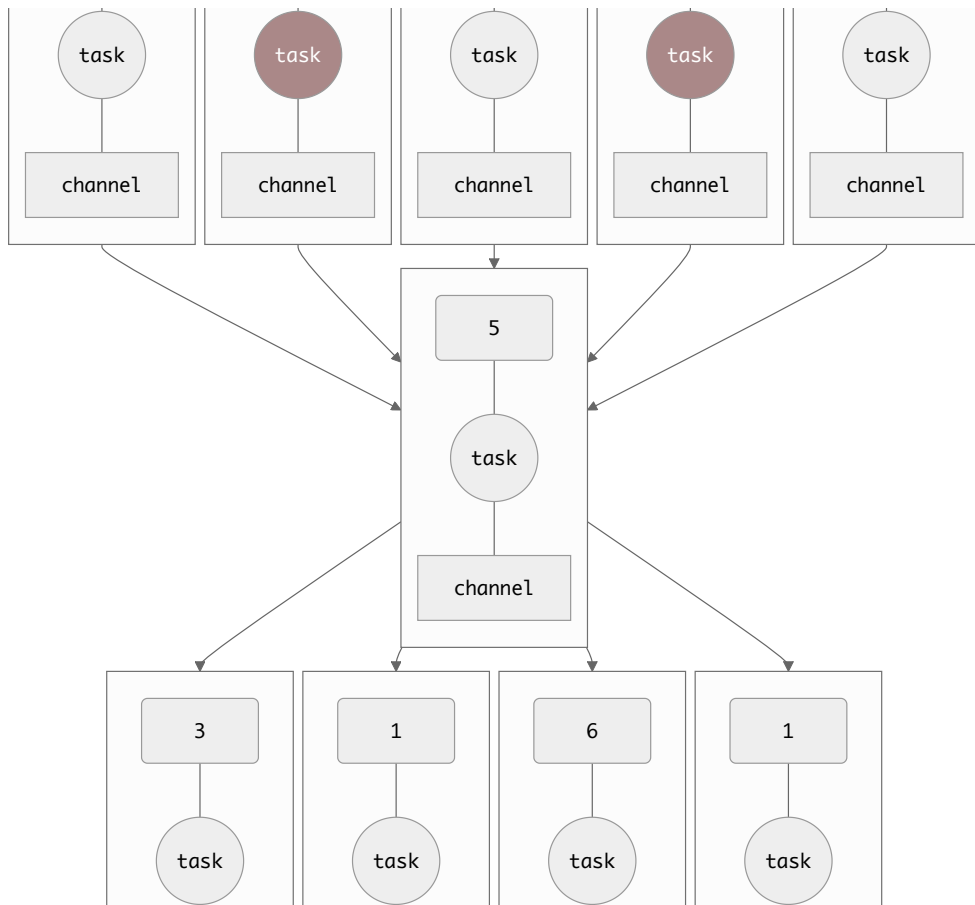
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



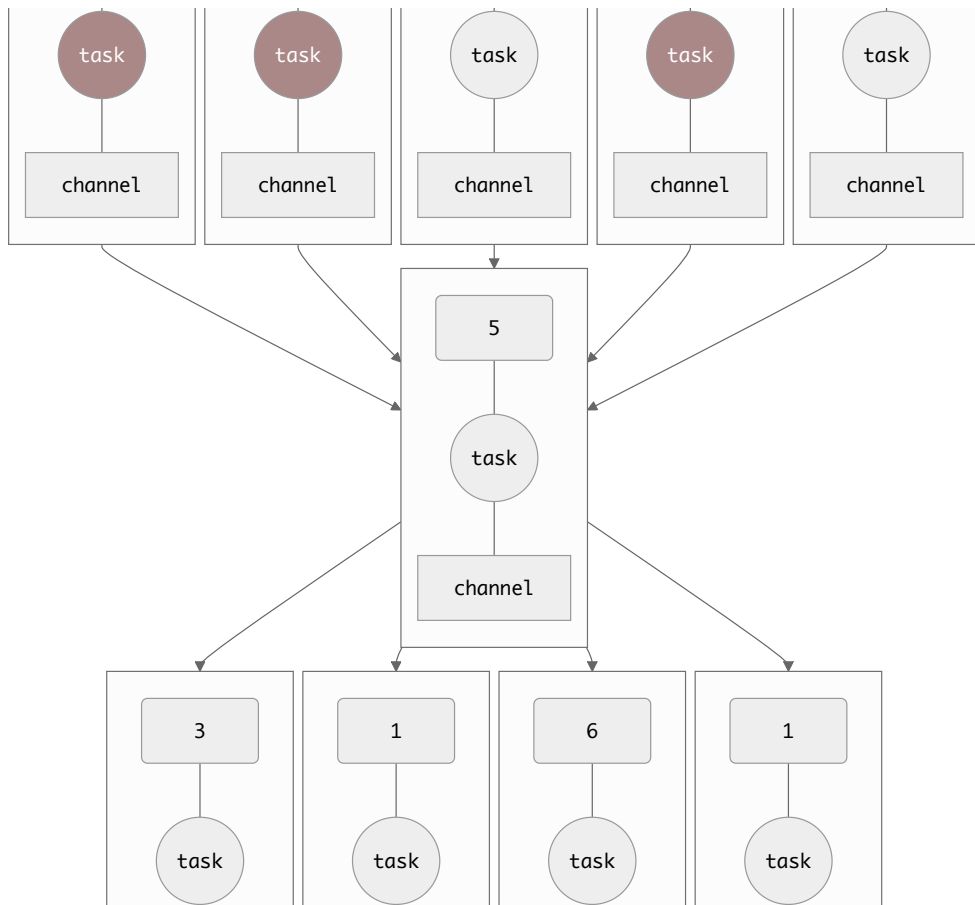
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



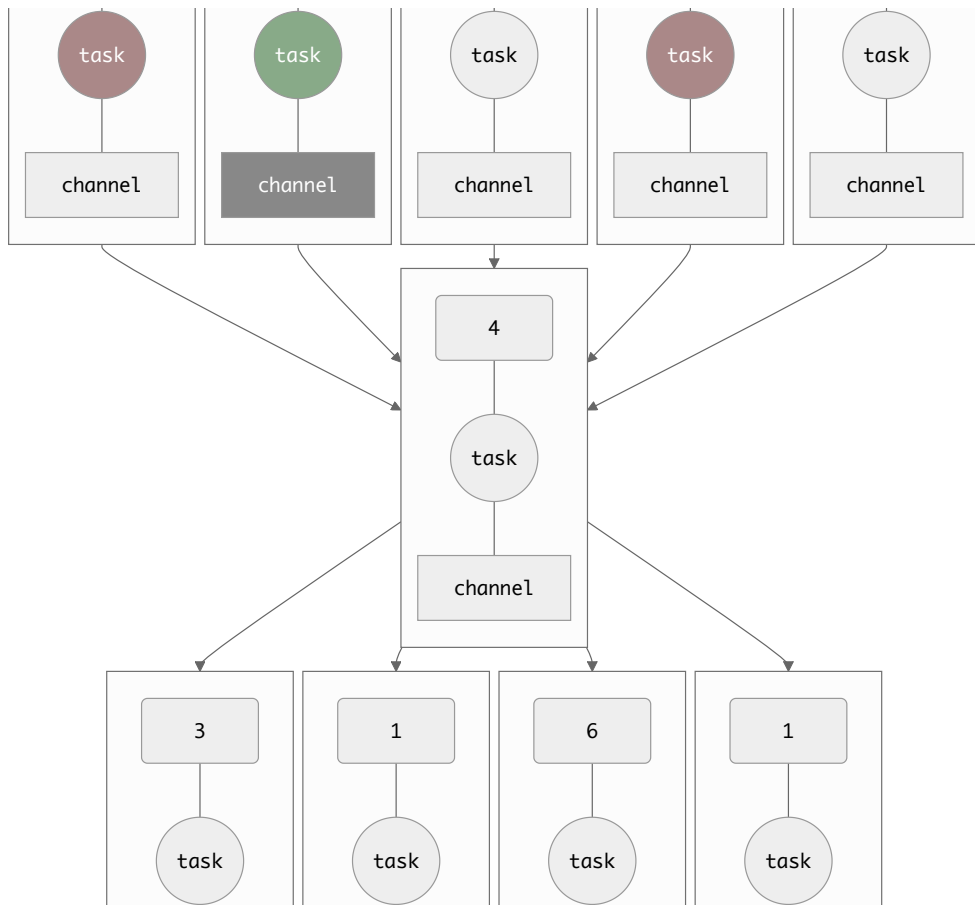
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



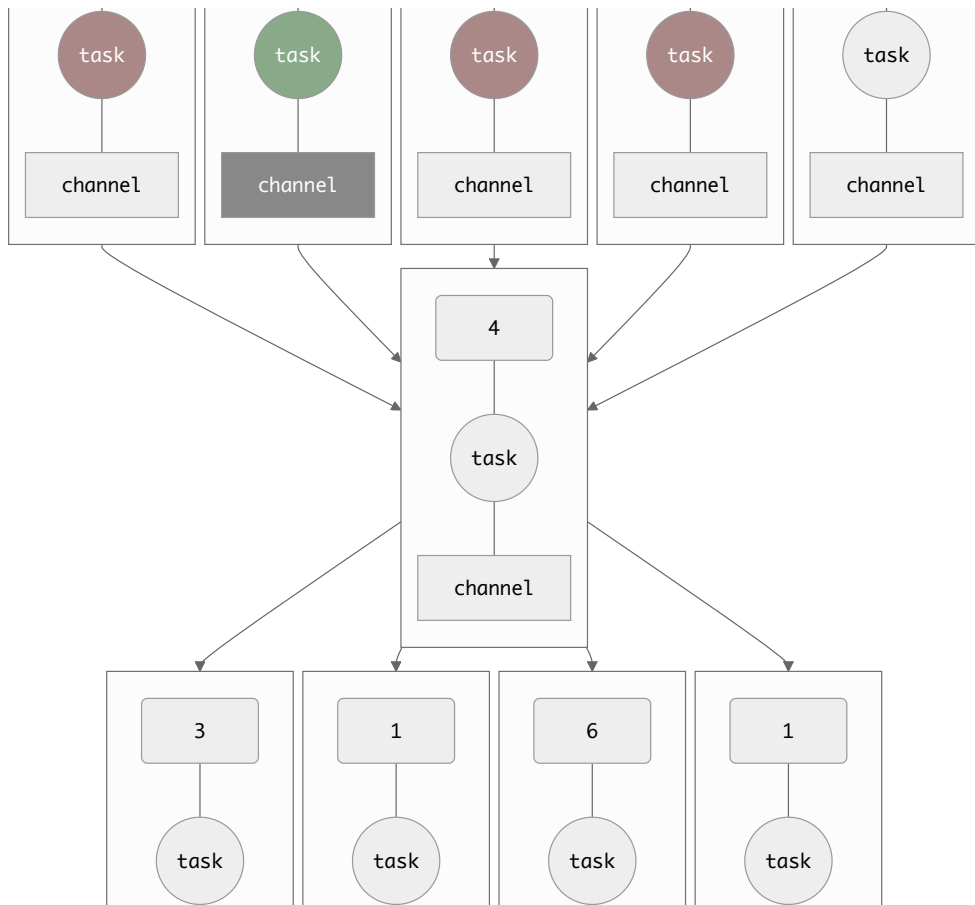
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



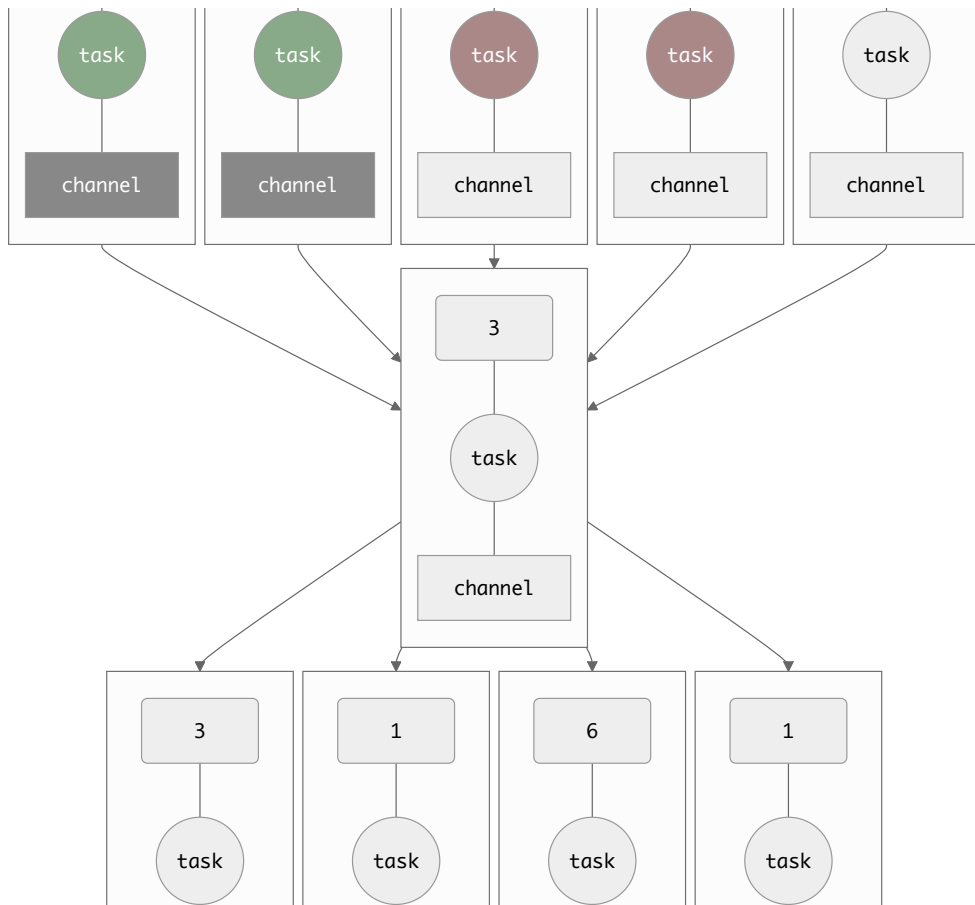
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



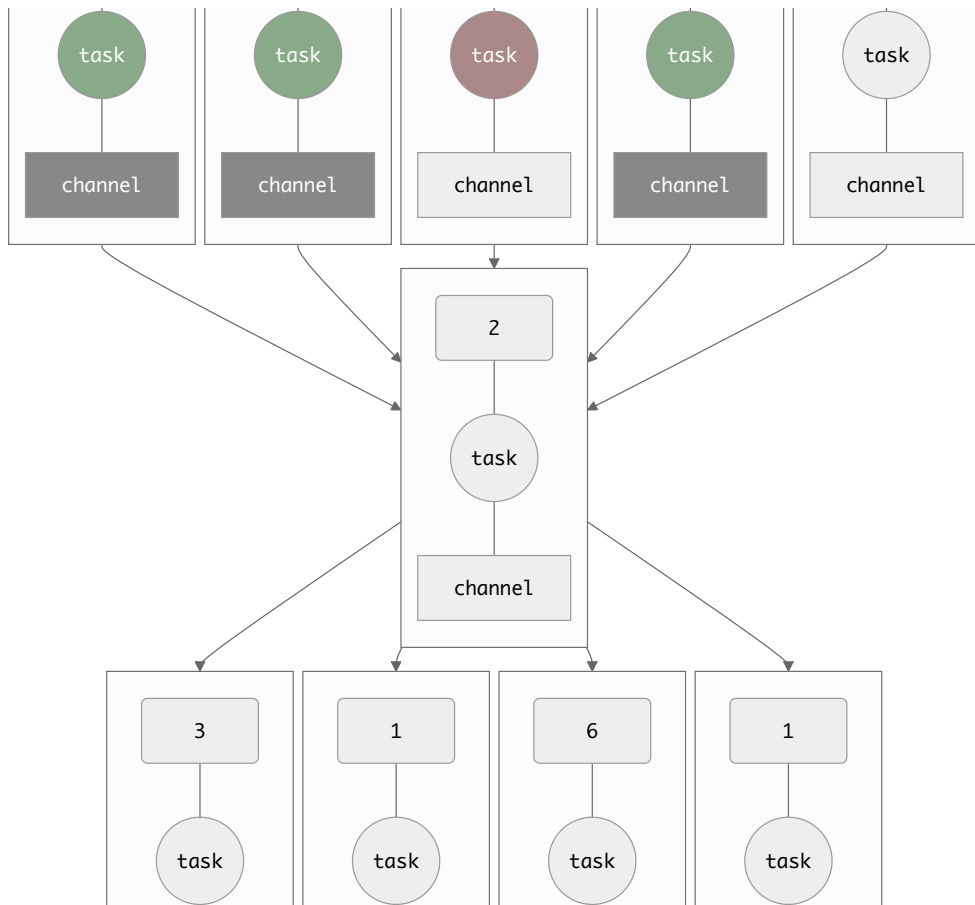
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



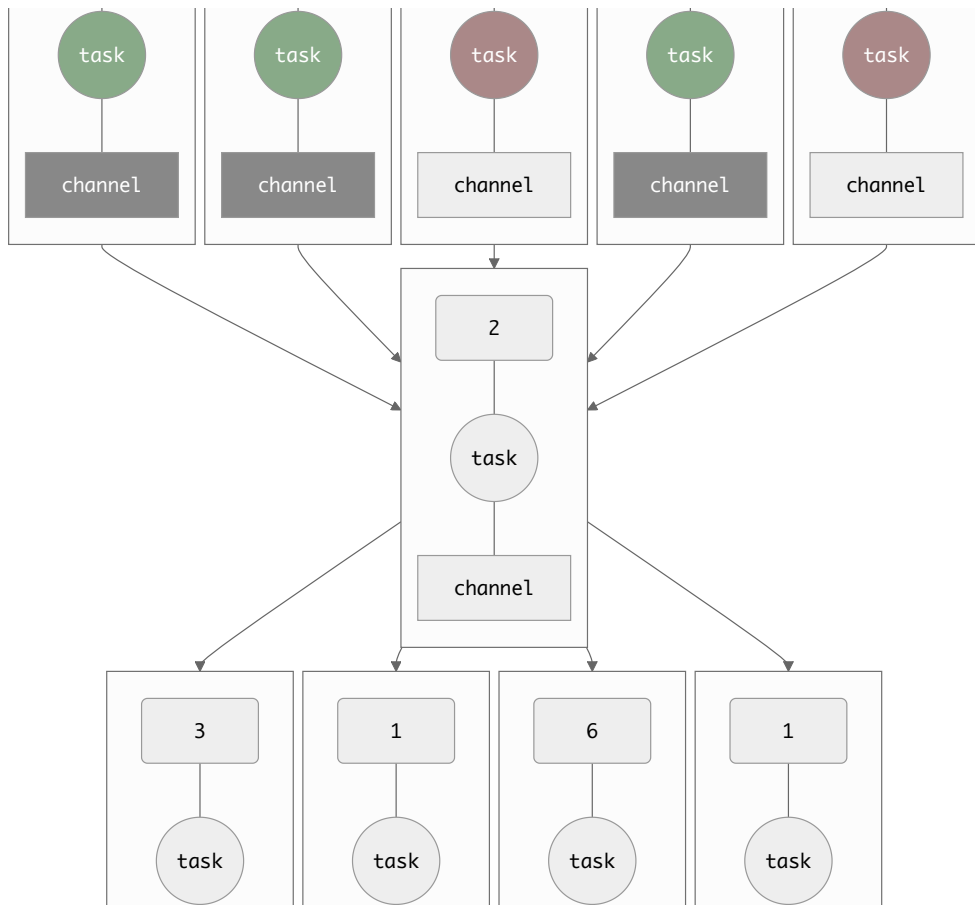
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



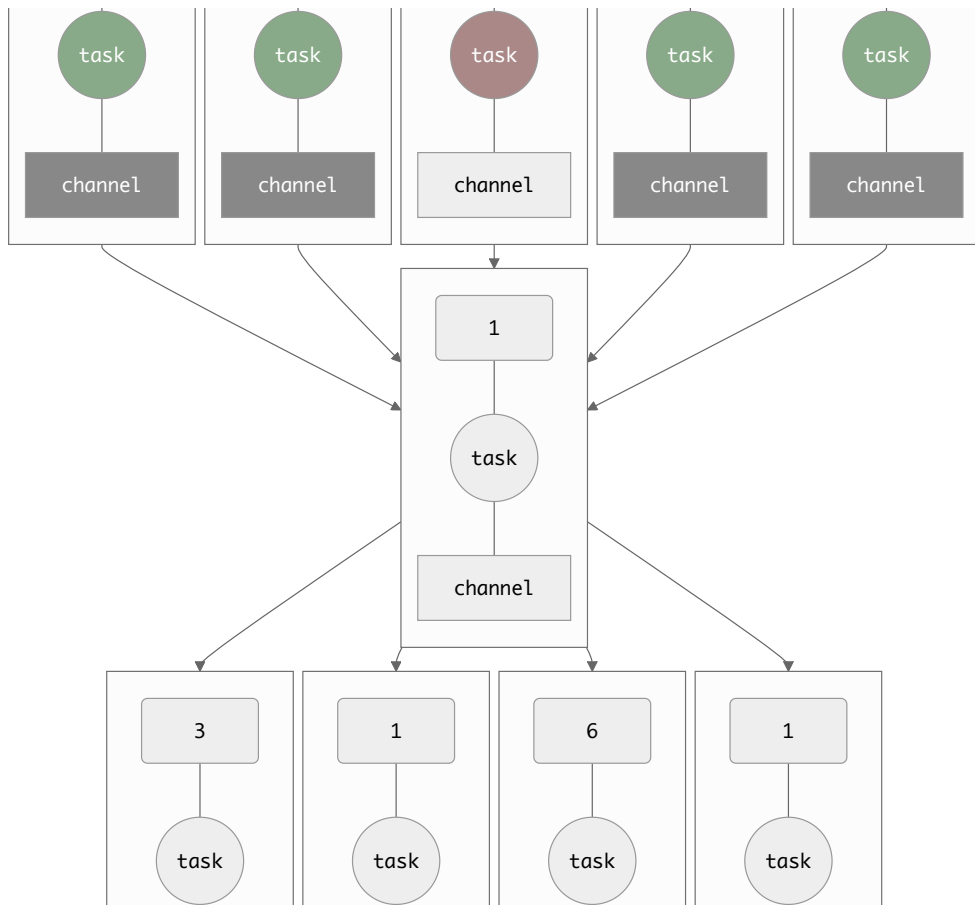
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



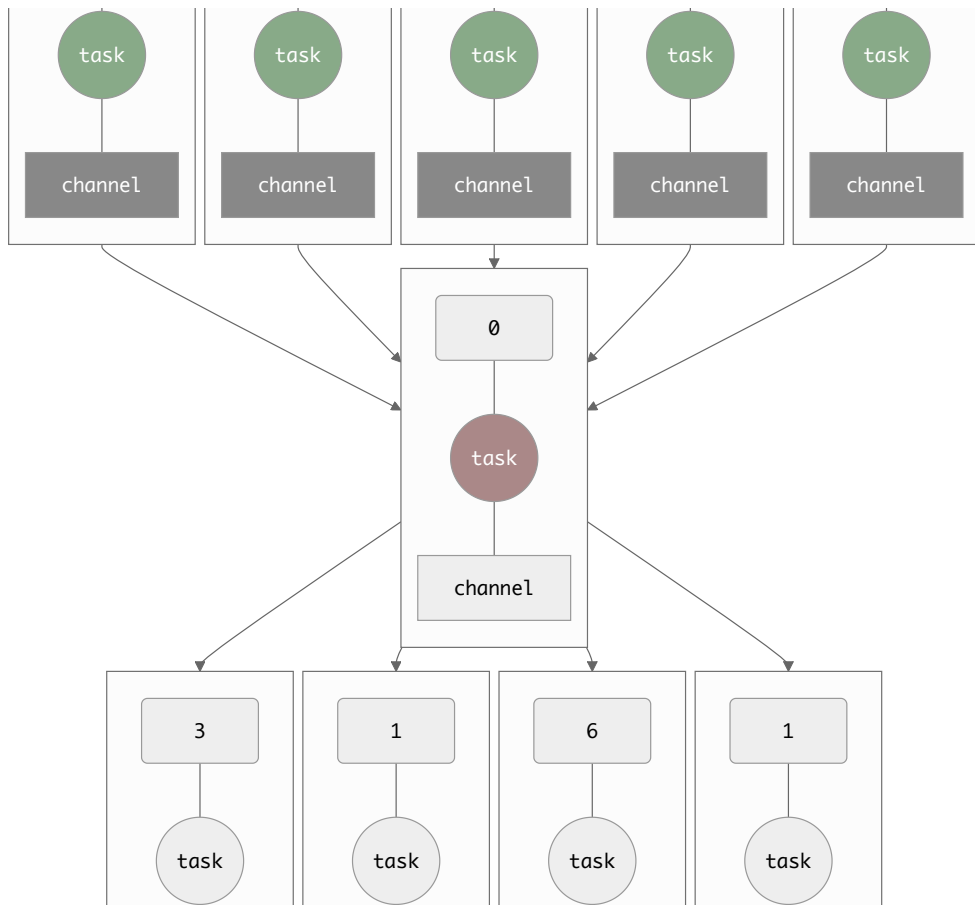
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



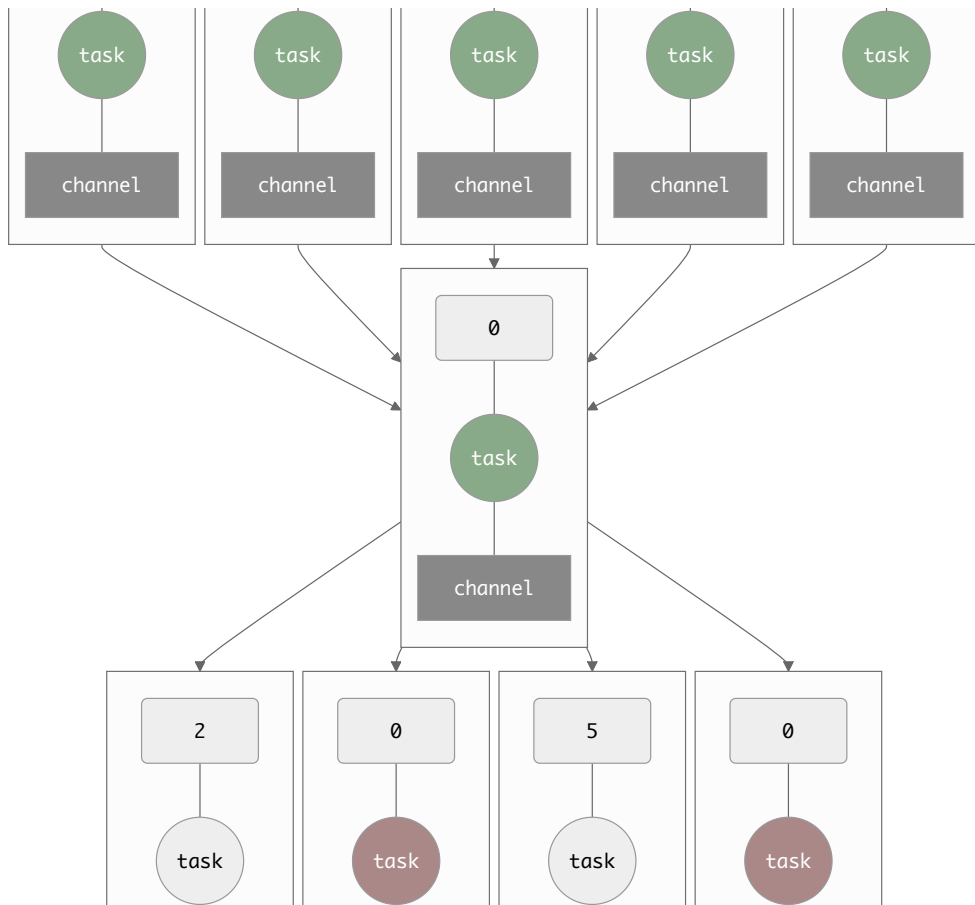
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



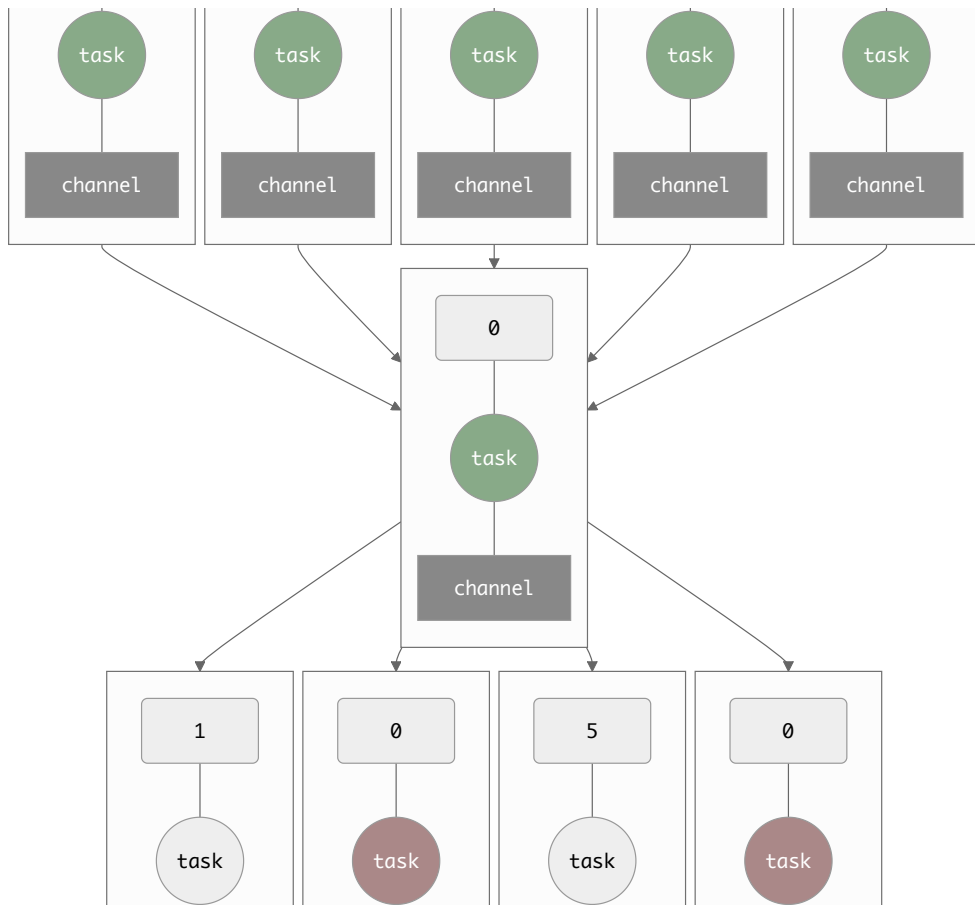
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



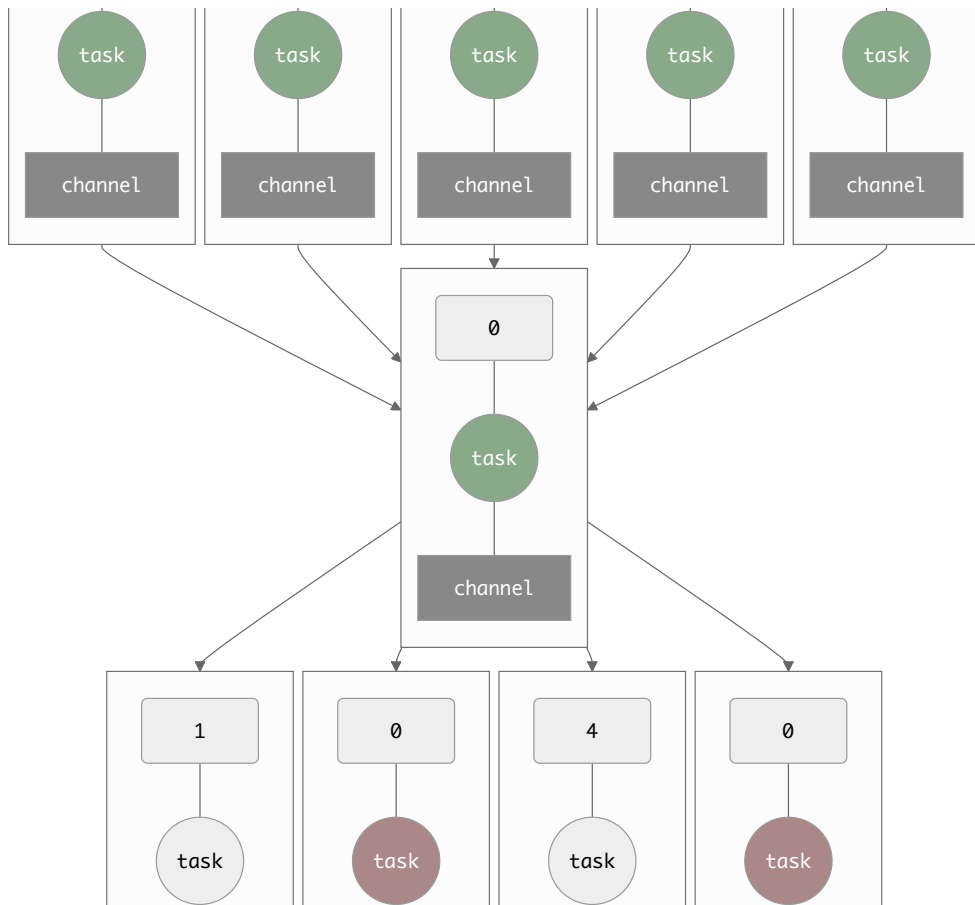
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



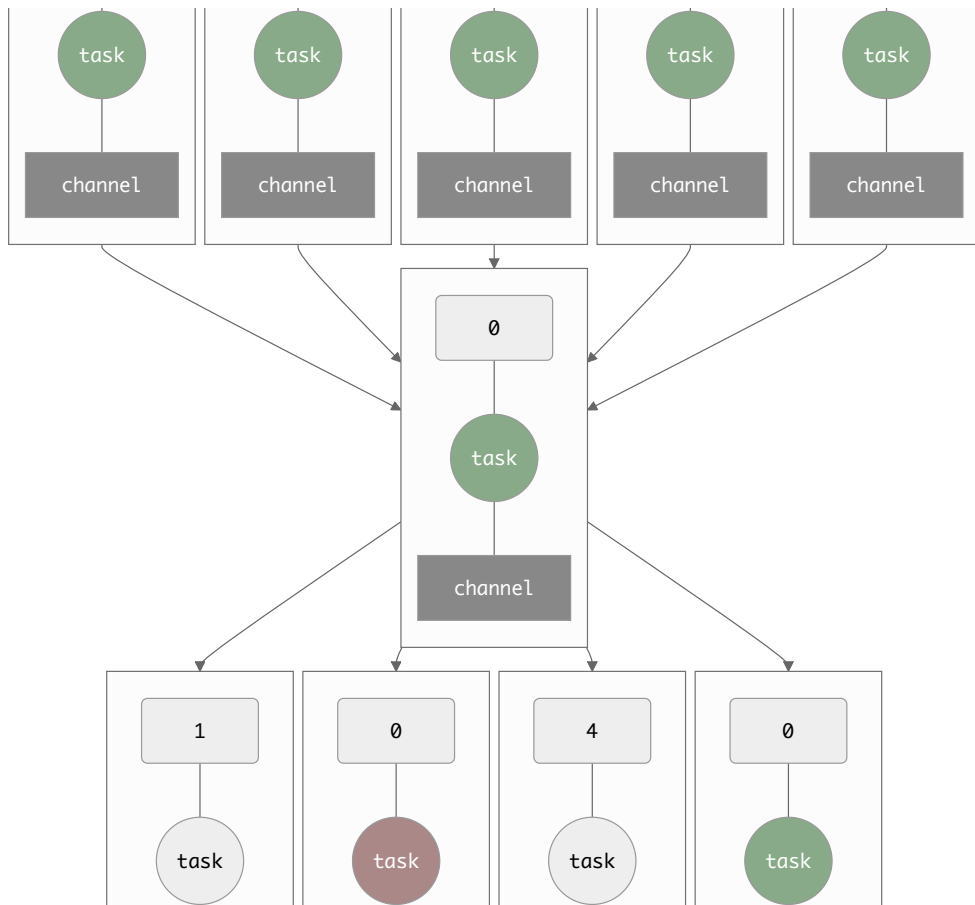
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



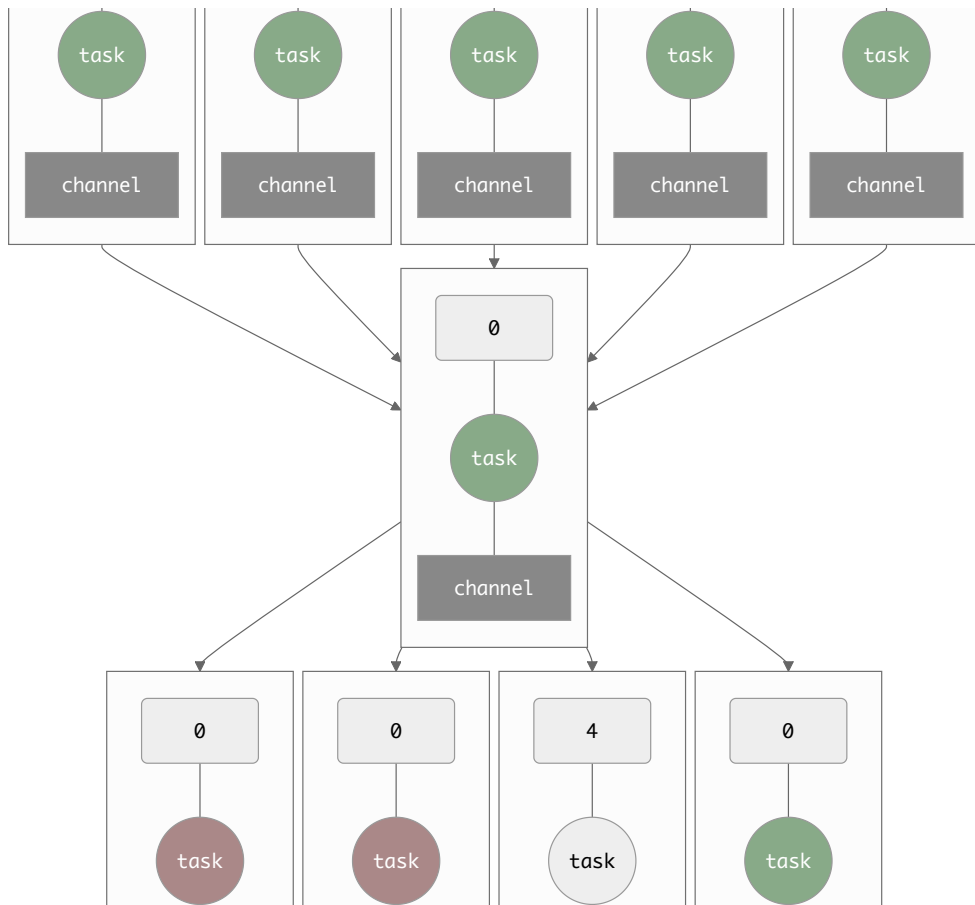
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



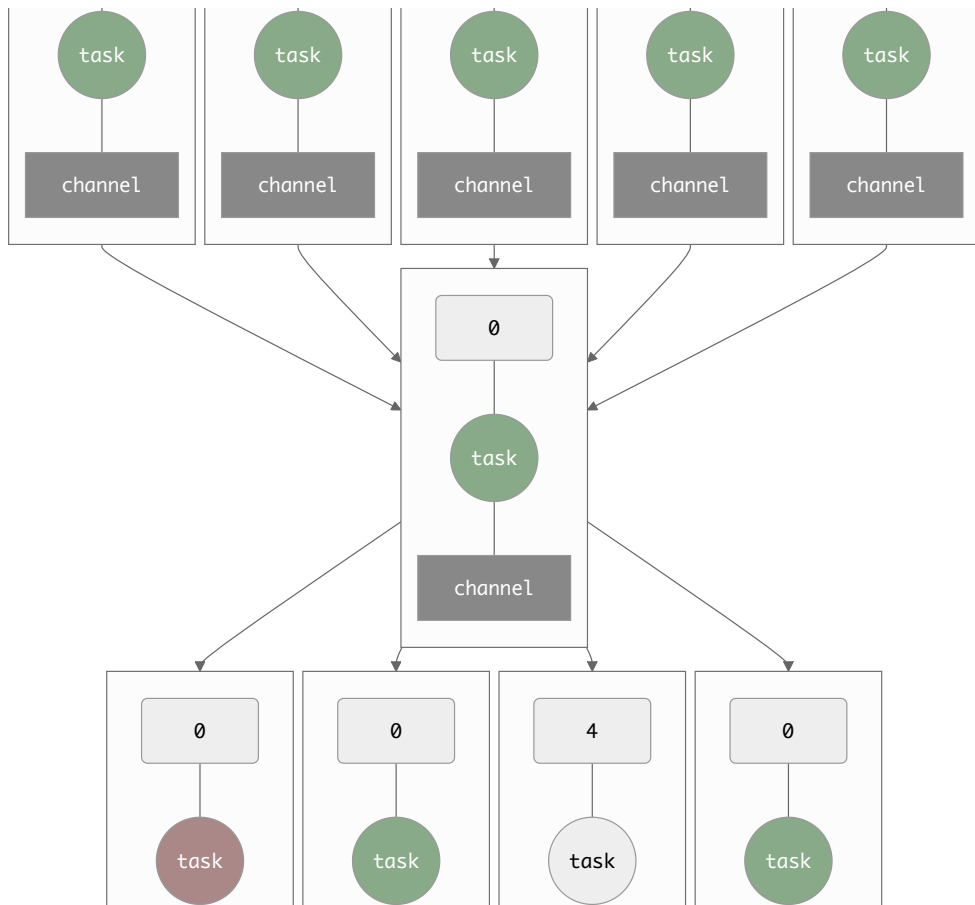
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



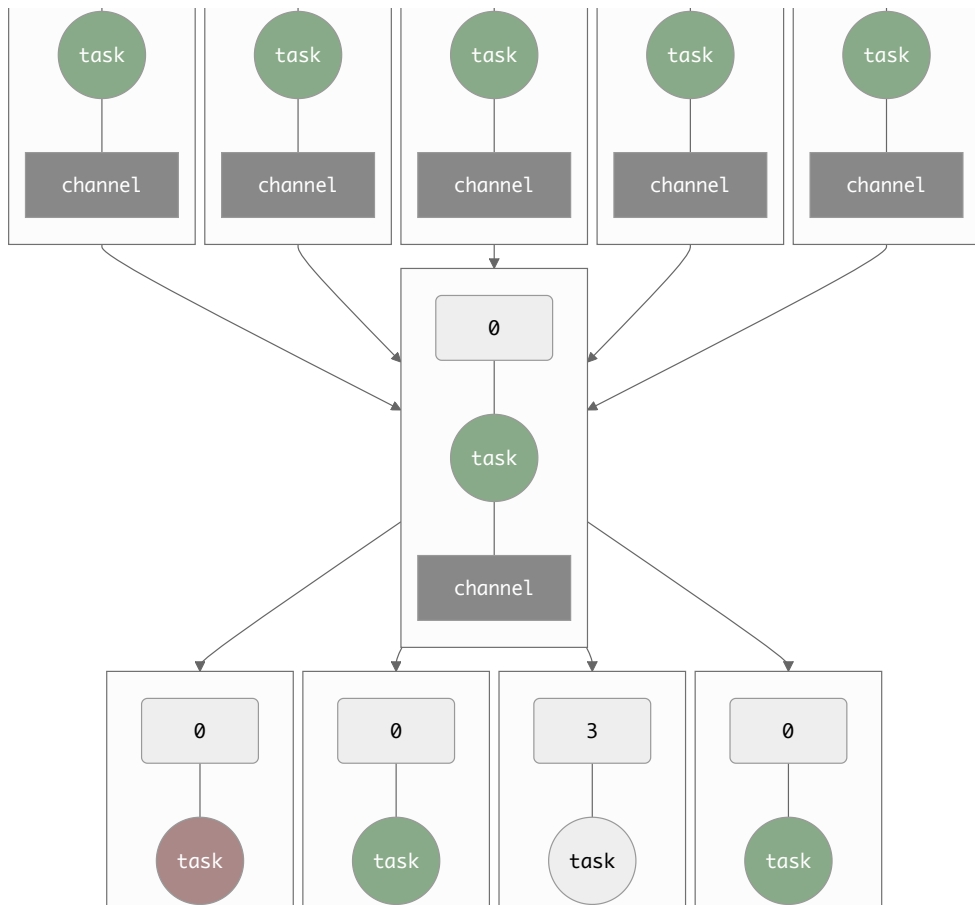
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



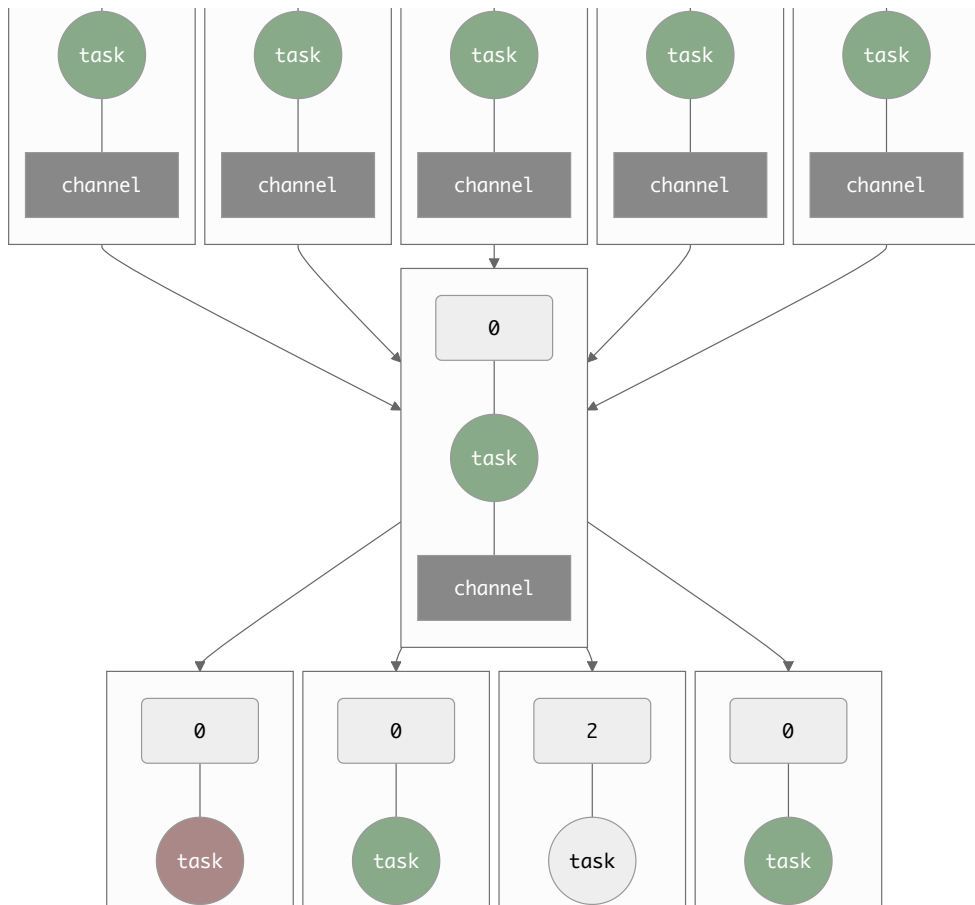
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



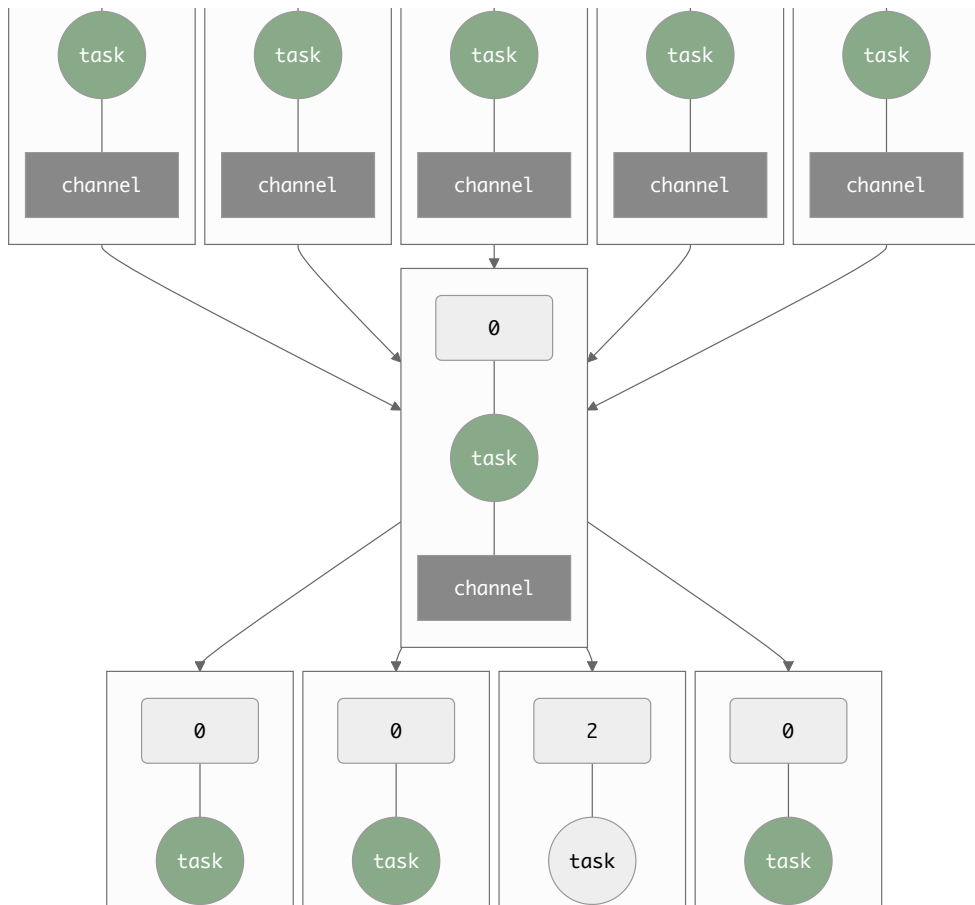
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



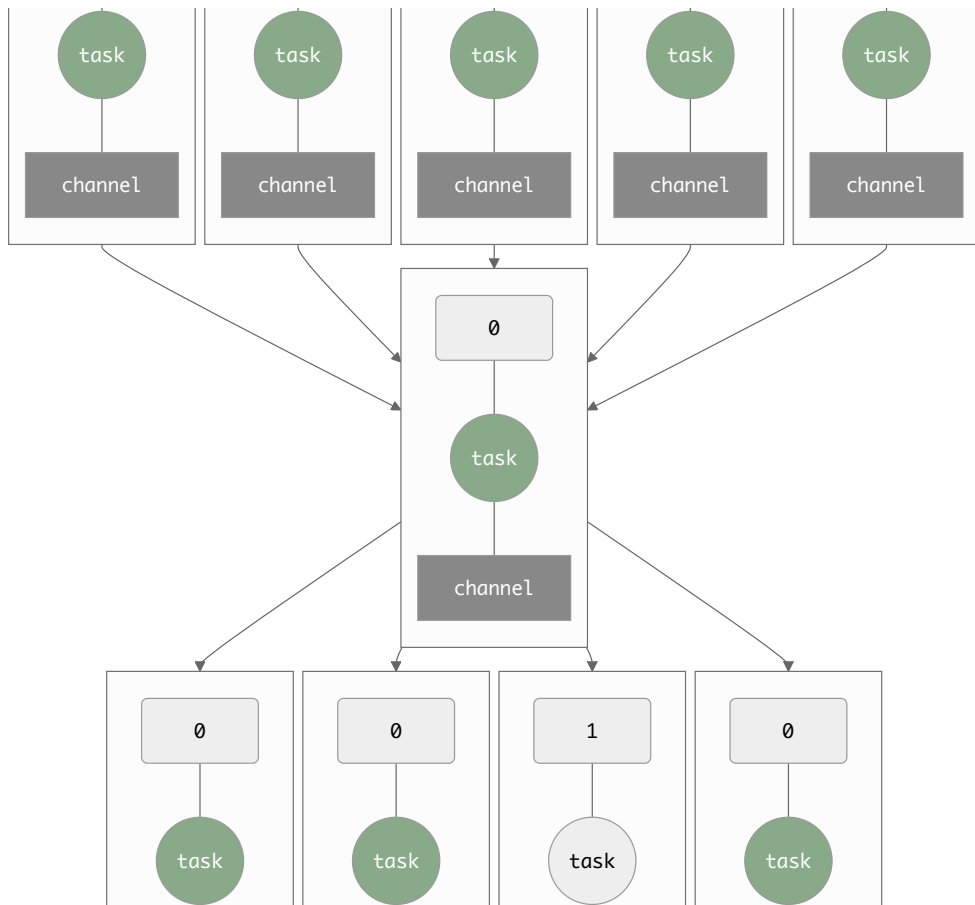
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



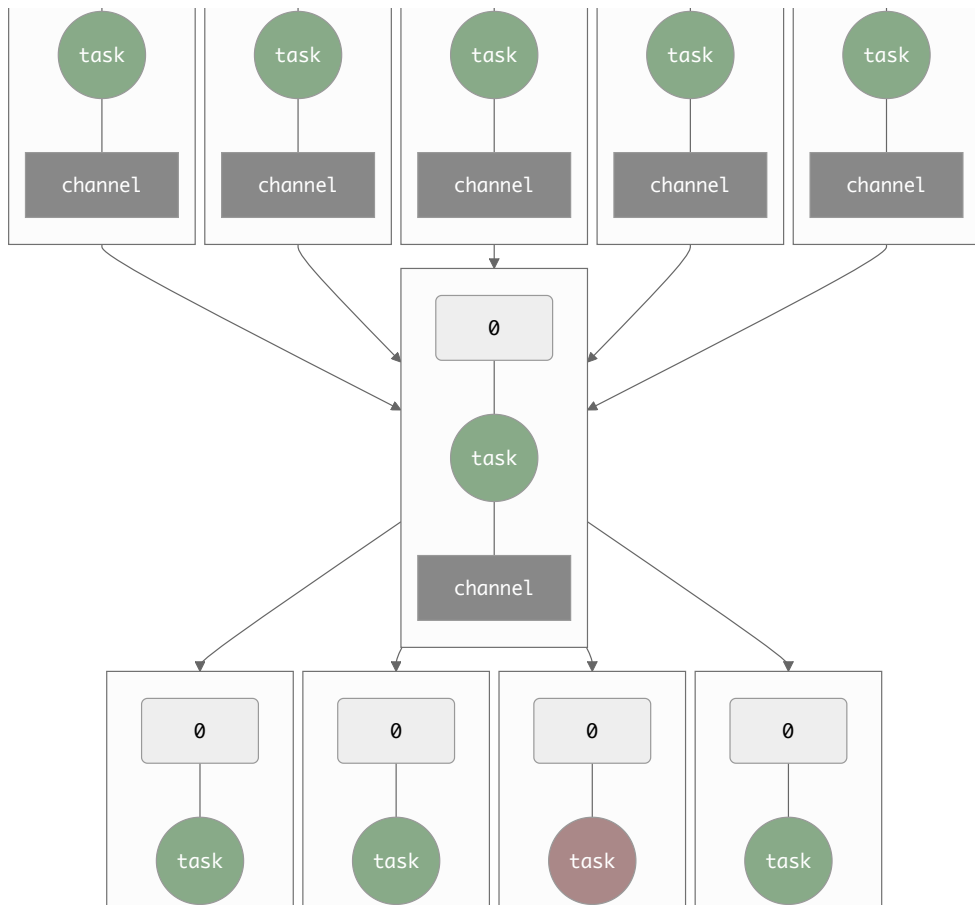
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



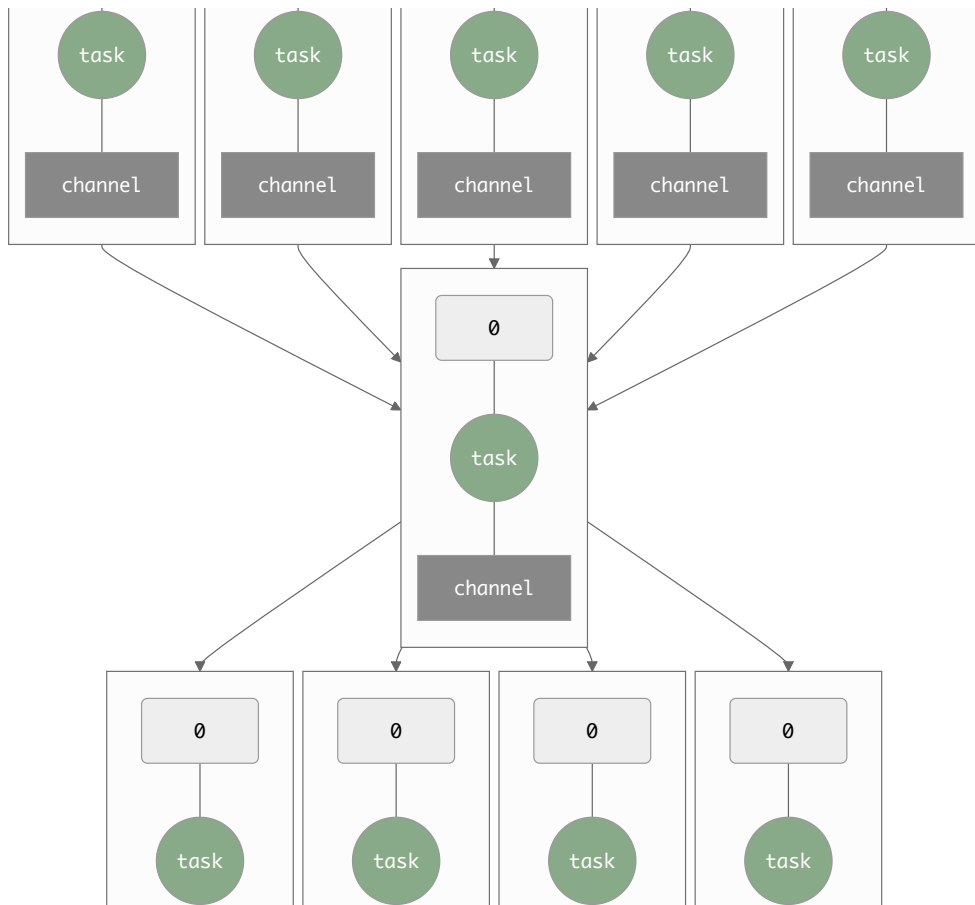
- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: synchronization



- *The output of a task is written into the communication channel, then is read by dependent tasks.*
- *A dependency counter synchronizes execution:*
 - *initially equals to the number of task's dependencies*
 - *is decremented when tasks that are dependencies have finished executing*
 - *triggers task execution when reaches zero*

tasking system: more info

tasking system: more info



sequence of synchronization primitives

sequence of synchronization primitives

A task graph must store dependency counters somehow:

```
class task_graph
{
    // ...
    std::vector<std::atomic_int> dependency_counters;
    // ...
};
```

sequence of synchronization primitives

A task graph must store dependency counters somehow:

```
class task_graph
{
    // ...
    std::vector<std::atomic_int> dependency_counters;
    // ...
};
```

Question: Is `std::vector<>` a viable choice?

sequence of synchronization primitives

A task graph must store dependency counters somehow:

```
class task_graph
{
    // ...
    std::vector<std::atomic_int> dependency_counters;
    // ...
};
```

Question: Is `std::vector<>` a viable choice?

```
auto create_vector (const std::size_t size)
{
    // Will this compile?
    //
    return std::vector<std::atomic_int>(size);
}
```

sequence of synchronization primitives

A task graph must store dependency counters somehow:

```
class task_graph
{
    // ...
    std::vector<std::atomic_int> dependency_counters;
    // ...
};
```

Question: Is `std::vector<>` a viable choice?

*Note: To avoid false sharing,
synchronization primitives should be aligned to
`std::hardware_destructive_interference_size` bytes.*

```
auto create_vector (const std::size_t size)
{
    // Will this compile?
    //
    return std::vector<std::atomic_int>(size);
}
```



```
#include <vector>
#include <atomic>

auto create_vector (const std::size_t size)
{
    return std::vector<std::atomic_int>(size);
}
```

```
create_vector(unsigned int):
```

```
    addi    sp,sp,-32
    sw      ra,28(sp)
    sw      s0,24(sp)
    li      a5,536870912
    bgeu    a1,a5,.L7
    mv      s0,a0
    beq     a1,zero,.L8
    slli    a2,a1,2
    mv      a0,a2
    sw      a2,12(sp)
    call    operator new(unsigned int)@plt
    lw      a2,12(sp)
    sw      a0,0(s0)
    li      a1,0
    add     a5,a0,a2
    sw      a5,8(s0)
    sw      a5,12(sp)
    call    memset@plt
    lw      a5,12(sp)
    lw      ra,28(sp)
    mv      a0,s0
    sw      a5,4(s0)
    lw      s0,24(sp)
    addi    sp,sp,32
    jr      ra

.L8:
    sw      zero,0(a0)
    sw      zero,8(a0)
    li      a5,0
    lw      ra,28(sp)
    sw      a5,4(s0)
    mv      a0,s0
    lw      s0,24(sp)
    addi    sp,sp,32
    jr      ra

.L7:
    lla     a0,.LC0
    call    std::__throw_length_error(char const*)@plt
```


std::vector<T,Allocator>::vector

```
explicit vector( size_type count,  
                const Allocator& alloc = Allocator() ); (1)
```

Constructs a new `vector`.

1) Constructs a `vector` with `count` default-inserted objects of `T`. No copies are made.

If `T` is not *DefaultInsertable* into `std::vector<T>`, the behavior is undefined.

C++ named requirements: *DefaultInsertable*

Specifies that an object of the type can be default-constructed in-place by a given allocator.

Requirements

Type	Definition
<code>T</code>	an object type
<code>A</code>	an allocator type
Value	Definition
<code>m</code>	an lvalue of type <code>A</code>
<code>p</code>	a pointer of type <code>T*</code>

If the expression `std::allocator_traits<A>::construct(m, p)` is well-formed, `T` is *DefaultInsertable*.

Notes

By default, this will [value-initialize](#) the object, as by `std::construct_at(p)`.

std::atomic<T>::atomic

```
constexpr atomic() noexcept(std::is_nothrow_default_constructible_v<T>); (1)
```

```
constexpr atomic( T desired ) noexcept; (2)
```

Constructs a new atomic variable.

- 1) Value-initializes the underlying object. The initialization is not atomic.
- 2) Initializes the underlying object with `desired`. The initialization is not atomic.

Doc No: WG21 P0883R2
Date: 2019-11-08
Reply to: Nicolai Josuttis

Fixing Atomic Initialization, Rev2

Motivation

Currently `std::atomic<>` is standardized to behave as follows:

```
std::atomic<int> x{}; // does NOT zero initialize
```

The reason is that the spec currently requires not to initialize any value inside a default-initialized atomic:

32.6.1 Operations on atomic types [atomics.types.operations]:

```
atomic() noexcept = default;
```

Effects: Leaves the atomic object in an uninitialized state.

[Note: These semantics ensure compatibility with C. —end note]

Proposed Wording

31.8.1 Operations on atomic types [atomics.types.operations]:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>) = default;
```

Mandates: `is_default_constructible_v<T>` is true.

Effects: ~~Leaves the atomic object in an uninitialized state.~~

~~[Note: These semantics ensure compatibility with C. —end note]~~

Initializes the atomic object with the value of `T()`. Initialization is not an atomic operation.

Doc No: WG21 P0883R2
Date: 2019-11-08
Reply to: Nicolai Josuttis

Fixing Atomic Initialization, Rev2

Motivation

Currently `std::atomic<>` is standardized to behave as follows:

```
std::atomic<int> x{}; // does NOT zero initialize
```

The reason is that the spec currently requires not to initialize any value inside a default-initialized atomic:

32.6.1 Operations on atomic types [atomics.types.operations]:

```
atomic() noexcept = default;
```

Effects: Leaves the atomic object in an uninitialized state.

[Note: These semantics ensure compatibility with C. —end note]

until C++17

Proposed Wording

31.8.1 Operations on atomic types [atomics.types.operations]:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>) = default;
```

Mandates: `is_default_constructible_v<T>` is true.

Effects: ~~Leaves the atomic object in an uninitialized state.~~

~~[Note: These semantics ensure compatibility with C. —end note]~~

Initializes the atomic object with the value of `T()`. Initialization is not an atomic operation.

Doc No: WG21 P0883R2
Date: 2019-11-08
Reply to: Nicolai Josuttis

Fixing Atomic Initialization, Rev2

Motivation

Currently `std::atomic<>` is standardized to behave as follows:

```
std::atomic<int> x{}; // does NOT zero initialize
```

The reason is that the spec currently requires not to initialize any value inside a default-initialized atomic:

32.6.1 Operations on atomic types [atomics.types.operations]:

```
atomic() noexcept = default;
```

Effects: Leaves the atomic object in an uninitialized state.

[Note: These semantics ensure compatibility with C. —end note]

until C++17

Proposed Wording

31.8.1 Operations on atomic types [atomics.types.operations]:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>) = default;
```

Mandates: `is_default_constructible_v<T>` is true.

Effects: ~~Leaves the atomic object in an uninitialized state.~~

~~[Note: These semantics ensure compatibility with C. —end note]~~

Initializes the atomic object with the value of `T()`. Initialization is not an atomic operation.

Doc No: WG21 P0883R2
Date: 2019-11-08
Reply to: Nicolai Josuttis

Fixing Atomic Initialization, Rev2

Motivation

Currently `std::atomic<>` is standardized to behave as follows:

```
std::atomic<int> x{}; // does NOT zero initialize
```

The reason is that the spec currently requires not to initialize any value inside a default-initialized atomic:

32.6.1 Operations on atomic types [atomics.types.operations]:

```
atomic() noexcept = default;
```

Effects: Leaves the atomic object in an uninitialized state.

[Note: ~~These semantics ensure compatibility with C.~~ —end note]

until C++17

Proposed Wording

31.8.1 Operations on atomic types [atomics.types.operations]:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>) = default;
```

Mandates: `is_default_constructible_v<T>` is true.

Effects: ~~Leaves the atomic object in an uninitialized state.~~

~~[Note: These semantics ensure compatibility with C. —end note]~~

Initializes the atomic object with the value of `T()`. Initialization is not an atomic operation.

since C++20

Doc No: WG21 P0883R2
Date: 2019-11-08
Reply to: Nicolai Josuttis

Fixing Atomic Initialization, Rev2

Motivation

Currently `std::atomic<>` is standardized to behave as follows:

```
std::atomic<int> x{}; // does NOT zero initialize
```

The reason is that the spec currently requires not to initialize any value inside a default-initialized atomic:

32.6.1 Operations on atomic types [atomics.types.operations]:

```
atomic() noexcept = default;
```

Effects: Leaves the atomic object in an uninitialized state.

[Note: ~~These semantics ensure compatibility with C. —end note~~]

until C++17

Proposed Wording

31.8.1 Operations on atomic types [atomics.types.operations]:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>) = default;
```

Mandates: `is_default_constructible_v<T>` is true.

Effects: ~~Leaves the atomic object in an uninitialized state.~~

~~[Note: These semantics ensure compatibility with C. —end note]~~

Initializes the atomic object with the value of T(). Initialization is not an atomic operation.

since C++20

sequence of synchronization primitives

sequence of synchronization primitives

Initially each dependency counter must equal to the number of dependencies of a given task.

sequence of synchronization primitives

Initially each dependency counter must equal to the number of dependencies of a given task.

Therefore, each `std::atomic_int` must be initialized to a specific value.

sequence of synchronization primitives

Initially each dependency counter must equal to the number of dependencies of a given task.

Therefore, each `std::atomic_int` must be initialized to a specific value.

```
auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int> { };

    dependency_counters.reserve(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters.emplace_back(number_of_dependencies(task_index));
    }

    return dependency_counters; // sorted by task_index
}
```



```
#include <vector>
#include <atomic>
#include <ranges>

auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>();

    dependency_counters.reserve(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters.emplace_back(number_of_dependencies(task_index));
    }

    return dependency_counters; // sorted by task_index
}
```

```
error: static assertion failed due to requirement
    '__is_cpp17_move_insertable_v<std::allocator<std::atomic<int>>>':
    The specified type does not meet
    the requirements of Cpp17MoveInsertable.
    | static_assert(__is_cpp17_move_insertable_v<Alloc>);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
error: no matching function for call to '__construct_at'
    | std::__construct_at(__p, std::forward<Args>(__args)...);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | allocator_traits<Alloc>::construct(__alloc,
    |                                     std::__to_address(__result),
    |                                     std::move_if_noexcept(*__iter));
    | ^
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
note: candidate template ignored: substitution failure
    [with _Tp = std::atomic<int>, _Args = <std::atomic<int>>]:
    call to implicitly-deleted copy constructor of 'std::atomic<int>'
    | template <class _Tp, class... _Args, class =
    | decltype(::new(std::declval<void*>()) _Tp(std::declval<Args>()...))>
    | _Tp* __construct_at(_Tp* __location, _Args&&... __args) {
    | ^
```



```
#include <vector>
#include <atomic>
#include <ranges>

auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>();

    dependency_counters.reserve(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters.emplace_back(number_of_dependencies(task_index));
    }

    return dependency_counters; // sorted by task_index
}
```

```
error: static assertion failed due to requirement
    '__is_cpp17_move_insertable_v<std::allocator<std::atomic<int>>>':
    The specified type does not meet
    the requirements of Cpp17MoveInsertable.
    | static_assert(__is_cpp17_move_insertable_v<Alloc>);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
error: no matching function for call to '__construct_at'
    | std::__construct_at(__p, std::forward<Args>(__args)...);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | allocator_traits<Alloc>::construct(__alloc,
    |                                     std::__to_address(__result),
    |                                     std::move_if_noexcept(*__iter));
    | ^
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
note: candidate template ignored: substitution failure
    [with _Tp = std::atomic<int>, _Args = <std::atomic<int>>]:
    call to implicitly-deleted copy constructor of 'std::atomic<int>'
    | template <class _Tp, class... _Args, class =
    | decltype(::new(std::declval<void*>()) _Tp(std::declval<Args>()...))>
    | _Tp* __construct_at(_Tp* __location, _Args&&... __args) {
    | ^
```



```
#include <vector>
#include <atomic>
#include <ranges>

auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>();

    dependency_counters.reserve(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters.emplace_back(number_of_dependencies(task_index));
    }

    return dependency_counters; // sorted by task_index
}
```

```
error: static assertion failed due to requirement
    '__is_cpp17_move_insertable_v<std::allocator<std::atomic<int>>>':
    The specified type does not meet
    the requirements of Cpp17MoveInsertable.
    | static_assert(__is_cpp17_move_insertable_v<Alloc>);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
error: no matching function for call to '__construct_at'
    | std::__construct_at(__p, std::forward<Args>(__args)...);
    | ^~~~~~
note: in instantiation of function template specialization requested here
    | allocator_traits<Alloc>::construct(__alloc,
    |                                     std::__to_address(__result),
    |                                     std::move_if_noexcept(*__iter));
    | ^
note: in instantiation of function template specialization requested here
    | std::__uninitialized_allocator_relocate(
    | ^
note: in instantiation of member function requested here
    | __swap_out_circular_buffer(__v);
    | ^
note: in instantiation of member function requested here
    | dependency_counters.reserve(task_count);
    | ^
note: candidate template ignored: substitution failure
    [with _Tp = std::atomic<int>, _Args = <std::atomic<int>>]:
    call to implicitly-deleted copy constructor of 'std::atomic<int>'
    | template <class _Tp, class... _Args, class =
    | decltype(::new(std::declval<void*>()) _Tp(std::declval<Args>()...))>
    | _Tp* __construct_at(_Tp* __location, _Args&&... __args) {
    | ^
```

sequence of synchronization primitives

sequence of synchronization primitives

*Even though, the intention is to only allocate the storage for the elements,
and then construct them in the allocated storage,
without any moving or copying,*

sequence of synchronization primitives

*Even though, the intention is to only allocate the storage for the elements,
and then construct them in the allocated storage,
without any moving or copying,
the inactive code path in which the reallocation takes place must also compile succesfully,
and that requires MoveInsertable elements.*

sequence of synchronization primitives

Even though, the intention is to only allocate the storage for the elements, and then construct them in the allocated storage, without any moving or copying, the inactive code path in which the reallocation takes place must also compile succesfully, and that requires MoveInsertable elements.

In short, since both `.reserve()` and `.emplace_back()` might trigger reallocation they do not support immovable elements.

sequence of synchronization primitives

Even though, the intention is to only allocate the storage for the elements, and then construct them in the allocated storage, without any moving or copying, the inactive code path in which the reallocation takes place must also compile successfully, and that requires MoveInsertable elements.

In short, since both `.reserve()` and `.emplace_back()` might trigger reallocation they do not support immovable elements.

This makes sense – immovable objects cannot be relocated to a different memory location, that is, change their address, and therefore storage containing such objects cannot be reallocated.

sequence of synchronization primitives

Even though, the intention is to only allocate the storage for the elements, and then construct them in the allocated storage, without any moving or copying, the inactive code path in which the reallocation takes place must also compile successfully, and that requires MoveInsertable elements.

In short, since both `.reserve()` and `.emplace_back()` might trigger reallocation they do not support immovable elements.

This makes sense – immovable objects cannot be relocated to a different memory location, that is, change their address, and therefore storage containing such objects cannot be reallocated.

The `std::atomic_int` does not satisfy the MoveInsertable named requirements.

std::vector<T,Allocator>::reserve

```
void reserve( size_type new_capacity );
```

Increase the capacity of the vector to a value that's greater or equal to `new_capacity`.

If `new_capacity` is greater than the current `capacity()`,
new storage is allocated, otherwise the function does nothing.

After a call to `reserve()`, insertions will not trigger reallocation
unless the insertion would make the size of the vector greater than `new_capacity`.

Type requirements

- `T` must meet the requirements of *MoveInsertable*.

std::vector<T,Allocator>::emplace_back

```
template< class... Args >  
reference emplace_back( Args&&... args );
```

Appends a new element to the end of the container.

If after the operation the new `size()` is greater than old `capacity()` a reallocation takes place, (...).

Type requirements

- `T` must meet the requirements of [MoveInsertable](#) and [EmplaceConstructible](#).

Notes

Since reallocation may take place, `emplace_back` requires the element type to be [MoveInsertable](#).

C++ named requirements: *MoveInsertable*

Specifies that an object of the type can be constructed into uninitialized storage from an rvalue of that type by a given allocator.

Requirements

Type	Definition
<code>T</code>	an object type
<code>A</code>	an allocator type
Value	Definition
<code>m</code>	an lvalue of type <code>A</code>
<code>p</code>	a pointer of type <code>T*</code>
Expression	Definition
<code>rv</code>	an expression denoting an rvalue of type <code>T</code>
<code>expr</code>	<code>std::allocator_traits<A>::construct(m, p, rv)</code>

Notes

If `A` is `std::allocator<T>`, then this will call placement `new`, as by `std::construct_at(p, rv)`. This effectively requires `T` to be move constructible.

std::atomic<T>::atomic

```
constexpr atomic() noexcept(std::is_nothrow_default_constructible_v<T>); (1)
```

```
constexpr atomic( T desired ) noexcept; (2)
```

```
atomic( const atomic& ) = delete; (3)
```

Constructs a new atomic variable.

- 1) Value-initializes the underlying object. The initialization is not atomic.
- 2) Initializes the underlying object with `desired`. The initialization is not atomic.
- 3) Atomic variables are not *CopyConstructible*.

`std::atomic<T>::atomic`

```
constexpr atomic() noexcept(std::is_nothrow_default_constructible_v<T>); (1)
```

```
constexpr atomic( T desired ) noexcept; (2)
```

```
atomic( const atomic& ) = delete; (3)
```

Constructs a new atomic variable.

- 1) Value-initializes the underlying object. The initialization is not atomic.
- 2) Initializes the underlying object with `desired`. The initialization is not atomic.
- 3) Atomic variables are not *CopyConstructible*.

`std::atomic<T>::atomic`

```
constexpr atomic() noexcept(std::is_nothrow_default_constructible_v<T>); (1)
```

```
constexpr atomic( T desired ) noexcept; (2)
```

```
atomic( const atomic& ) = delete; (3)
```

Constructs a new atomic variable.

- 1) Value-initializes the underlying object. The initialization is not atomic.
- 2) Initializes the underlying object with `desired`. The initialization is not atomic.
- 3) Atomic variables are not [CopyConstructible](#).

sequence of synchronization primitives

sequence of synchronization primitives

As a last resort, the dependency counters can be created in two steps:

sequence of synchronization primitives

As a last resort, the dependency counters can be created in two steps:

```
auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters[task_index].store(number_of_dependencies(task_index),
                                                std::memory_order_relaxed);
    }

    return dependency_counters; // sorted by task_index
}
```

sequence of synchronization primitives

As a last resort, the dependency counters can be created in two steps:

```
auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters[task_index].store(number_of_dependencies(task_index),
                                                std::memory_order_relaxed);
    }

    return dependency_counters; // sorted by task_index
}
```

However, the above function is suboptimal, because it initially fills the vector with zeros and then overwrites it, using atomic store operations.

sequence of synchronization primitives

As a last resort, the dependency counters can be created in two steps:

```
auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters[task_index].store(number_of_dependencies(task_index),
                                                std::memory_order_relaxed);
    }

    return dependency_counters; // sorted by task_index
}
```

However, the above function is suboptimal, because it initially fills the vector with zeros and then overwrites it, using atomic store operations.

Ideally, the dependency counters would be initialized with the correct values.



```
#include <vector>
#include <atomic>
#include <ranges>

auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters[task_index].store(number_of_dependencies(task_index),
                                                std::memory_order_relaxed);
    }

    return dependency_counters; // sorted by task_index
}
```

```
create_dependency_counters(unsigned long):
```

```
    addi    sp,sp,-64
    li      a5,-1
    sd      ra,56(sp)
    srli    a5,a5,3
    bgtu    a1,a5,.L15
    sd      s2,32(sp)
    sd      s3,24(sp)
    mv      s2,a0
    mv      s3,a1
    beq     a1,zero,.L16
    sd      s0,48(sp)
    slli    s0,a1,2
    mv      a0,s0
    sd      s1,40(sp)
    call    operator new(unsigned long)@plt
    add     s1,a0,s0
    mv      a2,s0
    sd      a0,0(s2)
    sd      s1,16(s2)
    li      a1,0
    sd      a0,8(sp)
    call    memset@plt
    ld      a5,8(sp)
    sd      s1,8(s2)
    li      s0,0
    j       .L5

.L17:
    ld      a5,0(s2)

.L5:
    slli    a4,s0,2
    mv      a0,s0
    add     s1,a5,a4
    call    number_of_dependencies(unsigned long)@plt
    sw      a0,0(s1)
    addi    s0,s0,1
```

```
#include <vector>
#include <atomic>
#include <ranges>

auto number_of_dependencies (std::size_t task_index) -> int;

auto create_dependency_counters (const std::size_t task_count)
{
    auto dependency_counters = std::vector<std::atomic_int>(task_count);

    for (const auto task_index : std::views::indices(task_count))
    {
        dependency_counters[task_index].store(number_of_dependencies(task_index),
                                                std::memory_order_relaxed);
    }

    return dependency_counters; // sorted by task_index
}
```

```

slli    a5,a5,3
bgtru   a1,a5,.L15
sd      s2,32(sp)
sd      s3,24(sp)
mv      s2,a0
mv      s3,a1
beq     a1,zero,.L16
sd      s0,48(sp)
slli    s0,a1,2
mv      a0,s0
sd      s1,40(sp)
call    operator new(unsigned long)@plt
add     s1,a0,s0
mv      a2,s0
sd      a0,0(s2)
sd      s1,16(s2)
li      a1,0
sd      a0,8(sp)
call    memset@plt
ld      a5,8(sp)
sd      s1,8(s2)
li      s0,0
j       .L5

.L17:
ld      a5,0(s2)

.L5:
slli    a4,s0,2
mv      a0,s0
add     s1,a5,a4
call    number_of_dependencies(unsigned long)@plt
sw      a0,0(s1)
addi    s0,s0,1
bne     s3,s0,.L17
ld      s0,48(sp)
ld      ra,56(sp)
ld      s1,40(sp)
ld      s3,24(sp)

```

the interface of a dependency counter

the interface of a dependency counter

For the best ergonomics and performance, an event counter should have the following interface, providing only a single operation named `.count_down()` :

the interface of a dependency counter

For the best ergonomics and performance, an event counter should have the following interface, providing only a single operation named `.count_down()` :

```
class event_counter
{
    explicit
    constexpr event_counter (std::ptrdiff_t initial_value) noexcept;
    constexpr ~event_counter () noexcept;

    event_counter (const event_counter&) = delete;
    auto operator = (const event_counter&) -> event_counter& = delete;

    auto count_down (std::ptrdiff_t update = 1) noexcept -> bool;
};
```

the interface of a dependency counter

For the best ergonomics and performance, an event counter should have the following interface, providing only a single operation named `.count_down()` :

```
class event_counter
{
    explicit
    constexpr event_counter (std::ptrdiff_t initial_value) noexcept;
    constexpr ~event_counter () noexcept;

    event_counter (const event_counter&) = delete;
    auto operator = (const event_counter&) -> event_counter& = delete;

    auto count_down (std::ptrdiff_t update = 1) noexcept -> bool;
};
```

Essentially, once an `event_counter` is created with its initial value, it can only be decremented or destroyed.

the interface of a dependency counter

For the best ergonomics and performance, an event counter should have the following interface, providing only a single operation named `.count_down()` :

```
class event_counter
{
    explicit
    constexpr event_counter (std::ptrdiff_t initial_value) noexcept;
    constexpr ~event_counter () noexcept;

    event_counter (const event_counter&) = delete;
    auto operator = (const event_counter&) -> event_counter& = delete;

    auto count_down (std::ptrdiff_t update = 1) noexcept -> bool;
};
```

Essentially, once an `event_counter` is created with its initial value, it can only be decremented or destroyed.

Additionally, `.count_down()` informs an execution thread whether an event have just occurred (meaning the `event_counter` have just reached zero) or not.

the interface of a dependency counter

For the best ergonomics and performance, an event counter should have the following interface, providing only a single operation named `.count_down()` :

```
class event_counter
{
    explicit
    constexpr event_counter (std::ptrdiff_t initial_value) noexcept;
    constexpr ~event_counter () noexcept;

    event_counter (const event_counter&) = delete;
    auto operator = (const event_counter&) -> event_counter& = delete;

    auto count_down (std::ptrdiff_t update = 1) noexcept -> bool;
};
```

A dependency counter is an event counter for which an event occurs when all tasks that are dependencies of a given task have finished executing.

Essentially, once an `event_counter` is created with its initial value, it can only be decremented or destroyed.

Additionally, `.count_down()` informs an execution thread whether an event have just occurred (meaning the `event_counter` have just reached zero) or not.

the interface of a dependency counter

the interface of a dependency counter

*The interface of `std::atomic_int` is designed to mimic the built-in `int` ,
and therefore using it as a dependency counter can be quite error-prone.*

the interface of a dependency counter

*The interface of `std::atomic_int` is designed to mimic the built-in `int` ,
and therefore using it as a dependency counter can be quite error-prone.*

*Of course, one can use the `jsc::event_counter` freely available in the JSC C++ Library,
which implements the described interface quite nicely.*

the interface of a dependency counter

The interface of `std::atomic_int` is designed to mimic the built-in `int` , and therefore using it as a dependency counter can be quite error-prone.

Of course, one can use the `jsc::event_counter` freely available in the JSC C++ Library, which implements the described interface quite nicely.

However, the `std::latch` from the C++ Concurrency Library can also be used, since it provides quite similar interface.

the interface of a dependency counter

The interface of `std::atomic_int` is designed to mimic the built-in `int` , and therefore using it as a dependency counter can be quite error-prone.

Of course, one can use the `jsc::event_counter` freely available in the JSC C++ Library, which implements the described interface quite nicely.

However, the `std::latch` from the C++ Concurrency Library can also be used, since it provides quite similar interface.

The main two differences are:

the interface of a dependency counter

The interface of `std::atomic_int` is designed to mimic the built-in `int` , and therefore using it as a dependency counter can be quite error-prone.

Of course, one can use the `jsc::event_counter` freely available in the JSC C++ Library, which implements the described interface quite nicely.

However, the `std::latch` from the C++ Concurrency Library can also be used, since it provides quite similar interface.

The main two differences are:

- 1. the `jsc::event_counter` does not allow blocking the current thread of execution via `.wait()` and `.arrive_and_wait()`*

the interface of a dependency counter

The interface of `std::atomic_int` is designed to mimic the built-in `int`, and therefore using it as a dependency counter can be quite error-prone.

Of course, one can use the `jsc::event_counter` freely available in the JSC C++ Library, which implements the described interface quite nicely.

However, the `std::latch` from the C++ Concurrency Library can also be used, since it provides quite similar interface.

The main two differences are:

- 1. the `jsc::event_counter` does not allow blocking the current thread of execution via `.wait()` and `.arrive_and_wait()`*
- 2. the `std::latch::count_down()` returns nothing; testing whether the internal counter has reached zero must be done via `.try_wait()`*

std::latch

```
class latch;
```

The `latch` class is a downward counter which can be used to synchronize threads. The value of the counter is initialized on creation. Threads may block on the latch until the counter is decremented to zero.

Data Members

`std::ptrdiff_t` *counter* the internal counter
(exposition-only member object)

Member functions

(constructor)	constructs a <code>latch</code> (public member function)
(destructor)	destroys the <code>latch</code> (public member function)
<code>count_down</code>	decrements the counter in a non-blocking manner (public member function)
<code>try_wait</code>	tests if the internal counter equals zero (public member function)
<code>wait</code>	blocks until the counter reaches zero (public member function)
<code>arrive_and_wait</code>	decrements the counter and blocks until it reaches zero (public member function)

sequence of synchronization primitives

sequence of synchronization primitives

The problem is neither `std::latch` nor `jsc::event_counter` provide a default constructor, and therefore it is literally impossible to create a non-zero sized `std::vector<>` that stores such elements.

sequence of synchronization primitives

The problem is neither `std::latch` nor `jsl::event_counter` provide a default constructor, and therefore it is literally impossible to create a non-zero sized `std::vector<>` that stores such elements.



ARMv8-A 64-bit

Clang 22.1.0

-std=c++23 -O3



```
#include <vector>
#include <latch>

auto create_vector (const std::size_t size)
{
    return std::vector<std::latch>(size);
}
```

sequence of synchronization primitives

The problem is neither `std::latch` nor `jsc::event_counter` provide a default constructor, and therefore it is literally impossible to create a non-zero sized `std::vector<>` that stores such elements.

 COMPILER EXPLORER

ARMv8-A 64-bit

Clang 22.1.0

-std=c++23 -O3 

```
#include <vector>
#include <latch>

auto create_vector (const std::size_t size)
{
    return std::vector<std::latch>(size);
}
```

Again, both `std::latch` and `jsc::event_counter` are not `DefaultInsertable` nor `MoveInsertable`.

std::latch::latch

```
constexpr explicit latch( std::ptrdiff_t initial_value ); (1)
```

```
latch( const latch& ) = delete; (2)
```

- 1) Constructs a `latch` and initializes its internal counter.
- 2) Copy constructor is deleted. `latch` is neither copyable nor movable.



```
#include <vector>
#include <latch>

auto create_vector (const std::size_t size)
{
    return std::vector<std::latch>(size);
}
```

```
error: no matching function for call to 'construct_at'
| std::construct_at(__p, std::forward<Args>(__args)...);
| ~~~~~^
note: in instantiation of function template specialization requested here
| std::_Construct(std::_addressof(*__first));
| ^
note: in instantiation of function template specialization requested here
| __uninit_default_n(__first, __n);
| ^
note: in instantiation of function template specialization requested here
| { return std::_uninitialized_default_n(__first, __n); }
| ^
note: in instantiation of function template specialization requested here
| std::_uninitialized_default_n_a(this->_M_impl._M_start, __n,
| ^
note: in instantiation of member function requested here
| { _M_default_initialize(__n); }
| ^
note: in instantiation of member function requested here
| return std::vector<std::latch>(size);
| ^
note: candidate template ignored: constraints not satisfied
| [with _Tp = std::latch, _Args = <>]
| construct_at(_Tp* __location, _Args&&... __args)
| ^
note: because 'std::new ((void *)0) _Tp(std::declval<Args>())' would be invalid:
| no matching constructor for initialization of 'std::latch'
| && requires { std::new((void*)0) _Tp(std::declval<Args>()); }
| ^

error: no matching constructor for initialization of 'std::latch'
| std::new(static_cast<void*>(__p)) _Tp(std::forward<Args>(__args)...);
| ^
note: candidate constructor not viable:
| requires single argument 'std::__expected', but no arguments were provided
| latch(ptrdiff_t __expected) noexcept
| ^
note: candidate constructor not viable: requires 1 argument, but 0 were provided
| latch(const latch&) = delete;
| ^
```



```
#include <vector>
#include <latch>

auto create_vector (const std::size_t size)
{
    return std::vector<std::latch>(size);
}
```

```
error: no matching function for call to 'construct_at'
| std::construct_at(__p, std::forward<Args>(__args)...);
| ~~~~~
note: in instantiation of function template specialization requested here
| std::_Construct(std::__addressof(*__first));
| ^
note: in instantiation of function template specialization requested here
| __uninit_default_n(__first, __n);
| ^
note: in instantiation of function template specialization requested here
| { return std::_uninitialized_default_n(__first, __n); }
| ^
note: in instantiation of function template specialization requested here
| std::_uninitialized_default_n_a(this->_M_impl._M_start, __n,
| ^
note: in instantiation of member function requested here
| { _M_default_initialize(__n); }
| ^
note: in instantiation of member function requested here
| return std::vector<std::latch>(size);
| ^
note: candidate template ignored: constraints not satisfied
| [with _Tp = std::latch, _Args = <>]
| construct_at(_Tp* __location, _Args&&... __args)
| ^
note: because 'std::new ((void *)0) _Tp(std::declval<Args>())...' would be invalid:
| no matching constructor for initialization of 'std::latch'
| && requires { ::new((void*)0) _Tp(std::declval<Args>())...; }
| ^

error: no matching constructor for initialization of 'std::latch'
| ::new(static_cast<void*>(__p)) _Tp(std::forward<Args>(__args)...);
| ^
note: candidate constructor not viable:
| requires single argument '__expected', but no arguments were provided
| latch(ptrdiff_t __expected) noexcept
| ^
note: candidate constructor not viable: requires 1 argument, but 0 were provided
| latch(const latch&) = delete;
| ^
```

sequence of synchronization primitives

sequence of synchronization primitives

The only possible way to construct a `std::vector<std::latch>` is to default-construct it:

sequence of synchronization primitives

The only possible way to construct a `std::vector<std::latch>` is to default-construct it:

**COMPILER EXPLORER**

RISC-V 64-bit

Clang 22.1.0

`-std=c++23 -O3`

✓

```
#include <vector>
#include <latch>

auto create_vector ()
{
    return std::vector<std::latch>();
}
```

```
create_vector():
    sd     zero, 0(a0)
    sd     zero, 8(a0)
    sd     zero, 16(a0)
    ret
```

sequence of synchronization primitives

The only possible way to construct a `std::vector<std::latch>` is to default-construct it:



RISC-V 64-bitClang 22.1.0-std=c++23 -O3

```
#include <vector>
#include <latch>

auto create_vector ()
{
    return std::vector<std::latch>();
}
```

```
create_vector():
    sd    zero, 0(a0)
    sd    zero, 8(a0)
    sd    zero, 16(a0)
    ret
```

*Once constructed, however,
the container will remain empty: there is no way to add elements to it.*

sequence of synchronization primitives

sequence of synchronization primitives

- *Surprisingly,
even the from-range constructor
does not solve the problem.*

sequence of synchronization primitives

COMPILER EXPLORER ARM 64-bit GCC 16.1.0 -std=c++26 -O3

```
#include <vector>
#include <ranges>
#include <latch>

auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

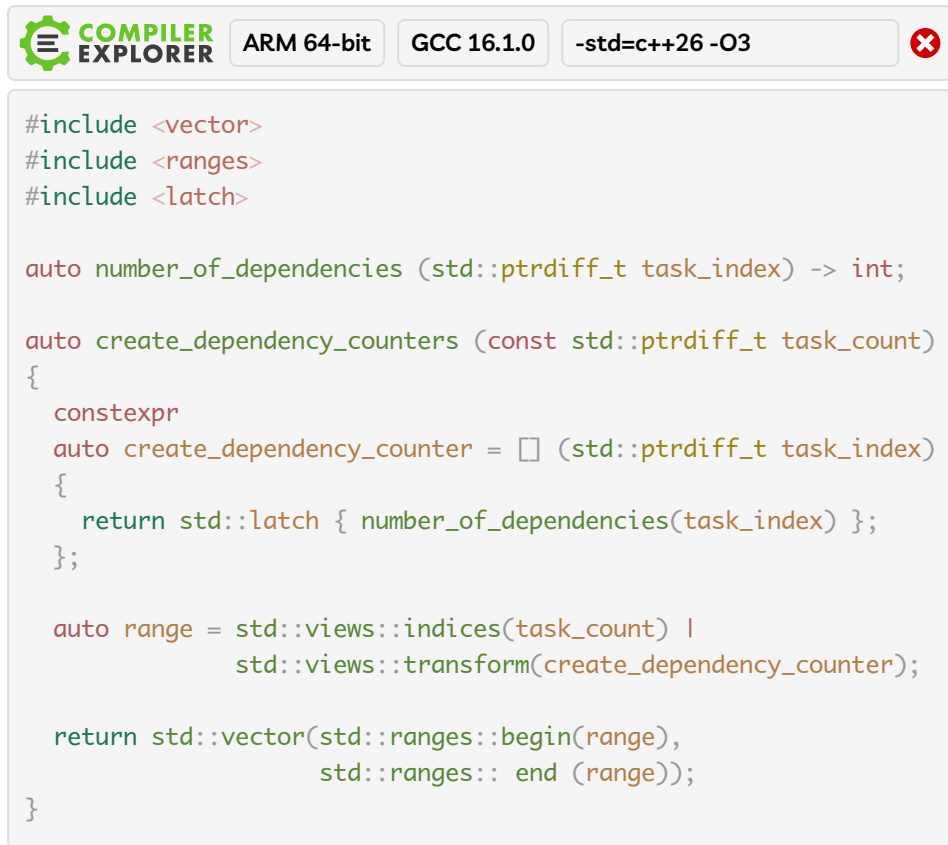
auto create_dependency_counters (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
        std::views::transform(create_dependency_counter);

    return std::vector(std::from_range, range);
}
```

- *Surprisingly, even the from-range constructor does not solve the problem.*

sequence of synchronization primitives



The image shows a screenshot of the Compiler Explorer web interface. At the top, there is a header bar with the 'COMPILER EXPLORER' logo on the left, and three buttons in the center: 'ARM 64-bit', 'GCC 16.1.0', and '-std=c++26 -O3'. A red close button is on the right. Below the header is a large text area containing C++ code. The code defines a function `create_dependency_counters` that takes a `task_count` and returns a `std::vector` of `std::latch` objects. It uses `std::views::indices` and `std::views::transform` to create the latches. The code is as follows:

```
#include <vector>
#include <ranges>
#include <latch>

auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_dependency_counters (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                 std::views::transform(create_dependency_counter);

    return std::vector(std::ranges::begin(range),
                       std::ranges::end (range));
}
```

- *Surprisingly,
even the from-range constructor
does not solve the problem.*
- *Likewise,
the iterator-pair constructor
is not the solution either.*

`std::vector<T,Allocator>::vector`

```
template< class InputIterator >  
vector( InputIterator first, InputIterator last, (2)  
        const Allocator& alloc = Allocator() );
```

```
template< container-compatible-range<T> InputRange >  
constexpr vector( std::from_range_t, InputRange&& input_range, (3)  
                  const Allocator& alloc = Allocator() );
```

2) Constructs a `vector` with the contents of the range `[ first , last )`.

Participates in overload resolution only if `InputIterator` is a *LegacyInputIterator*.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*first`.

3) Constructs a `vector` with the contents of the range `input_range`.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*std::ranges::begin(input_range)`.

`std::vector<T,Allocator>::vector`

```
template< class InputIterator >  
vector( InputIterator first, InputIterator last, (2)  
        const Allocator& alloc = Allocator() );
```

```
template< container-compatible-range<T> InputRange >  
constexpr vector( std::from_range_t, InputRange&& input_range, (3)  
                  const Allocator& alloc = Allocator() );
```

2) Constructs a `vector` with the contents of the range `[ first , last )`.

Participates in overload resolution only if `InputIterator` is a *LegacyInputIterator*.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*first`.

3) Constructs a `vector` with the contents of the range `input_range`.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*std::ranges::begin(input_range)`.

`std::vector<T,Allocator>::vector`

```
template< class InputIterator >  
vector( InputIterator first, InputIterator last, (2)  
        const Allocator& alloc = Allocator() );
```

```
template< container-compatible-range<T> InputRange >  
constexpr vector( std::from_range_t, InputRange&& input_range, (3)  
                  const Allocator& alloc = Allocator() );
```

2) Constructs a `vector` with the contents of the range `[ first , last )`.

Participates in overload resolution only if `InputIterator` is a *LegacyInputIterator*.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*first`.

3) Constructs a `vector` with the contents of the range `input_range`.

- `T` must be *EmplaceConstructible* into `std::vector<T>` from `*std::ranges::begin(input_range)`.

C++ named requirements: *EmplaceConstructible*

Specifies that an object of the type can be constructed from a given set of arguments in uninitialized storage by a given allocator.

Requirements

Type	Definition
<code>T</code>	an object type
<code>A</code>	an allocator type
Value	Definition
<code>m</code>	an lvalue of type <code>A</code>
<code>p</code>	the pointer of type <code>T*</code>
<code>args</code>	zero or more arguments

The type `T` is ***EmplaceConstructible*** from the arguments `args` ,
if the following expression is well-formed:

```
std::allocator_traits<A>::construct(m, p, args);
```

std::vector

```
template<class T, class Allocator = std::allocator<T>>
class vector;
```

The `std::vector` meets the requirements of *Container*, *AllocatorAwareContainer*, *SequenceContainer* and *ContiguousContainer*.

Template parameters

T - The type of the elements.

It is required that element type meets the requirements of *Erasable*, but many member functions impose stricter requirements.

Allocator - An allocator that is used to acquire/release memory and to construct/destroy the elements in that memory.
The type must meet the requirements of *Allocator*.

std::vector

```
template<class T, class Allocator = std::allocator<T>>
class vector;
```

The `std::vector` meets the requirements of *Container*, *AllocatorAwareContainer*, *SequenceContainer* and *ContiguousContainer*.

Template parameters

T - The type of the elements.

It is required that element type meets the requirements of *Erasable*, but many member functions impose stricter requirements.

Allocator - An allocator that is used to acquire/release memory and to construct/destroy the elements in that memory.
The type must meet the requirements of *Allocator*.

std::vector

```
template<class T, class Allocator = std::allocator<T>>
class vector;
```

The `std::vector` meets the requirements of *Container*, *AllocatorAwareContainer*, *SequenceContainer* and *ContiguousContainer*.

Template parameters

T - The type of the elements.

It is required that element type meets the requirements of *Erasable*, but many member functions impose stricter requirements.

Allocator - An allocator that is used to acquire/release memory and to construct/destroy the elements in that memory.
The type must meet the requirements of *Allocator*.

C++ named requirements: *AllocatorAwareContainer*

An ***AllocatorAwareContainer*** is a [Container](#) that holds an instance of an [Allocator](#) and uses that instance in all its member functions to allocate and deallocate memory and to construct and destroy objects in that memory.

Notes

An *AllocatorAwareContainer* always calls `std::allocator_traits<A>::construct(m, p, args)` to construct an object of type `T` at `p` using `args`, with `m == get_allocator()`.

The `std::allocator` has no `construct` member and `std::construct_at(p, args)` is called when constructing elements, but specialized allocators may choose a different definition.

C++ named requirements: *AllocatorAwareContainer*

An ***AllocatorAwareContainer*** is a *Container* that holds an instance of an *Allocator* and uses that instance in all its member functions to allocate and deallocate memory and to construct and destroy objects in that memory.

Notes

An *AllocatorAwareContainer* always calls `std::allocator_traits<A>::construct(m, p, args)` to construct an object of type `T` at `p` using `args`, with `m == get_allocator()`.

The `std::allocator` has no `construct` member and `std::construct_at(p, args)` is called when constructing elements, but specialized allocators may choose a different definition.

C++ named requirements: *AllocatorAwareContainer*

An ***AllocatorAwareContainer*** is a *Container* that holds an instance of an *Allocator* and uses that instance in all its member functions to allocate and deallocate memory and to construct and destroy objects in that memory.

Notes

An *AllocatorAwareContainer* always calls `std::allocator_traits<A>::construct(m, p, args)` to construct an object of type `T` at `p` using `args`, with `m == get_allocator()`.

The `std::allocator` has no `construct` member and `std::construct_at(p, args)` is called when constructing elements, but specialized allocators may choose a different definition.

C++ named requirements: *AllocatorAwareContainer*

An ***AllocatorAwareContainer*** is a *Container* that holds an instance of an *Allocator* and uses that instance in all its member functions to allocate and deallocate memory and to construct and destroy objects in that memory.

Notes

An *AllocatorAwareContainer* always calls `std::allocator_traits<A>::construct(m, p, args)` to construct an object of type `T` at `p` using `args`, with `m == get_allocator()`.

The `std::allocator` has no `construct` member and `std::construct_at(p, args)` is called when constructing elements, but specialized allocators may choose a different definition.

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Storage and lifetime operations		
Expression	Return type	Requirements
<code>a.allocate(n)</code>	<code>A::pointer</code>	Allocates storage for an array of type <code>T[n]</code> and creates the array, but does not construct array elements.
<code>a.deallocate(p, n)</code>	<i>(not used)</i>	Deallocates storage pointed to <code>p</code> .
<code>a.construct(xp, args...)</code> <i>(optional)</i>	<i>(not used)</i>	Constructs an object in previously-allocated storage.
<code>a.destroy(xp)</code> <i>(optional)</i>	<i>(not used)</i>	Destructs an object, but does not deallocate any storage.

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Storage and lifetime operations

Expression	Return type	Requirements
<code>a.allocate(n)</code>	<code>A::pointer</code>	Allocates storage for an array of type <code>T[n]</code> and creates the array, but does not construct array elements.
<code>a.deallocate(p, n)</code>	<i>(not used)</i>	Deallocates storage pointed to <code>p</code> .
<code>a.construct(xp, args...)</code> (optional)	<i>(not used)</i>	Constructs an object in previously-allocated storage.
<code>a.destroy(xp)</code> (optional)	<i>(not used)</i>	Destructs an object, but does not deallocate any storage.

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Storage and lifetime operations

Expression	Return type	Requirements
<code>a.allocate(n)</code>	<code>A::pointer</code>	Allocates storage for an array of type <code>T[n]</code> and creates the array, but does not construct array elements.
<code>a.deallocate(p, n)</code>	<i>(not used)</i>	Deallocates storage pointed to <code>p</code> .
<code>a.construct(xp, args...)</code> (optional)	<i>(not used)</i>	Constructs an object in previously-allocated storage.
<code>a.destroy(xp)</code> (optional)	<i>(not used)</i>	Destructs an object, but does not deallocate any storage.

std::allocator_traits

```
template< class Alloc >  
struct allocator_traits;
```

The `allocator_traits` class template provides the standardized way to access various properties of an *Allocator*.

Member functions

<code>allocate</code> [static]	allocates uninitialized storage using the allocator (public static member function)
<code>deallocate</code> [static]	deallocates storage using the allocator (public static member function)
<code>construct</code> [static]	constructs an object in the allocated storage (function template)
<code>destroy</code> [static]	destructs an object stored in the allocated storage (function template)

`std::allocator_traits<Alloc>::construct`

```
template< class T, class... Args >  
static void construct( Alloc& a, T* p, Args&&... args );
```

Constructs an object of type `T` in allocated uninitialized storage pointed to by `p`.

If `Alloc` does have the member function `construct()`,

then calls `a.construct(p, std::forward<Args>(args)...) ,`

otherwise calls `std::construct_at(p, std::forward<Args>(args)...) .`

guaranteed copy elision

deferred prvalue materialization

guaranteed copy elision

deferred prvalue materialization

*Since C++17, thanks to Richard Smith, URVO
has been formalized as guaranteed copy elision.*

guaranteed copy elision

deferred prvalue materialization

*Since C++17, thanks to Richard Smith, URVO
has been formalized as guaranteed copy elision.*

Intuitively, guaranteed copy elision can be roughly understood as follows:

guaranteed copy elision

deferred prvalue materialization

*Since C++17, thanks to Richard Smith, URVO
has been formalized as guaranteed copy elision.*

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

A temporary is no longer implicitly moved:

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

A temporary is no longer implicitly moved:

- from the return statement to the return channel, nor*

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch();
    // ...
}
```

A temporary is no longer implicitly moved:

- from the return statement to the return channel, nor*

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch();
    // ...
}
```

A temporary is no longer implicitly moved:

- from the return statement to the return channel, nor*
- from the return channel to the storage of the variable being initialized.*

guaranteed copy elision

deferred prvalue materialization

Since C++17, thanks to Richard Smith, URVO has been formalized as guaranteed copy elision.

Intuitively, guaranteed copy elision can be roughly understood as follows:

- a prvalue is not materialized until it's used to initialize a variable*
- at that point, the prvalue is constructed directly in the storage of the variable, which is also treated as an initialization of that variable*

```
auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch();
    // ...
}
```

A temporary is no longer implicitly moved:

- from the return statement to the return channel, nor*
- from the return channel to the storage of the variable being initialized.*

Essentially, no temporary object is created; the prvalue materializes as the variable itself.

guaranteed copy elision

deferred prvalue materialization

guaranteed copy elision

deferred prvalue materialization

 **COMPILER EXPLORER**

RISC-V 32-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch();
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return make_latch();
}
```

- *does not rely on inlining;*
providing function definitions is not required

guaranteed copy elision

deferred prvalue materialization



RISC-V 32-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto main () -> int
{
    auto latch = create_latch(4);

    return latch.try_wait();
}
```

- *does not rely on inlining;*
providing function definitions is not required
- *is independent of linking*

guaranteed copy elision

deferred prvalue materialization



RISC-V 32-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}
```

- *does not rely on inlining;
providing function definitions is not required*
- *is independent of linking*
- *works by passing an extra parameter,
holding the memory address
that will become the `this` pointer*

```
create_latch(long):
    sw     a1,0(a0)
    ret
```

guaranteed copy elision

deferred prvalue materialization

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    auto latch = create_latch(8);
    // ...
}
```


guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3✓

```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    auto&& reference = create_latch(8);
    // ...
}
```

- *initializing a reference from a prvalue is also possible; however, that works differently*

guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3✓

```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    auto&& reference = create_latch(8);
    // ...
}
```

- *initializing a reference from a prvalue is also possible; however, that works differently*
- *a prvalue materializes as a temporary object, then a reference binds to it, extending its lifetime*

guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3✓

```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    const auto& reference = create_latch(8);
    // ...
}
```

- *initializing a reference from a prvalue is also possible; however, that works differently*
- *a prvalue materializes as a temporary object, then a reference binds to it, extending its lifetime*
- *binding a prvalue to any kind of reference triggers a materialization of the prvalue*

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    const std::latch&& reference = create_latch(8);
    // ...
}
```

- *initializing a reference from a prvalue is also possible; however, that works differently*
- *a prvalue materializes as a temporary object, then a reference binds to it, extending its lifetime*
- *binding a prvalue to any kind of reference triggers a materialization of the prvalue*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3

```
#include <latch>

auto create_latch (std::ptrdiff_t init_value) -> std::latch
{
    return std::latch { init_value };
}

auto use_latch () -> void
{
    auto& reference = create_latch(8);
    // ...
}
```

- *initializing a reference from a prvalue is also possible; however, that works differently*
- *a prvalue materializes as a temporary object, then a reference binds to it, extending its lifetime*
- *binding a prvalue to any kind of reference triggers a materialization of the prvalue*
 - *providing that kind of reference can bind to a prvalue in the first place*

guaranteed copy elision

deferred prvalue materialization

guaranteed copy elision

deferred prvalue materialization



```
#include <latch>

auto take_latch (std::latch latch) -> void;

auto give_latch () -> void
{
    take_latch(std::latch { 8 });
}
```

- *works with function parameters*

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>
```

```
auto take_latch (std::latch&& latch) -> void;
```

```
auto give_latch () -> void
```

```
{
```

```
    take_latch(std::latch { 8 });
```

```
}
```

- *works with function parameters*
- *supports passing arguments by-value and by-reference*

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto make_latch () -> std::latch;

auto take_latch (std::latch&& latch) -> void;

auto give_latch () -> void
{
    take_latch(make_latch());
}
```

- *works with function parameters*
- *supports passing arguments by-value and by-reference*
- *origin of the prvalue does not matter; any kind of prvalue expression can be used*

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>

auto make_latch () -> std::latch;

auto take_latch (std::latch latch) -> void;

auto give_latch () -> void
{
    take_latch(make_latch());
}
```

- *works with function parameters*
- *supports passing arguments by-value and by-reference*
- *origin of the prvalue does not matter; any kind of prvalue expression can be used*

guaranteed copy elision

deferred prvalue materialization

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch();
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return make_latch();
}
```

guaranteed copy elision

deferred prvalue materialization



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = std::latch { 8 };
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return std::latch { 8 };
}
```

guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = std::latch { make_latch() };
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*

guaranteed copy elision

deferred prvalue materialization

 **COMPILER EXPLORER**

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = std::latch { std::latch { 8 } };
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return std::latch { std::latch { 8 } };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*

guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = std::latch { std::latch { make_latch() } };
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return std::latch { std::latch { make_latch() } };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*

guaranteed copy elision

deferred prvalue materialization

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = std::latch { std::latch { std::latch { 8 } } };
    // ...
}

auto build_latch () -> std::latch
{
    // ...
    return std::latch { std::latch { std::latch { 8 } } };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto pointer = new std::latch { make_latch() };
    // ...
    delete pointer;
}

auto new_latch () -> std::latch*
{
    // ...
    return new std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a new expression*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto pointer = new std::latch { make_latch() };
    // ...
    delete pointer;
}

auto new_latch () -> std::latch*
{
    // ...
    return new std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a `new` expression*
- *using raw pointers in application code is usually a **Bad Idea™**;*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto pointer = new std::latch { make_latch() };
    // ...
    delete pointer;
}

auto new_latch () -> std::latch*
{
    // ...
    return new std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a `new` expression*
- *using raw pointers in application code is usually a **Bad Idea™**; raw pointers should be used internally, implementation details of generic components*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto pointer = new std::latch { make_latch() };
    // ...
    delete pointer;
}

auto new_latch () -> std::latch*
{
    // ...
    return new std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a `new` expression*
- *using raw pointers in application code is usually a **Bad Idea™**; raw pointers should be used internally, implementation details of generic components*
 - *prefer using `std::indirect<>`*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto pointer = new std::latch { make_latch() };
    // ...
    delete pointer;
}

auto new_latch () -> std::latch*
{
    // ...
    return new std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a `new` expression*
- *using raw pointers in application code is usually a **Bad Idea™**; raw pointers should be used internally, implementation details of generic components*
 - *prefer using `std::indirect<>`*
 - *when using a raw pointer is necessary, consider `std::unique_ptr<>`*

guaranteed copy elision

deferred prvalue materialization

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto make_latch () -> std::latch;

auto use_latch () -> void
{
    alignas(std::latch) std::byte storage [sizeof(std::latch)];

    auto pointer = new(static_cast<void*>(storage))
                    std::latch { make_latch() };
    // ...
    using std_latch = std::latch;    pointer->~std_latch();
}

auto construct_latch (std::latch* location) -> std::latch*
{
    // ...
    return new(static_cast<void*>(location))
            std::latch { make_latch() };
}
```

- *occurs when an object is direct-initialized using an initializer in the form of a prvalue expression of the same type as that object*
- *can be used with a new expression, including a placement new expression*
- *using raw pointers in application code is usually a Bad Idea™; raw pointers should be used internally, implementation details of generic components*
 - *prefer using `std::indirect<>`*
 - *when using a raw pointer is necessary, consider `std::unique_ptr<>`*

perfect forwarding & guaranteed copy elision

perfect forwarding & guaranteed copy elision

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
#include <latch>

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch(); // guaranteed copy elision
    // ...
}
```

perfect forwarding & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>
#include <utility>

constexpr auto pass_through (auto&& reference) noexcept -> auto&&
{
    return std::forward<decltype(reference)>(reference);
}

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = make_latch(); // guaranteed copy elision
    // ...
}
```

perfect forwarding & guaranteed copy elision

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3



```
#include <latch>
#include <utility>

constexpr auto pass_through (auto&& reference) noexcept -> auto&&
{
    return std::forward<decltype(reference)>(reference);
}

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = pass_through(make_latch()); // move construction
    // ...
}
```

- *Unfortunately, perfect forwarding is not truly perfect – it does not support guaranteed copy elision!*

perfect forwarding & guaranteed copy elision

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3



```
#include <latch>
#include <utility>

constexpr auto pass_through (auto&& reference) noexcept -> auto&&
{
    return std::forward<decltype(reference)>(reference);
}

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = pass_through(make_latch()); // move construction
    // ...
}
```

- *Unfortunately, perfect forwarding is not truly perfect – it does not support guaranteed copy elision!*
 - *firstly, the prvalue returned from the `make_latch()` function, materializes as a temporary object and binds to the `reference` parameter*

perfect forwarding & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
#include <latch>
#include <utility>

constexpr auto pass_through (auto&& reference) noexcept -> auto&&
{
    return std::forward<decltype(reference)>(reference);
}

auto make_latch () -> std::latch;

auto use_latch () -> void
{
    auto latch = pass_through(make_latch()); // move construction
    // ...
}
```

- *Unfortunately, perfect forwarding is not truly perfect – it does not support guaranteed copy elision!*
 - *firstly, the prvalue returned from the `make_latch()` function, materializes as a temporary object and binds to the `reference` parameter*
 - *then, the compiler tries to construct the `latch` variable from an rvalue reference, returned by the `pass_through()` function*

materializable range

materializable range



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                std::views::transform(create_dependency_counter);

    using proxy = decltype(*std::ranges::begin(range));

    static_assert(std::same_as<proxy, std::latch>);
}
```

materializable range

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                std::views::transform(create_dependency_counter);

    using proxy = decltype(*std::ranges::begin(range));

    static_assert(std::same_as<proxy, std::latch>);
}
```

- *The result of an expression*

```
std::views::indices(size) | std::views::transform(func)
```

is a lazy range, elements of which are created on-demand/on-the-fly, while that range is being iterated over.

materializable range

 COMPILER EXPLORER

RISC-V 64-bitGCC 16.1.0-std=c++20 -O3

```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                std::views::transform(create_dependency_counter);

    using proxy = decltype(*std::ranges::begin(range));

    static_assert(std::same_as<proxy, std::latch>);
}
```

- *The result of an expression `std::views::indices(size) | std::views::transform(func)` is a lazy range, elements of which are created on-demand/on-the-fly, while that range is being iterated over.*
- *In particular, when dereferenced, the iterator of `std::ranges::transform_view<>` invokes the transform function, and then returns the result of that invocation.*

materializable range

COMPILER EXPLORER RISC-V 64-bit GCC 16.1.0 -std=c++20 -O3 ✓

```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                 std::views::transform(create_dependency_counter);

    using proxy = decltype(*std::ranges::begin(range));

    static_assert(std::same_as<proxy, std::latch>);
}
```

- *The result of an expression*
`std::views::indices(size) | std::views::transform(func)`
is a lazy range, elements of which are created on-demand/on-the-fly, while that range is being iterated over.
- *In particular, when dereferenced, the iterator of* `std::ranges::transform_view<>`
invokes the transform function, and then returns the result of that invocation.
- *The use of a materializable range is intentional – it gives* `std::vector<>`
an opportunity to materialize the range's elements directly inside its memory buffer, without any copying or moving.

allocators & guaranteed copy elision

allocators & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
                 std::views::transform(create_dependency_counter);

    auto allocator = std::allocator<std::latch> { };

    const auto location = std::allocator_traits<decltype(allocator)>
                          ::allocate(allocator, 1);

    std::allocator_traits<decltype(allocator)>
        ::construct(allocator, location, *std::ranges::begin(range));
}
```

allocators & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
        std::views::transform(create_dependency_counter);

    auto allocator = std::allocator<std::latch> { };

    const auto location = std::allocator_traits<decltype(allocator)>
        ::allocate(allocator, 1);

    std::allocator_traits<decltype(allocator)>
        ::construct(allocator, location, *std::ranges::begin(range));
}
```

- *Allocators not only provide the means to allocate and free memory, but also control construction and destruction of objects.*

allocators & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
        std::views::transform(create_dependency_counter);

    auto allocator = std::allocator<std::latch> { };

    const auto location = std::allocator_traits<decltype(allocator)>
        ::allocate(allocator, 1);

    std::allocator_traits<decltype(allocator)>
        ::construct(allocator, location, *std::ranges::begin(range));
}
```

- *Allocators not only provide the means to allocate and free memory, but also control construction and destruction of objects.*
- *This property makes Allocators inherently incompatible with guaranteed copy elision.*

allocators & guaranteed copy elision



RISC-V 64-bit

GCC 16.1.0

-std=c++20 -O3



```
auto number_of_dependencies (std::ptrdiff_t task_index) -> int;

auto create_range (const std::ptrdiff_t task_count)
{
    constexpr
    auto create_dependency_counter = [] (std::ptrdiff_t task_index)
    {
        return std::latch { number_of_dependencies(task_index) };
    };

    auto range = std::views::indices(task_count) |
        std::views::transform(create_dependency_counter);

    auto allocator = std::allocator<std::latch> { };

    const auto location = std::allocator_traits<decltype(allocator)>
        ::allocate(allocator, 1);

    std::allocator_traits<decltype(allocator)>
        ::construct(allocator, location, *std::ranges::begin(range));
}
```

- *Allocators not only provide the means to allocate and free memory, but also control construction and destruction of objects.*
- *This property makes Allocators inherently incompatible with guaranteed copy elision.*
- *When an object is created from a prvalue, via an Allocator, the prvalue materializes into a temporary that binds to a parameter of the `std::allocator_traits<>::construct()` member function, resulting in a move construction of the final object.*

`std::allocator_traits<Alloc>::construct`

```
template< class T, class... Args >  
static void construct( Alloc& a, T* p, Args&&... args );
```

Constructs an object of type `T` in allocated uninitialized storage pointed to by `p`.

If `Alloc` does have the member function `construct()`,

then calls `a.construct(p, std::forward<Args>(args)...) ,`

otherwise calls `std::construct_at(p, std::forward<Args>(args)...) .`

`std::allocator_traits<Alloc>::construct`

```
template< class T, class... Args >  
static void construct( Alloc& a, T* p, Args&&... args );
```

Constructs an object of type `T` in allocated uninitialized storage pointed to by `p`.

If `Alloc` does have the member function `construct()`,

then calls `a.construct(p, std::forward<Args>(args)...) ,`

otherwise calls `std::construct_at(p, std::forward<Args>(args)...) .`



`jsc::bounded_vector<>`

key observations

key observations

Support for immovable elements in `std::vector<` is inadequate, since:

key observations

Support for immovable elements in `std::vector<>` is inadequate, since:

- 1. operations that insert elements may trigger a reallocation*

key observations

Support for immovable elements in `std::vector<>` is inadequate, since:

- 1. operations that insert elements may trigger a reallocation*
- 2. construction and destruction of elements is performed via an Allocator*

key observations

Support for immovable elements in `std::vector<>` is inadequate, since:

- 1. operations that insert elements may trigger a reallocation*
- 2. construction and destruction of elements is performed via an Allocator*
- 3. inserting operations do not take advantage of the guaranteed copy elision*

design decisions: reallocations

design decisions: reallocations

Why does `.emplace_back()` perform a reallocation, in the first place?

design decisions: reallocations

Why does `.emplace_back()` perform a reallocation, in the first place?

Well, when a `std::vector<>` is full, meaning `.size() == .capacity()`, there is no space left for the new element;

design decisions: reallocations

Why does `.emplace_back()` perform a reallocation, in the first place?

Well, when a `std::vector<>` is full, meaning `.size() == .capacity()`, there is no space left for the new element; thus, `.emplace_back()` performs a reallocation in order to increase capacity before constructing the new element.

design decisions: reallocations

Why does `.emplace_back()` perform a reallocation, in the first place?

Well, when a `std::vector<>` is full, meaning `.size() == .capacity()`, there is no space left for the new element; thus, `.emplace_back()` performs a reallocation in order to increase capacity before constructing the new element.

Question: Should `.emplace_back()` perform a reallocation when a `std::vector<>` is full?

design decisions: reallocations

Why does `.emplace_back()` perform a reallocation, in the first place?

Well, when a `std::vector<>` is full, meaning `.size() == .capacity()`, there is no space left for the new element; thus, `.emplace_back()` performs a reallocation in order to increase capacity before constructing the new element.

Question: Should `.emplace_back()` perform a reallocation when a `std::vector<>` is full?

I don't think so. Why?

design decisions: reallocations

design decisions: reallocations

First of all, assuming that yes,

the `.emplace_back()` should perform a reallocation when a `std::vector<>` is full,

design decisions: reallocations

*First of all, assuming that yes,
the `.emplace_back()` should perform a reallocation when a `std::vector<>` is full,
its name should probably be `.maybe_realloc_and_emplace_back()` ,
because it reflects much more accurately what is actually happening in the code ...*

design decisions: reallocations

*First of all, assuming that yes,
the `.emplace_back()` should perform a reallocation when a `std::vector<>` is full,
its name should probably be `.maybe_realloc_and_emplace_back()` ,
because it reflects much more accurately what is actually happening in the code ...*

*Performing sporadically such an expensive and unpredictable operation
in one of the most fundamental and commonly used function,
while its name hides that fact, is IMO far from ideal.*

design decisions: reallocations

*First of all, assuming that yes,
the `.emplace_back()` should perform a reallocation when a `std::vector<>` is full,
its name should probably be `.maybe_realloc_and_emplace_back()` ,
because it reflects much more accurately what is actually happening in the code ...*

*Performing sporadically such an expensive and unpredictable operation
in one of the most fundamental and commonly used function,
while its name hides that fact, is IMO far from ideal.*

*My point is: the `.emplace_back()` does not just
emplace a new element at the back and that's it.*

design decisions: reallocations

design decisions: reallocations

*Moreover, in real-time systems,
it is critical to control when memory is allocated and freed,
because these operations have non-deterministic execution time.*

design decisions: reallocations

*Moreover, in real-time systems,
it is critical to control when memory is allocated and freed,
because these operations have non-deterministic execution time.*

*Implementing separate functions for inserting elements and reallocating
allows for easy code verification regarding the lack of memory management operations.*

design decisions: reallocations

*Moreover, in real-time systems,
it is critical to control when memory is allocated and freed,
because these operations have non-deterministic execution time.*

*Implementing separate functions for inserting elements and reallocating
allows for easy code verification regarding the lack of memory management operations.*

*Such a design, prioritizing determinism and explicitness,
is ideal for real-time systems, and therefore
equally suitable for any high-performance system.*

design decisions: reallocations

*Moreover, in real-time systems,
it is critical to control when memory is allocated and freed,
because these operations have non-deterministic execution time.*

*Implementing separate functions for inserting elements and reallocating
allows for easy code verification regarding the lack of memory management operations.*

*Such a design, prioritizing determinism and explicitness,
is ideal for real-time systems, and therefore
equally suitable for any high-performance system.*

*As a bonus, such inserting functions will work flawlessly on GPUs,
since allocating memory does not work well on them.*

design decisions: reallocations

*Moreover, in real-time systems,
it is critical to control when memory is allocated and freed,
because these operations have non-deterministic execution time.*

*Implementing separate functions for inserting elements and reallocating
allows for easy code verification regarding the lack of memory management operations.*

*Such a design, prioritizing determinism and explicitness,
is ideal for real-time systems, and therefore
equally suitable for any high-performance system.*

*As a bonus, such inserting functions will work flawlessly on GPUs,
since allocating memory does not work well on them.*

No more reallocations behind our backs – they must be invoked explicitly!

design decisions: reallocations

design decisions: reallocations

The `.emplace_back()` *in the* `jsc::bounded_vector<>` :

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*
- has a precondition – expects that a vector is not full*

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*
- has a precondition – expects that a vector is not full*

When inserting new elements, the burden is on the programmer to ensure that a `jsc::bounded_vector<>` has enough space left to fit all of them, which forces the programmer to track not only the size of the vector, but also its capacity.

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*
- has a precondition – expects that a vector is not full*

When inserting new elements, the burden is on the programmer to ensure that a `jsc::bounded_vector<>` has enough space left to fit all of them, which forces the programmer to track not only the size of the vector, but also its capacity.

That means it's an unsafe function, which on the surface sounds problematic.

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*
- has a precondition – expects that a vector is not full*

When inserting new elements, the burden is on the programmer to ensure that a `jsc::bounded_vector<>` has enough space left to fit all of them, which forces the programmer to track not only the size of the vector, but also its capacity.

That means it's an unsafe function, which on the surface sounds problematic.

However, forgetting to `.reserve()` sufficient capacity for a `std::vector<>` is even worse: a reallocation will occur silently and the program will continue executing without any indication of this performance issue.

design decisions: reallocations

The `.emplace_back()` in the `jsc::bounded_vector<>` :

- only creates a new element at the back*
- never, under any circumstances, performs a reallocation*
- has a precondition – expects that a vector is not full*

When inserting new elements, the burden is on the programmer to ensure that a `jsc::bounded_vector<>` has enough space left to fit all of them, which forces the programmer to track not only the size of the vector, but also its capacity.

That means it's an unsafe function, which on the surface sounds problematic.

However, forgetting to `.reserve()` sufficient capacity for a `std::vector<>` is even worse: a reallocation will occur silently and the program will continue executing without any indication of this performance issue.

Legitimizing slow code is wrong;

the `jsc::bounded_vector<>` will report this issue as a contract violation in the debug mode.

design decisions: reallocations

design decisions: reallocations

*Still, there are real use cases,
where it's unknown how many elements will be stored in a vector,
meaning there is no useful bound that can be computed upfront,
while potentially every insertsion could require a reallocation.*

design decisions: reallocations

*Still, there are real use cases,
where it's unknown how many elements will be stored in a vector,
meaning there is no useful bound that can be computed upfront,
while potentially every insertsion could require a reallocation.*

*In these use cases, using `std::vector<>` is justified;
giving up control and relying on the growth of the `std::vector<>`,
to minimize the number of reallocations, is reasonable.*

design decisions: reallocations

*Still, there are real use cases,
where it's unknown how many elements will be stored in a vector,
meaning there is no useful bound that can be computed upfront,
while potentially every insertsion could require a reallocation.*

*In these use cases, using `std::vector<>` is justified;
giving up control and relying on the growth of the `std::vector<>` ,
to minimize the number of reallocations, is reasonable.*

In other use cases, I recommend using the `jsc::bounded_vector<>` .

design decisions: reallocations

design decisions: reallocations

*Furthermore,
the interfaces of `.emplace_back()` and `.pop_back()` should be symmetric.*

design decisions: reallocations

Furthermore,

the interfaces of `.emplace_back()` and `.pop_back()` should be symmetric.

Since `.pop_back()` has a precondition checking that a vector is not empty, then `.emplace_back()` should also have a precondition checking that a vector is not full.

std::vector<T,Allocator>::pop_back

```
void pop_back();
```

Removes the last element of the container.

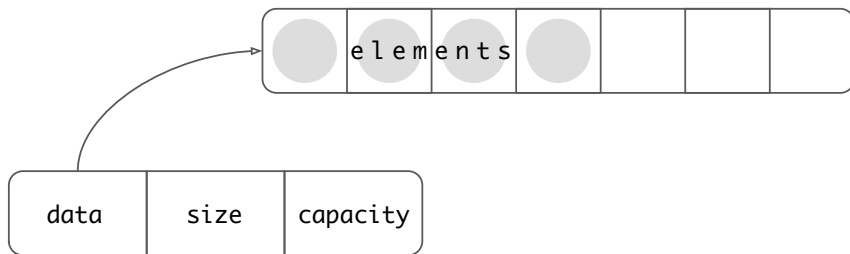
If `empty()` is `true`:

- If the implementation is [hardened](#), a [contract violation](#) occurs.
- If the implementation is not hardened, the behavior is undefined.

memory layout of a vector

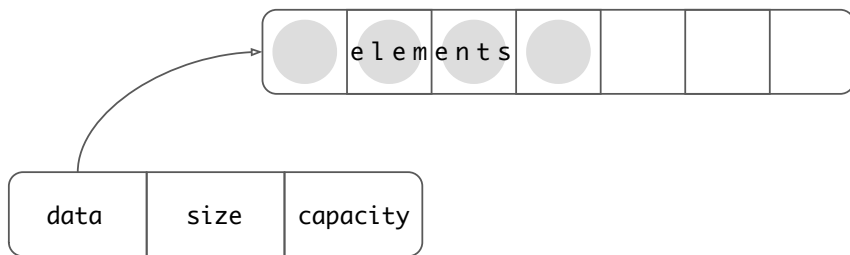
memory layout of a vector

`std::vector<T, Allocator>`



memory layout of a vector

`std::vector<T, Allocator>`

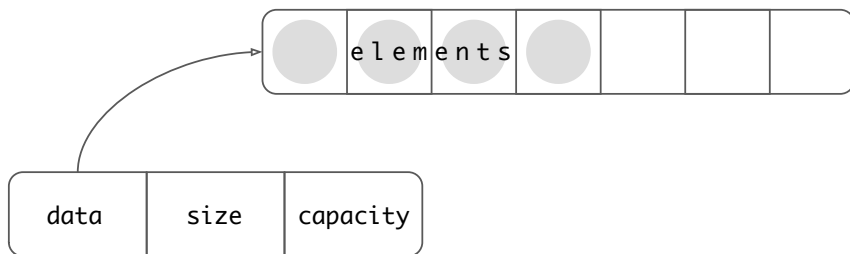


`std::inplace_vector<T, Capacity>`



memory layout of a vector

`std::vector<T, Allocator>`



- *resizable*

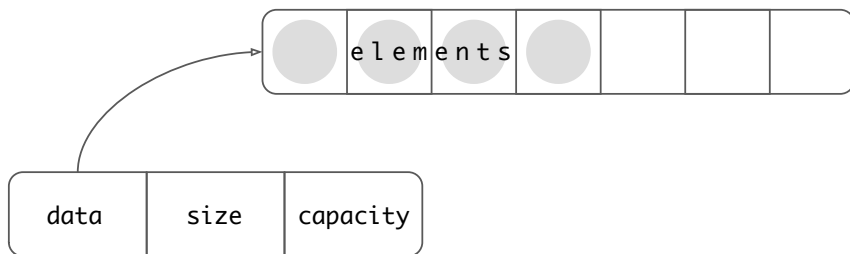
`std::inplace_vector<T, Capacity>`



- *resizable*

memory layout of a vector

`std::vector<T, Allocator>`



`std::inplace_vector<T, Capacity>`

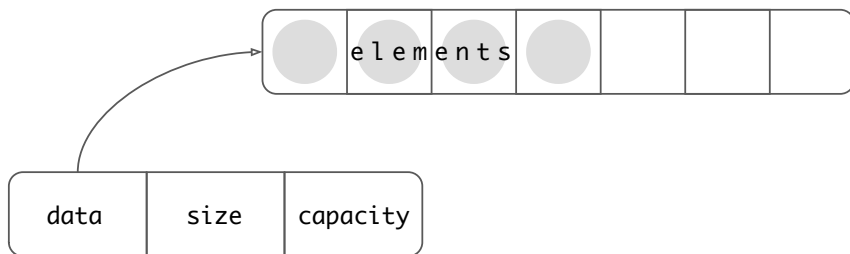


- *resizable*
- *allocates a remote storage*

- *resizable*
- *has a local storage*

memory layout of a vector

`std::vector<T, Allocator>`



`std::inplace_vector<T, Capacity>`



- *resizable*
- *allocates a remote storage*
- *capacity defined at run-time*

- *resizable*
- *has a local storage*
- *capacity fixed at compile-time*

std::inplace_vector

```
template<class T, std::size_t Capacity>  
struct inplace_vector;           (since C++26)
```

`std::inplace_vector` is a dynamically-resizable array with contiguous inplace storage.

The elements are stored and properly aligned within the object itself.

The capacity of the internal storage is fixed at compile-time.

`std::inplace_vector` meets the requirements of [Container](#), [ContiguousContainer](#), and [SequenceContainer](#), (...).

Any member function of `std::inplace_vector` that would cause insertion beyond the capacity throws `std::bad_alloc`.

Template parameters

T - element type

Capacity - capacity, i.e. the maximum number of elements in the `std::inplace_vector` (might be `0`).

Notes

The number of elements in a `std::inplace_vector` may vary dynamically up to a fixed capacity because elements are stored within the object itself.

However, objects are initialized as they are inserted into `std::inplace_vector`.

std::inplace_vector

```
template<class T, std::size_t Capacity>  
struct inplace_vector;      (since C++26)
```

Member functions

<code>begin</code>	returns an iterator to the beginning (public member function)
<code>end</code>	returns an iterator to the end (public member function)
<code>data</code>	direct access to the underlying contiguous storage (public member function)
<code>size</code>	returns the number of elements (public member function)
<code>capacity</code> [static]	returns the number of elements that can be held in currently allocated storage (public static member function)
<code>reserve</code> [static]	reserves storage (public static member function)
<code>emplace_back</code>	constructs an element in-place at the end (public member function)
<code>try_emplace_back</code>	tries to construct an element in-place at the end (public member function)
<code>unchecked_emplace_back</code>	unconditionally constructs an element in-place at the end (public member function)
<code>pop_back</code>	removes the last element (public member function)

std::inplace_vector<T,N>::emplace_back

```
template< class... Args >  
constexpr reference emplace_back( Args&&... args );
```

Appends a new element to the end of the container.

Typically, the element is constructed using placement-`new` to construct the element in-place at the location provided by the container.

Exceptions

- `std::bad_alloc` if `size() == capacity()` before invocation.
- Any exception thrown by initialization of the inserted element.

std::inplace_vector<T,N>::unchecked_emplace_back

```
template< class... Args >  
constexpr reference unchecked_emplace_back( Args&&... args );
```

Appends a new element to the end of the container.

Typically, the element is constructed using placement-`new` to construct the element in-place at the location provided by the container.

Before the call to this function `size() < capacity()` must be `true`. Otherwise, the behavior is undefined.

Exceptions

Any exception thrown by initialization of inserted element.

errors

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

A function should report a recoverable error only because of the following reasons:

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

A function should report a recoverable error only because of the following reasons:

- 1. the state of the execution environment is such that achieving postconditions is not possible*

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

A function should report a recoverable error only because of the following reasons:

- 1. the state of the execution environment is such that achieving postconditions is not possible*
- 2. checking preconditions would have a cost comparable to performing the operation itself*

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

A function should report a recoverable error only because of the following reasons:

- 1. the state of the execution environment is such that achieving postconditions is not possible*
- 2. checking preconditions would have a cost comparable to performing the operation itself*

The error reporting mechanism is irrelevant; these rules apply when using error codes, exceptions or result types.

errors

The `.emplace_back()` should not throw `std::bad_alloc` when a `std::inplace_vector<>` is full.

A function should report a recoverable error only because of the following reasons:

- 1. the state of the execution environment is such that achieving postconditions is not possible*
- 2. checking preconditions would have a cost comparable to performing the operation itself*

The error reporting mechanism is irrelevant; these rules apply when using error codes, exceptions or result types.

A programming error, such as a precondition violation, should never be reported to the caller, because it is not recoverable.

errors

errors

A function creating a symbolic link may fail due to:

errors

A function creating a symbolic link may fail due to:

- an I/O error that occurred while reading from or writing to the file system*

errors

A function creating a symbolic link may fail due to:

- an I/O error that occurred while reading from or writing to the file system*
- resource exhaustion, meaning there is no space left on the file system*

errors

A function creating a symbolic link may fail due to:

- an I/O error that occurred while reading from or writing to the file system*
- resource exhaustion, meaning there is no space left on the file system*



```
void create_symlink (const std::filesystem::path& target,  
                    const std::filesystem::path& link, std::error_code& err) noexcept;
```

errors

A function creating a symbolic link may fail due to:

- *an I/O error that occurred while reading from or writing to the file system*
- *resource exhaustion, meaning there is no space left on the file system*



```
void create_symlink (const std::filesystem::path& target,  
                    const std::filesystem::path& link, std::error_code& err) noexcept;
```

*A function parsing a string should not expect the input string to be valid,
because verifying that requires nearly as much work as the parsing itself*

errors

A function creating a symbolic link may fail due to:

- an I/O error that occurred while reading from or writing to the file system*
- resource exhaustion, meaning there is no space left on the file system*



```
void create_symlink (const std::filesystem::path& target,  
                    const std::filesystem::path& link, std::error_code& err) noexcept;
```

*A function parsing a string should not expect the input string to be valid,
because verifying that requires nearly as much work as the parsing itself*



```
pub fn parse<T: FromStr>(&self) -> Result<T, T::Err> { /* ... */ }
```

errors: more info

errors: more info

Zero-overhead deterministic exceptions: Throwing values

Document Number: **P0709 R4**
Reply-to: Herb Sutter

Date: 2019-08-04
Audience: EWG, LEWG

errors: more info

Zero-overhead deterministic exceptions: Throwing values

Document Number: **P0709 R4**
Reply-to: Herb Sutter

Date: 2019-08-04
Audience: EWG, LEWG



errors: more info

C++ now 2018 MAY 7 - 11 cppnow.org

Domain Total function Codomain

string $f(x) \rightarrow y$ optional<int> +

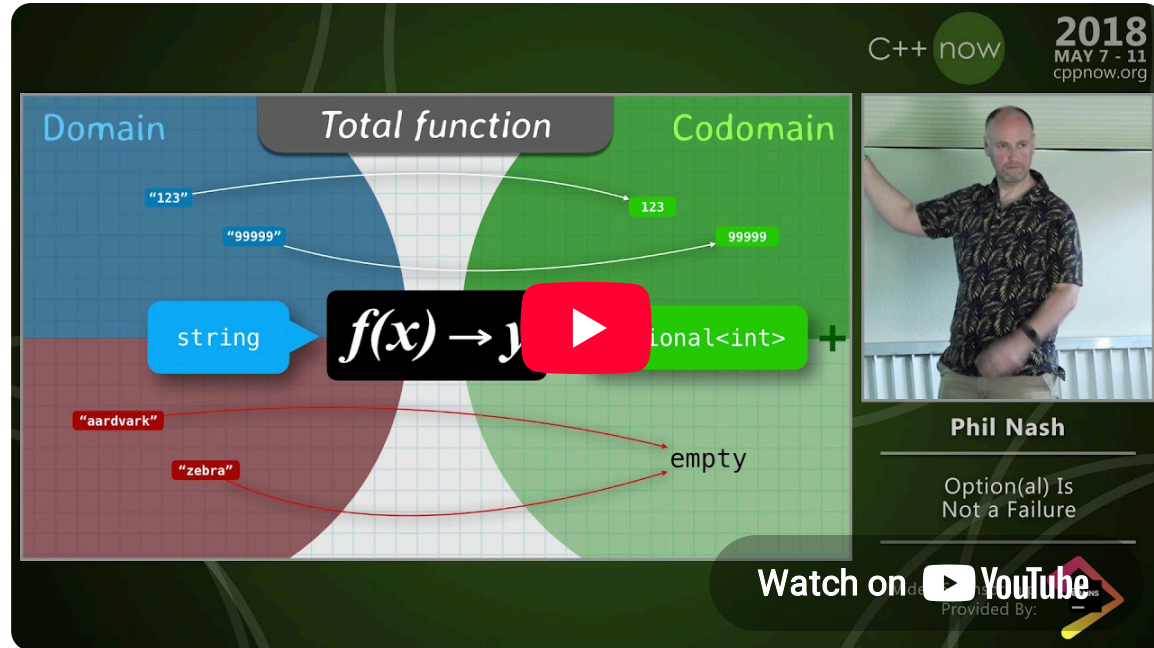
"123" "99999" 123 99999

"aardvark" "zebra" empty

Phil Nash

Option(al) Is Not a Failure

Watch on YouTube Provided By:



design decisions: reallocations

design decisions: reallocations

*Checking whether a `std::inplace_vector<>` is not full is extremely cheap,
and the state of the execution environment has no influence on that condition.*

design decisions: reallocations

Checking whether a `std::inplace_vector<>` is not full is extremely cheap, and the state of the execution environment has no influence on that condition.

Because of that, calling `.emplace_back()` on a full `std::inplace_vector<>` should be treated as a programming error.

design decisions: reallocations

Checking whether a `std::inplace_vector<>` is not full is extremely cheap, and the state of the execution environment has no influence on that condition.

Because of that, calling `.emplace_back()` on a full `std::inplace_vector<>` should be treated as a programming error.

Essentially, `.emplace_back()` should behave exactly like `.unchecked_emplace_back()`, that is, it should have a precondition instead of throwing `std::bad_alloc`, while the `.unchecked_emplace_back()` would become redundant and should be removed.

design decisions: reallocations

Checking whether a `std::inplace_vector<>` is not full is extremely cheap, and the state of the execution environment has no influence on that condition.

Because of that, calling `.emplace_back()` on a full `std::inplace_vector<>` should be treated as a programming error.

Essentially, `.emplace_back()` should behave exactly like `.unchecked_emplace_back()`, that is, it should have a precondition instead of throwing `std::bad_alloc`, while the `.unchecked_emplace_back()` would become redundant and should be removed.

The `jsc::bounded_vector<>` implements this exact design.

design decisions: reallocations

design decisions: reallocations

This is the origin of the name `bounded_vector` ;

*there exists an uncrossable boundary beyond which elements cannot exist,
meaning the size of a vector cannot exceed its capacity.*

design decisions: reallocations

*This is the origin of the name `bounded_vector` ;
there exists an uncrossable boundary beyond which elements cannot exist,
meaning the size of a vector cannot exceed its capacity.*

*Both `std::inplace_vector` and `jsc::bounded_vector` have this uncrossable boundary,
the difference being, the former has it fixed at compile-time,
while the latter can change it at run-time via `.realloc()` .*

design decisions: reallocations

*This is the origin of the name `bounded_vector` ;
there exists an uncrossable boundary beyond which elements cannot exist,
meaning the size of a vector cannot exceed its capacity.*

*Both `std::inplace_vector<>` and `jsc::bounded_vector<>` have this uncrossable boundary,
the difference being, the former has it fixed at compile-time,
while the latter can change it at run-time via `.realloc()` .*

*Since `std::vector<>` grows while elements are inserted into it,
it does not have this uncrossable boundary.
However, the names of such operations should reflect that fact.*

std::inplace_vector<T,N>::reserve

```
static constexpr void reserve( size_type new_capacity );
```

(since C++26)

Does nothing, except that may throw [std::bad_alloc](#).

The request to increase the capacity (i.e., the internal storage size) is ignored because `std::inplace_vector<T, N>` is a fixed-capacity container.

Exceptions

[std::bad_alloc](#) if `new_capacity > capacity()` is `true`.

Notes

This function exists for compatibility with vector-like interfaces.

design decisions: reallocations

design decisions: reallocations

Despite the fact that both `std::vector<>` *and* `std::inplace_vector<>`

design decisions: reallocations

Despite the fact that both `std::vector<>` and `std::inplace_vector<>`

- model the `std::ranges::contiguous_range` concept, and*

design decisions: reallocations

Despite the fact that both `std::vector<>` and `std::inplace_vector<>`

- model the `std::ranges::contiguous_range` concept, and*
- satisfy the `SequenceContainer` named requirements,*

design decisions: reallocations

Despite the fact that both `std::vector<>` and `std::inplace_vector<>`

- model the `std::ranges::contiguous_range` concept, and*
- satisfy the `SequenceContainer` named requirements,*

the `std::vector<>` is growable, while the `std::inplace_vector<>` is not.

design decisions: reallocations

Despite the fact that both `std::vector<>` and `std::inplace_vector<>`

- model the `std::ranges::contiguous_range` concept, and*
- satisfy the `SequenceContainer` named requirements,*

the `std::vector<>` is growable, while the `std::inplace_vector<>` is not.

*This fundamentally changes the use cases they support,
meaning they cannot be used interchangeably,
thus forcing artificial uniformity of their interfaces is IMO a mistake.*

design decisions: reallocations

design decisions: reallocations

*A generic function, inserting elements into a vector,
cannot work well on `std::inplace_vector<>` , because it either:*

design decisions: reallocations

*A generic function, inserting elements into a vector,
cannot work well on `std::inplace_vector<>` , because it either:*

- inserts a run-time specified number of elements,
and therefore has genuine risk of failure, or*

design decisions: reallocations

A generic function, inserting elements into a vector, cannot work well on `std::inplace_vector<>`, because it either:

- inserts a run-time specified number of elements, and therefore has genuine risk of failure, or*
- inserts a compile-time known number of elements, and therefore checks redundantly for an out-of-memory error.*

design decisions: reallocations

A generic function, inserting elements into a vector, cannot work well on `std::inplace_vector<>`, because it either:

- inserts a run-time specified number of elements, and therefore has genuine risk of failure, or*
- inserts a compile-time known number of elements, and therefore checks redundantly for an out-of-memory error.*

Rather than switching manually between these two vector representations, a dedicated data structure should be used, such as a small-buffer-optimized vector, e.g. `llvm::SmallVector<T, N>`.

design decisions: allocators

design decisions: allocators

What is an `Allocator` anyway?

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Storage and lifetime operations		
Expression	Return type	Requirements
<code>a.allocate(n)</code>	<code>A::pointer</code>	Allocates storage for an array of type <code>T[n]</code> and creates the array, but does not construct array elements.
<code>a.deallocate(p, n)</code>	<i>(not used)</i>	Deallocates storage pointed to <code>p</code> .
<code>a.construct(xp, args...)</code> <i>(optional)</i>	<i>(not used)</i>	Constructs an object in previously-allocated storage.
<code>a.destroy(xp)</code> <i>(optional)</i>	<i>(not used)</i>	Destructs an object, but does not deallocate any storage.

design decisions: allocators

What is an Allocator anyway?

design decisions: allocators

What is an `Allocator` anyway?

Well, it's an object that provides the means to allocate and free memory.

design decisions: allocators

What is an Allocator anyway?

Well, it's an object that provides the means to allocate and free memory.

*Question: Why does it also control
how objects are constructed and destroyed in the allocated memory?*

design decisions: allocators

What is an Allocator anyway?

Well, it's an object that provides the means to allocate and free memory.

*Question: Why does it also control
how objects are constructed and destroyed in the allocated memory?*

I don't think there's a good reason for this.

Not even a single example has convinced me otherwise. Why?

design decisions: allocators

design decisions: allocators

- 1. Allocators should perform all the necessary bookkeeping, in the `.allocate()` and `.deallocate()` member functions.*

design decisions: allocators

1. *Allocators should perform all the necessary bookkeeping, in the `.allocate()` and `.deallocate()` member functions.*
 - *Maintaining data structures that store information about the allocated and free memory does not require customizing the `.construct()` and `.destroy()` member functions.*

design decisions: allocators

- 1. Allocators should perform all the necessary bookkeeping, in the `.allocate()` and `.deallocate()` member functions.*
 - Maintaining data structures that store information about the allocated and free memory does not require customizing the `.construct()` and `.destroy()` member functions.*
- 2. Allocators should give callers the freedom to arbitrarily manipulate the allocated memory, instead of imposing restrictions on how objects must be constructed and destroyed.*

design decisions: allocators

- 1. Allocators should perform all the necessary bookkeeping, in the `.allocate()` and `.deallocate()` member functions.*
 - Maintaining data structures that store information about the allocated and free memory does not require customizing the `.construct()` and `.destroy()` member functions.*
- 2. Allocators should give callers the freedom to arbitrarily manipulate the allocated memory, instead of imposing restrictions on how objects must be constructed and destroyed.*

Allocators should allocate and free memory – that's it!

design decisions: allocators

design decisions: allocators

3. *For inter-process communication via shared memory, it's best to use a dedicated data structure / synchronization primitive, e.g. `message_queue` .*

design decisions: allocators

3. *For inter-process communication via shared memory, it's best to use a dedicated data structure / synchronization primitive, e.g. `message_queue`.*
 - *Remember, such a software component must not only allocate & map shared memory, but also synchronize communicating processes.*

design decisions: allocators

3. *For inter-process communication via shared memory, it's best to use a dedicated data structure / synchronization primitive, e.g. `message_queue`.*
 - *Remember, such a software component must not only allocate & map shared memory, but also synchronize communicating processes.*

4. *If it's necessary to propagate an `Allocator`, from an `AllocatorAwareContainer` to its `AllocatorAwareObject` elements, then that responsibility should be delegated to the `AllocatorAwareContainer`, not the `Allocator` itself.*

design decisions: allocators

3. *For inter-process communication via shared memory, it's best to use a dedicated data structure / synchronization primitive, e.g. `message_queue`.*
 - *Remember, such a software component must not only allocate & map shared memory, but also synchronize communicating processes.*

4. *If it's necessary to propagate an `Allocator`, from an `AllocatorAwareContainer` to its `AllocatorAwareObject` elements, then that responsibility should be delegated to the `AllocatorAwareContainer`, not the `Allocator` itself.*
 - *often an issue when nesting `AllocatorAwareContainers`*

design decisions: allocators

3. *For inter-process communication via shared memory, it's best to use a dedicated data structure / synchronization primitive, e.g. `message_queue`.*
 - *Remember, such a software component must not only allocate & map shared memory, but also synchronize communicating processes.*

4. *If it's necessary to propagate an `Allocator`, from an `AllocatorAwareContainer` to its `AllocatorAwareObject` elements, then that responsibility should be delegated to the `AllocatorAwareContainer`, not the `Allocator` itself.*
 - *often an issue when nesting `AllocatorAwareContainers`*
 - `std::scoped_allocator_adaptor`

design decisions: allocators

design decisions: allocators

However,

there is a much bigger problem with Allocators ...

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Relationship between instances

Expression	Return type	Requirements
<code>a1 == a2</code>	<code>bool</code>	<code>true</code> only if the storage allocated by the allocator <code>a1</code> can be deallocated through the allocator <code>a2</code>. - Establishes reflexive, symmetric, and transitive relationship. - Does not throw exceptions.
Declaration	Effect	Requirements
<code>A a1(a)</code>	Copy-constructs <code>a1</code> such that <code>a1 == a</code> .	Does not throw exceptions.
Type-id	Aliased type	Requirements
<code>A::is_always_equal</code> (optional)	<code>std::true_type</code> or <code>std::false_type</code> or derived from such.	<code>true</code> if any two allocators of type <code>A</code> always compare equal. - If not provided, <code>std::allocator_traits</code> defaults to <code>std::is_empty<A>::value</code>.

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Relationship between instances		
Expression	Return type	Requirements
<code>a1 == a2</code>	<code>bool</code>	<code>true</code> only if the storage allocated by the allocator <code>a1</code> can be deallocated through the allocator <code>a2</code>. - Establishes reflexive, symmetric, and transitive relationship. - Does not throw exceptions.
Declaration	Effect	Requirements
<code>A a1(a)</code>	Copy-constructs <code>a1</code> such that <code>a1 == a</code> .	Does not throw exceptions.
Type-id	Aliased type	Requirements
<code>A::is_always_equal</code> (optional)	<code>std::true_type</code> or <code>std::false_type</code> or derived from such.	<code>true</code> if any two allocators of type <code>A</code> always compare equal. - If not provided, <code>std::allocator_traits</code> defaults to <code>std::is_empty<A>::value</code>.

C++ named requirements: *Allocator*

Encapsulates strategies for access/addressing, allocation/deallocation and construction/destruction of objects.

Every standard library component that may need to allocate or deallocate storage does so through an ***Allocator***.

Requirements

Relationship between instances		
Expression	Return type	Requirements
<code>a1 == a2</code>	<code>bool</code>	<code>true</code> only if the storage allocated by the allocator <code>a1</code> can be deallocated through the allocator <code>a2</code>. - Establishes reflexive, symmetric, and transitive relationship. - Does not throw exceptions.
Declaration	Effect	Requirements
<code>A a1(a)</code>	Copy-constructs <code>a1</code> such that <code>a1 == a</code> .	Does not throw exceptions.
Type-id	Aliased type	Requirements
<code>A::is_always_equal</code> (optional)	<code>std::true_type</code> or <code>std::false_type</code> or derived from such.	<code>true</code> if any two allocators of type <code>A</code> always compare equal. - If not provided, <code>std::allocator_traits</code> defaults to <code>std::is_empty<A>::value</code>.

stateful & stateless allocators

stateful & stateless allocators

Every `Allocator` is either stateful or stateless.

stateful & stateless allocators

Every `Allocator` is either stateful or stateless.

stateful allocator

stateless allocator

stateful & stateless allocators

Every `Allocator` is either stateful or stateless.

stateful allocator

- *denotes distinct memory resources*

stateless allocator

- *denotes a single memory resource*

stateful & stateless allocators

Every `Allocator` is either stateful or stateless.

stateful allocator

- *denotes distinct memory resources*
- *values may be unequal*

stateless allocator

- *denotes a single memory resource*
- *values are always equal*

stateful & stateless allocators

Every `Allocator` is either stateful or stateless.

stateful allocator

- *denotes distinct memory resources*
- *values may be unequal*
- *typically contains state*

stateless allocator

- *denotes a single memory resource*
- *values are always equal*
- *typically is an empty class*

design decisions: allocators

However,

there is a much bigger problem with Allocators ...

design decisions: allocators

However,

there is a much bigger problem with Allocators ...

The problem is: stateful allocators break value semantics of containers!

design decisions: allocators

design decisions: allocators

The Allocator named requirements are specified such that:

design decisions: allocators

The Allocator named requirements are specified such that:

- *a stateful allocator is essentially a handle to a memory resource, from which memory is allocated*

design decisions: allocators

The Allocator named requirements are specified such that:

- *a stateful allocator is essentially a handle to a memory resource, from which memory is allocated*
- *a stateful allocator does not represent the memory resource itself*

design decisions: allocators

The Allocator named requirements are specified such that:

- a stateful allocator is essentially a handle to a memory resource, from which memory is allocated*
- a stateful allocator does not represent the memory resource itself*

This is a consequence of the fact that two stateful allocators are considered equal when they share the same memory resource, meaning, a memory allocated through one can be deallocated through another.

design decisions: allocators

The Allocator named requirements are specified such that:

- a stateful allocator is essentially a handle to a memory resource, from which memory is allocated*
- a stateful allocator does not represent the memory resource itself*

This is a consequence of the fact that two stateful allocators are considered equal when they share the same memory resource, meaning, a memory allocated through one can be deallocated through another.

On the surface, this design makes total sense – the whole idea behind stateful allocators is to share (reuse) the same memory resource across multiple containers.

design decisions: allocators

design decisions: allocators

Usually sharing by itself is enough to break value semantics.

design decisions: allocators

Usually sharing by itself is enough to break value semantics.

*However, this is not a problem for memory allocators,
because sharing the same memory resource does not imply sharing memory.*

design decisions: allocators

Usually sharing by itself is enough to break value semantics.

*However, this is not a problem for memory allocators,
because sharing the same memory resource does not imply sharing memory.*

*After all, a correctly implemented allocator
will never allocate overlapping memory buffers,
thus the problem of sharing memory is non-existent.*

design decisions: allocators

Usually sharing by itself is enough to break value semantics.

*However, this is not a problem for memory allocators,
because sharing the same memory resource does not imply sharing memory.*

*After all, a correctly implemented allocator
will never allocate overlapping memory buffers,
thus the problem of sharing memory is non-existent.*

There is, however, a problem with life-times!

design decisions: allocators

Usually sharing by itself is enough to break value semantics.

*However, this is not a problem for memory allocators,
because sharing the same memory resource does not imply sharing memory.*

*After all, a correctly implemented allocator
will never allocate overlapping memory buffers,
thus the problem of sharing memory is non-existent.*

There is, however, a problem with life-times!

*A stateful allocator can be destroyed prematurely, that is,
before the life-time of a container, referencing that stateful allocator, has ended.*

Thank you for listening!

JSC C++ Library